Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа
«Системное и прикладное программное обеспечение»

Курсовая работа
По дисциплине «Информационные системы»

Информационная система для поиска и возврата потерянных вещей «Потеряшки»

Этапы 1–4

Выполнил студент группы [номер группы]
Евграфов Артём Андреевич

Проверил:
[ФИО преподавателя]

Санкт-Петербург 2026

Содержание

Задание....3
1. Предметная область....3
2. Подробное описание предметной области....4
3. Назначение системы и решаемые задачи....11
4. Требования к информационной системе....12
5. Модели и описания основных прецедентов....17
6. Предлагаемая архитектура....36
Вывод....45
Список использованных источников....46
Приложение А. Термины....47
Приложение Б. План проверки требований....47
7. ER-модель базы данных....49
8. Даталогическая модель....55
9. Реализация в PostgreSQL....69
10. Триггеры и целостность бизнес-процессов....70
11. Скрипты создания, удаления и заполнения....73
12. Функции и процедуры PL/pgSQL....74
13. Индексы и обоснование....76
14. Проверка реализации и демонстрация....78
15. Вывод по второму этапу....80
Дополнительные источники к этапу 2....80
Приложение В. Соответствие заданию этапа 2....81
Задание третьего этапа....83
16. Архитектура реализации....83
17. Уровень хранения....87
18. Уровень бизнес-логики....90
19. Проверка и демонстрация на helios....94
20. Результаты третьего этапа....97
Источники третьего этапа....97
Задание четвёртого этапа....98

21. Уровень представления..... 98
22. Доступ и обработка данных.....111
23. Проверка веб-приложения.....113
24. Демонстрация и итоговые материалы..... 114
Источники четвёртого этапа..... 115

Задание

Первый этап курсовой работы включает:

1. Выбор предметной области информационной системы.

2. Подробное текстовое описание предметной области.

3. Обоснование назначения системы и решаемых задач.

4. Формулирование функциональных и нефункциональных требований.

5. Построение моделей основных прецедентов и описание их сценариев.

6. Выбор архитектуры, технологий и фреймворков с учётом демонстрации на helios.

7. Подготовку отчёта.

Второй этап курсовой работы включает:

1. ER-модель не менее чем с 10 сущностями и отношением «многие ко многим».

2. Построение даталогической модели на основе ER-модели.

3. Реализацию модели в PostgreSQL.

4. Обеспечение целостности средствами DDL и триггеров.

5. Скрипты создания, удаления и заполнения тестовыми данными.

6. Функции и процедуры PL/pgSQL для критически важных запросов.

7. Индексы по сценариям первого этапа и обоснование их полезности.

8. Подготовку отчёта.

1. Предметная область

Предметная область — поиск и возврат потерянных вещей в городской среде. Разрабатываемая информационная система «Потеряшки» предназначена для публикации объявлений о потерянных и найденных предметах с указанием места и времени события, поиска совпадений, проверки заявителей и фиксации возврата. Дополнительно предусматриваются справочная интеграция с Петербургским метрополитеном, доступ к пользовательским функциям по подписке и аукционы вещей, которыми продавец вправе распоряжаться.

В качестве начального региона выбран Санкт-Петербург. Приложение проектируется как адаптивное веб-приложение. Подписка предоставляет доступ к пользовательским операциям после верификации учётной записи. Регистрация, оплата, восстановление доступа и официальные инструкции доступны до активации подписки. Цена задаётся администратором; конкретная стоимость не является требованием этапа 1. Базовый расчётный период подписки — 30 суток без автоматического списания.

Для демонстрации оплаты используется тестовый платёжный адаптер. Проверка пользователя включает подтверждение электронной почты и рассмотрение профиля модератором. Паспортные данные при регистрации не запрашиваются. После безуспешного поиска владельца пользователь может подать заявку на аукцион; возможность продажи зависит от основания распоряжения вещью и решения по заявке.

2. Подробное описание предметной области

2.1. Участники и их интересы

Владелец потерянной вещи стремится быстро найти предмет и подтвердить своё право на его получение. Нашедший стремится сообщить о находке, передать её надлежащему получателю и избежать многократных необоснованных обращений. Один пользователь может выступать в обеих ролях в разных ситуациях. Покупатель участвует в аукционе; продавцом становится пользователь, право распоряжения вещью которого проверено.

Постоянные роли доступа — участник, модератор и администратор. Нашедший, владелец, продавец и покупатель обозначают положение участника в конкретном процессе. Модератор проверяет учётные записи, объявления, жалобы и документы для допуска аукционов. Администратор управляет полномочиями сотрудников, тарифами, справочником организаций и аудитом. Физическое хранение и выдачу вещей могут осуществлять пользователи либо сторонние организации: метрополитен, городское бюро находок, полиция. Эти организации не считаются зарегистрированными партнёрами без отдельной договорённости.

Платёжный сервис подтверждает результаты оплаты подписки. Пользователь обращается к нему через платёжную форму. Почтовый сервис

доставляет ссылки подтверждения и восстановления доступа. В самой системе уведомления сохраняются независимо от успешности доставки письма.

2.2. Существующий процесс и проблемы

Сведения о находках обычно распределены между объявлениями в социальных сетях, локальными чатами, бюро находок и транспортными организациями. Объявления оформляются неодинаково: часто отсутствуют время, точное место, категория или актуальный статус. Потерявшему приходится повторять поиск на нескольких площадках и связываться с авторами уже неактуальных сообщений.

Нашедший не всегда знает порядок передачи вещи и нужную организацию. Публикация полной фотографии документов или всех отличительных признаков увеличивает вероятность злоупотреблений. Несколько претендентов могут одновременно заявить права на один предмет. Отсутствие истории решений затрудняет разбор споров, а свободная публикация без проверки повышает долю рекламы, ложных находок и повторяющихся сообщений.

Подписка повышает стоимость массового злоупотребления и поддерживает работу сервиса, но сама по себе не гарантирует добросовестность. Поэтому в проекте она дополняется верификацией профиля, предварительной модерацией, ограничением частоты действий и механизмом жалоб. Сроки подписки и судьба уже опубликованной находки управляются раздельно.

2.3. Процесс работы с потерянными и найденными вещами

Пользователь регистрируется, подтверждает почту, проходит проверку профиля и активирует подписку. Затем он создаёт объявление типа «Потерял» или «Нашёл». Для находки фиксируются предмет, категория, описание, место, дата и время обнаружения, текущее место хранения. Для потери допускается временной интервал и приблизительный район, поскольку точный момент может быть неизвестен.

Часть признаков является публичной: категория, цвет, общее описание, район, дата, безопасная фотография. Контрольные признаки — например,

содержимое сумки или индивидуальная отметка — видны только автору и уполномоченному модератору. Контактные сведения и точный адрес хранения не публикуются в общем каталоге.

Объявление проходит модерацию. После публикации пользователи ищут предметы по типу объявления, категории, месту и времени. Система предлагает кандидатов на совпадение по структурированным полям. Совпадение не считается доказательством принадлежности: пользователь направляет заявку и описывает скрытые признаки. Автор находки рассматривает заявку, может запросить уточнения и согласовать передачу через внутренний диалог.

После согласования система резервирует находку за одним претендентом. Фактическая передача происходит вне приложения, затем обе стороны подтверждают её. До второго подтверждения вещь имеет статус «Передача ожидает подтверждения»; при споре дело передаётся модератору. Если предмет хранится у организации, система показывает её порядок выдачи; самостоятельное подтверждение в приложении не заменяет решения организации.

2.4. Находки в метрополитене

Для места типа «Метро» указываются город, станция и при наличии линия или участок поездки. В карточке показываются официальные инструкции: справочная Петербургского метрополитена 8-800-350-11-55; вещи передаются дежурному по станции и находятся в метро 10 дней, затем передаются в городской Центр утерянных документов и средств связи по адресу: ул. Большая Монетная, д. 16, офис-центр № 1. Для получения вещи требуется паспорт и заявление. Эти сведения взяты с официальной страницы метрополитена [1].

Найденную в метро вещь пользователь передаёт сотруднику станции и может опубликовать информационное объявление с указанием организации хранения. Приложение предлагает уточнить актуальные условия по официальному контакту. Десятидневный срок не используется как срок допуска к аукциону. Данные справочника содержат ссылку на источник, дату проверки и дату следующего пересмотра; администратор проверяет их не реже одного раза в 30 дней.

Интеграция с метрополитеном реализуется через справочник организаций: приложение подбирает контакты и инструкцию по выбранному месту. Пользователь обращается в организацию по указанному телефону или через официальный сайт. Прямой обмен сведениями о находках может быть добавлен при наличии доступного программного интерфейса и договорённости с метрополитеном.

2.5. Условия аукциона

Обычная находка не становится товаром только потому, что на объявление долго нет откликов. Статья 227 ГК РФ предусматривает уведомление известного владельца; находку в помещении или транспорте передают представителю владельца помещения или транспорта. Когда надлежащий получатель неизвестен, предусмотрено заявление в полицию либо орган местного самоуправления [3]. Публикация в «Потеряшках» не заменяет такого заявления.

Для обычной находки статья 228 ГК РФ связывает приобретение собственности с отсутствием установленного или заявившегося правообладателя в течение шести месяцев после заявления о находке в компетентный орган. При отказе нашедшего приобрести вещь она поступает в муниципальную собственность [4]. Поэтому проект использует дату официального заявления и проверку подтверждающих материалов, а не дату публикации объявления.

Продавец подаёт заявку на допуск вещи к аукциону, указывает основание распоряжения и прикладывает подтверждение обращения и права. Модератор проверяет комплектность, отсутствие открытого спора и принадлежность допустимой категории; сомнительное основание ведёт к отказу. Автоматически подтверждается только комплектность данных, решение о допуске принимает модератор. Вещи, переданные метро или иной организации, обычный пользователь на аукцион не выставляет.

Из аукционного каталога исключаются документы, банковские карты, предметы с персональными данными, опасные и запрещённые предметы, а также вещи с неразрешёнными заявлениями владельцев. Для электронных устройств требуется подтверждение законного распоряжения и удаления

пользовательских данных. Частные случаи продажи скоропортящихся вещей не включены в учебный процесс.

Аукцион проводится на повышение: задаются стартовая цена, шаг и время начала и окончания. Продавец не может делать ставки на собственный лот. Победителем становится автор наибольшей принятой ставки до срока окончания. Отсутствие ставок означает завершение без победителя. Расчёты за лот и доставку стороны организуют самостоятельно; система фиксирует договорённость и подтверждение передачи.

2.6. Основные информационные объекты

Объект	Основные сведения	Назначение
Пользователь	ID, имя, email, хеш пароля, набор ролей, статус проверки и блокировки, версия токенов	Учёт участника и управление доступом
Токен обновления	Пользователь, хеш токена, срок действия, отметка отзыва	Обновление доступа и завершение авторизации
Верификация	Пользователь, дата подачи, решение, причина, модератор	История рассмотрения профиля
Подписка	Пользователь, начало и окончание, статус, оплаченный тариф	Проверка права доступа
Платёж	ID операции, сумма, статус, внешний идентификатор	Связь оплаты с подпиской
Объявление	Тип, автор, категория, описание, время, место, статус	Учёт потери или находки
Фотография	Объявление, ключ файла, MIME, размер, доступность	Безопасная иллюстрация предмета
Контрольный признак	Объявление, закрытое описание	Проверка претендента
Место	Город, тип, район/адрес, станция, координаты при наличии	Поиск и подбор организации
Организация	Название, контакты, инструкция, URL источника, даты проверки	Справочная интеграция

Объект	Основные сведения	Назначение
Заявка на возврат	Находка, претендент, доказательства, статус	Рассмотрение права на получение
Диалог и сообщение	Участники, заявка, текст, время	Согласование возврата
Передача	Находка или лот, стороны, подтверждения, дата	Завершение жизненного цикла
Жалоба	Автор, объект, причина, решение	Контроль качества и спорные ситуации
Допуск к аукциону	Находка, основание, документы, решение	Контроль возможности продажи
Аукцион и ставка	Лот, продавец, цены, сроки, участник, серверное время	Организация торгов
Уведомление и аудит	Получатель/исполнитель, событие, время, результат	Информирование и прослеживаемость

Это концептуальный состав данных. Логическая и физическая схемы PostgreSQL разрабатываются на следующем этапе.

2.7. Статусы и бизнес-правила

Объявление проходит состояния «Черновик → На модерации → Опубликовано». Из модерации возможны «Отклонено» или «На исправлении»; изменённое объявление проходит повторную проверку. Опубликованная находка может перейти в «Зарезервировано → Передача ожидает подтверждения → Возвращено». Отмена согласования возвращает её в опубликованное состояние. Автор может закрыть поиск; модератор — скрыть нарушающее правила объявление. Закрытая находка сохраняется в истории и не участвует в выдаче активных результатов.

У аукциона состояния «Черновик → На проверке → Допущен → Запланирован → Активен → Завершён с победителем / Завершён без ставок». Дополнительно предусмотрены «Отказано», «Приостановлен», «Отменён», «Передан покупателю». Неурегулированная претензия блокирует запуск и новые ставки; модератор определяет дальнейшее решение, оно сохраняется в аудите.

• BR-01. Публикация, поиск каталога, заявки, сообщения и ставки доступны проверенному, незаблокированному пользователю с активной подпиской. Сотрудники работают по служебным ролям без платной подписки.

• BR-02. До подписки разрешены регистрация, подтверждение почты, проверка профиля, покупка или продление подписки, восстановление доступа, просмотр собственных статусов и официальных контактов.

• BR-03. После истечения подписки новые пользовательские операции закрываются; ранее опубликованные объявления остаются видимыми. Участник может прочитать существующий диалог, подтвердить уже согласованную передачу, закрыть своё объявление и направить жалобу по своему делу без продления.

• BR-04. Проверка профиля не подтверждает принадлежность конкретной вещи. Каждая заявка на возврат рассматривается отдельно. Оплата не отменяет модерацию.

• BR-05. Для находки обязательны место и дата/время. Для потери допускается интервал; его начало не позже конца. Время события не может находиться в будущем. Ввод и отображение выполняются в часовом поясе Санкт-Петербурга Europe/Moscow; хранение и сравнение — как момент времени UTC.

• BR-06. Для метро обязательна станция либо описание участка поездки. Приложение не обещает наличие вещи в реестре организации.

• BR-07. У находки может быть несколько заявок, но не более одного действующего резервирования. Автор не направляет заявку на собственную находку.

• BR-08. Возврат закрывается после двух подтверждений или мотивированного решения модератора. Скрытые признаки и материалы спора не входят в публичный каталог.

• BR-09. Продление оплаченной подписки добавляет 30 суток от более позднего значения: текущего момента или окончания действующей подписки. Повторное платёжное событие не добавляет срок повторно.

• BR-10. Блокировка запрещает действия независимо от подписки. Принятые до истечения подписки ставки сохраняются; блокировка участника активного аукциона приостанавливает его до решения модератора.

• BR-11. Аукцион возможен только после отдельного допуска. Отсчёт шести месяцев по обычной находке ведётся от официального заявления; истечение срока лишь позволяет подать заявку на проверку и не создаёт допуск автоматически.

• BR-12. Ставка принимается при активном аукционе и серверном времени строго раньше окончания. Минимальная ставка — стартовая цена при отсутствии ставок, иначе последняя принятая цена плюс шаг. Денежные суммы хранятся точно, без float.

• BR-13. Одновременные ставки обрабатываются последовательно в транзакции для одного лота. При одинаковом предложении принимается только первая допустимая ставка; второй запрос получает актуальную минимальную цену.

• BR-14. Уведомление о похожем объявлении — рекомендация для проверки. Передача вещи и возникновение права собственности не следуют из алгоритма поиска.

• BR-15. Одна находка не может одновременно участвовать в возврате и продаже. После допуска и планирования аукциона обычное резервирование блокируется, а новые заявления владельцев поступают модератору и приостанавливают торги. На одну находку допускается только один незавершённый аукцион. Возврат возможен после отмены продажи и рассмотрения заявления.

3. Назначение системы и решаемые задачи

Цель системы — создать единое управляемое пространство поиска и возврата вещей, в котором сведения структурированы, участники проверены, а действия прослеживаются. Система должна сокращать количество площадок, которые просматривает пользователь, и помогать выбирать правильный маршрут обращения.

Система решает следующие задачи:

1. Собирать объявления о потерях и находках в едином каталоге со стандартными полями места, времени и предмета.

2. Давать поиск и подбор потенциально связанных объявлений без раскрытия контрольных признаков.

3. Обеспечивать доступ по подписке, проверку профилей, модерацию публикаций и рассмотрение жалоб.

4. Сопровождать заявку на возврат, общение сторон, резервирование и подтверждение фактической передачи.

5. Показывать проверенные контакты и порядок действий для находок в метрополитене.

6. Поддерживать контролируемый аукцион вещей с подтверждённым основанием распоряжения.

7. Сохранять историю изменений, платежей, ставок и решений для разрешения спорных ситуаций.

Результативность системы оценивается по числу подтверждённых возвратов, времени от публикации до первого обоснованного отклика, доле отклонённых нарушающих правила объявлений и сроку обработки модерации. Технические критерии приёмки определены в разделе 4.

4. Требования к информационной системе

4.1. Функциональные требования

Функциональные требования определяют действия системы и наблюдаемый результат для участника или сотрудника. Требования к скорости, защите, надёжности и условиям запуска выделены в раздел 4.2. Обозначение «Обязательно» означает включение в учебную реализацию; «Дополнительно» — развитие после основного набора. Внешняя оплата подписки заменяется учебным адаптером, прямой API организаций в демонстрацию не входит.

ID	Требование	Проверяемый результат	Приоритет
FR-01	Регистрация по уникальному email, подтверждение почты, вход,	Неподтверждённая почта не даёт пройти проверку; ссылка одноразовая; после выхода refresh-токен отозван,	Обязательно

ID	Требование	Проверяемый результат	Приоритет
	выход и восстановление доступа	при смене пароля прежние токены аннулированы	
FR-02	Проверка профиля модератором с причиной решения	Пока проверка не одобрена, покупка подписки и функции каталога закрыты	Обязательно
FR-03	Покупка и продление подписки; учёт успешных, неуспешных и повторных событий	Только подтверждённая оплата меняет срок; повторный callback не дублирует продление	Обязательно
FR-04	Разграничение действий участника, модератора и администратора с учётом профиля, подписки и принадлежности дела	Чужой диалог и служебное действие недоступны; собственный кабинет до оплаты и завершение согласованной передачи после её истечения доступны по BR-02/03	Обязательно
FR-05	Создание и редактирование объявлений «Потерял» и «Нашёл»	Без категории, описания, места и времени отправка на модерацию невозможна	Обязательно
FR-06	Загрузка до пяти JPEG/PNG, каждая до 5 МБ, и запись закрытых признаков	Неверный формат отклонён; закрытые поля отсутствуют в ответах каталога	Обязательно
FR-07	Предварительная и повторная модерация объявлений	Черновик, отклонённое и непроверенное изменение не отображаются в каталоге	Обязательно
FR-08	Поиск потерь и находок по тексту, категории, месту, времени, типу и публичному статусу	Фильтры применяются совместно; до 20 карточек на странице; переход сохраняет условия, черновики и скрытые материалы не выдаются	Обязательно
FR-09	Подбор возможных совпадений потери и находки	Показываются до 10 кандидатов и причины подбора; принадлежность не подтверждается автоматически	Дополнительно
FR-10	Справочник метро, контакты и инструкции по выбранному месту	Для Петербургского метро показаны официальный контакт, ссылка и дата проверки; справочник доступен без подписки	Обязательно
FR-11	Заявки на возврат, скрытые доказательства, запрос уточнений, решение автора	Нельзя заявиться на своё объявление; повторная активная заявка того же пользователя отклоняется	Обязательно
FR-12	Диалог сторон в рамках заявки	Чтение и отправка доступны только сторонам и модератору при разборе дела; общие контакты не раскрываются	Обязательно

ID	Требование	Проверяемый результат	Приоритет
FR-13	Согласование передачи одной вещи одному получателю и фиксация подтверждений сторон	Создано одно действующее дело; первое подтверждение оставляет передачу открытой, второе завершает её; после первого подтверждения отмена направляется на разбор	Обязательно
FR-14	Внутренние уведомления о решениях, сообщениях, окончании подписки и торгов	Событие доступно в кабинете; письмо не требуется для сохранения события	Обязательно
FR-15	Жалобы на объявления, действия и спорные передачи	Жалоба получает статус и мотивированное решение; автор видит ход своего дела	Обязательно
FR-16	Рассмотрение основания распоряжения вещью и допуск аукциона	Без проверенных материалов, при открытом споре или действующем возврате запуск невозможен	Обязательно
FR-17	Создание лота с ценой, шагом, сроками; приём ставок и определение победителя	Самостоятельные ставки продавца, просроченные и недостаточные ставки отклоняются	Обязательно
FR-18	Приостановка/отмена аукциона, завершение без ставок, подтверждение передачи покупателю	История ставок сохраняется; приостановленный лот не принимает ставки	Обязательно
FR-19	Назначение служебных полномочий, изменение тарифов и инструкций организаций администратором	Назначаются только предусмотренные роли; последний администратор защищён; новая цена действует для новых заказов; инструкция содержит источник и дату проверки	Обязательно
FR-20	Журнал решений и существенных изменений	Запись содержит исполнителя, время, объект, действие и результат; обычный пользователь не изменяет аудит	Обязательно

4.2. Нефункциональные требования

Нефункциональные требования задают измеримые условия работы приложения. Нагрузочные показатели проверяются на helios с одной JVM и PostgreSQL, после прогрева и с учебными адаптерами оплаты и почты. Значения NFR-01/02 и RPO/RTO являются проектными целями; результаты измерений и восстановления фиксируются отдельно после реализации.

Поведение операций определяется функциональными требованиями, а устойчивость при одновременных запросах — NFR-03.

ID	Требование	Критерий проверки
NFR-01	Производительность основного поиска	При 10 000 объявлениях и 20 одновременных пользователях 95% запросов каталога завершаются за ≤ 2 с; измерение 5 минут после прогрева
NFR-02	Скорость фиксации операций	95% запросов публикации заявки и ставки выполняются за ≤ 2 с при той же нагрузке, без учёта загрузки файлов и внешних сервисов
NFR-03	Целостность конкурентных действий	Два соединения одновременно резервируют одну находку: принят один запрос. Две одинаковые ставки: принята одна допустимая ставка. Повтор оплаты создаёт один период; завершение торгов не допускает ставку после срока
NFR-04	Контроль доступа и защита токенов	Spring Security проверяет JWT, роль и состояние пользователя для каждой операции; пароли хешируются BCrypt; refresh-токен передаётся в Cookie HttpOnly и SameSite, в HTTPS-режиме Secure; операции с Cookie защищаются от CSRF
NFR-05	Приватность	Контрольные признаки, личные контакты и документы допуска не доступны чужим пользователям через страницы, файловые URL или ответы контроллеров
NFR-06	Безопасная обработка ввода	HTML пользователя экранируется; доступ к БД параметризован; формат, размер и содержимое изображения проверяются; EXIF удаляется; SVG и исполняемые файлы не принимаются
NFR-07	Ограничение злоупотреблений	Не более 5 ошибочных попыток входа на учётную запись за 15 минут; не более 10 новых объявлений в сутки и 30 сообщений в минуту на пользователя; превышение даёт контролируемый отказ
NFR-08	Надёжность оплаты и уведомлений	Повтор успешного события не добавляет срок подписки второй раз; сбой доставки не откатывает бизнес-операцию; повтор обработки одного события не создаёт второе внутреннее уведомление
NFR-09	Восстановление после сбоя	Для учебной установки ежедневная копия БД и вложений, хранение 7 суток; целевые RPO ≤ 24 ч, RTO ≤ 2 ч; проверяется пробным восстановлением

ID	Требование	Критерий проверки
NFR-10	Удобство использования	Формы и сообщения на русском; ошибка ввода сохраняет корректные поля и указывает причину; подготовка черновика с одной фотографией занимает до 5 минут без учёта ожидания модерации
NFR-11	Совместимость интерфейса	Формы и каталог работают при ширине 360–1920 px; проверяются актуальные стабильные Chrome, Firefox и Edge на момент приёмки
NFR-12	Переносимость	Сборка через Gradle Wrapper на JDK 17 или новее; JAR со всеми шаблонами запускается на helios обычным пользователем; адрес PostgreSQL, порт и каталог файлов задаются конфигурацией
NFR-13	Наблюдаемость	Логи содержат ID запроса и ошибки; в них нет паролей, платёжных секретов, контрольных признаков и полных доказательств; аудит доступен только служебным ролям
NFR-14	Проверяемость и сопровождение	Предметные сервисы отделены от контроллеров; SQL-изменения версионируются в Git и применяются явно; учебные адаптеры отделены; интеграционные проверки используют временные объекты PostgreSQL
NFR-15	Актуальность справочных данных	У каждой инструкции есть официальный URL и дата проверки; после 30 суток появляется признак необходимости обновления; недоступность внешнего сайта не ломает карточку
NFR-16	Изоляция и хранение данных	Учебные пользователи и платежи синтетические; временные проверки не изменяют исходные объекты и последовательности общей базы; вложения хранятся в отдельном закрытом каталоге; сроки хранения реальных данных задаются до эксплуатации

4.3. Границы реализации

В обязательный объём входят аккаунты, модерация, учебная оплата подписки, каталог, заявки, внутренние сообщения, возврат, справочник метро, контролируемые аукционы, уведомления и аудит. Нативные мобильные приложения, распознавание вещей по фотографии, геокодирование внешними сервисами, курьерская доставка, денежные расчёты за лоты и интеграция

с реестрами метро не включены. Карта и отдельный процесс удаления и обезличивания профиля относятся к дальнейшему развитию. В базовой версии место выбирается из справочника и описывается текстом.

5. Модели и описания основных прецедентов

5.1. Роли и границы прецедентов

В системе выделены три роли доступа. Роль определяет набор разрешённых операций; состояние профиля и срок подписки проверяются отдельно.

Роль	Ответственность
Участник	Получает доступ к сервису, публикует объявления, ищет вещи, согласовывает передачу и участвует в торгах. До проверки профиля и оплаты доступны регистрация, собственный кабинет и справочная информация.
Модератор	Проверяет профили, объявления и основания продажи, рассматривает жалобы и принимает решения по спорным передачам. Не рассматривает собственные материалы.
Администратор	Назначает служебные полномочия, меняет тарифы и справочник организаций, контролирует историю служебных действий. Может выполнять операции модератора.

Владелец вещи, нашедший, продавец и покупатель — обозначения участников конкретного процесса. Отдельные роли доступа для них не создаются. В UC-03 взаимодействуют две учётные записи участников: автор находки и её предполагаемый владелец. В UC-05 продавец и покупатели также имеют роль «Участник», но выполняют разные действия в одном процессе.

Почтовая и платёжная службы являются внешними техническими актёрами, а не ролями пользователей. Метрополитен служит источником справочных сведений: в выбранном варианте интеграции его сотрудники не работают в приложении. Планировщик является внутренним компонентом системы.

Каждый прецедент описывает одну цель и законченный результат. Регистрация, проверка профиля и первая оплата составляют процесс получения доступа; подготовка и модерация объявления — процесс его публикации; поиск, заявка, диалог и два подтверждения — процесс возврата.

Нажатия кнопок, проверка подписки и сохранение записи являются шагами этих процессов. Вход служит предусловием защищённых действий.

ID	Цель прецедента	Основные участники	Результат успешного завершения
UC-01	Получить доступ к сервису	Участник, модератор; почтовая и платёжная службы	Проверенный участник имеет действующую подписку
UC-02	Опубликовать объявление	Участник — автор, модератор	Проверенное объявление доступно в каталоге
UC-03	Вернуть найденную вещь владельцу	Два участника: нашедший и владелец	Передача подтверждена обеими сторонами
UC-04	Разрешить спор по находке	Заявитель, другая сторона, модератор	Сохранены решение и допустимое состояние дела
UC-05	Продать вещь на аукционе	Продавец, покупатели, модератор	Победитель определён, передача подтверждена
UC-06	Изменить полномочия сотрудника	Администратор, участник — сотрудник	Служебный доступ соответствует назначенным ролям
UC-07	Актуализировать инструкцию организации	Администратор, участник — читатель	Пользователь получает обновлённые контакты и инструкцию
UC-08	Изменить условия новых подписок	Администратор, участник — плательщик	Новые заказы используют новый тариф; старые

ID	Цель прецедента	Основные участники	Результат успешного завершения
			условия сохранены

Сплошная линия UML связывает актёра с прецедентом. Обобщение администратора и модератора показывает наследование служебных возможностей. Один актёр «Участник» может представлять несколько разных учётных записей внутри сценария. Последовательность их действий показана в описаниях и на диаграмме взаимодействия возврата.



Рисунок 1 — Прецедент получения доступа к сервису

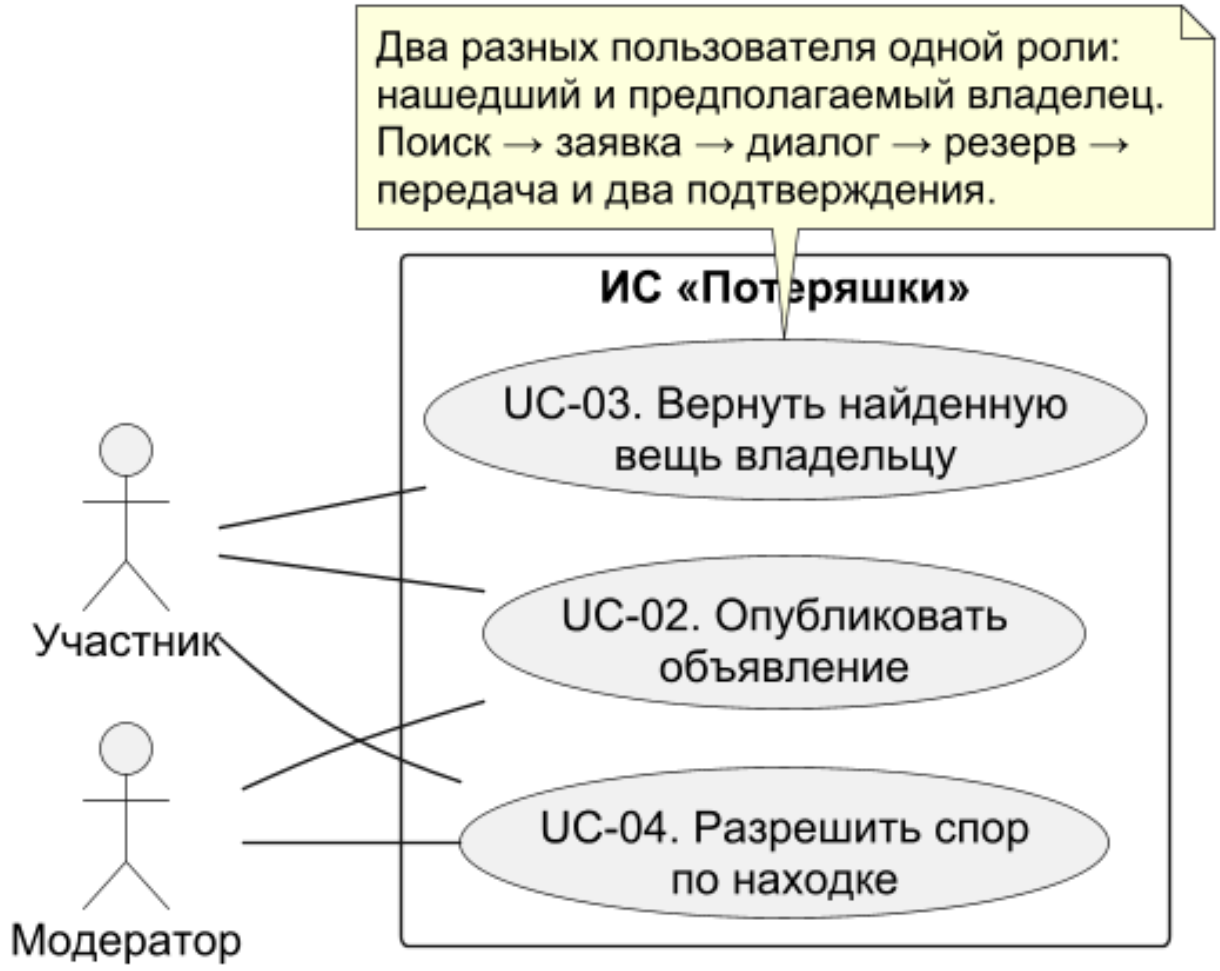


Рисунок 2 — Прецеденты публикации, возврата и разрешения спора

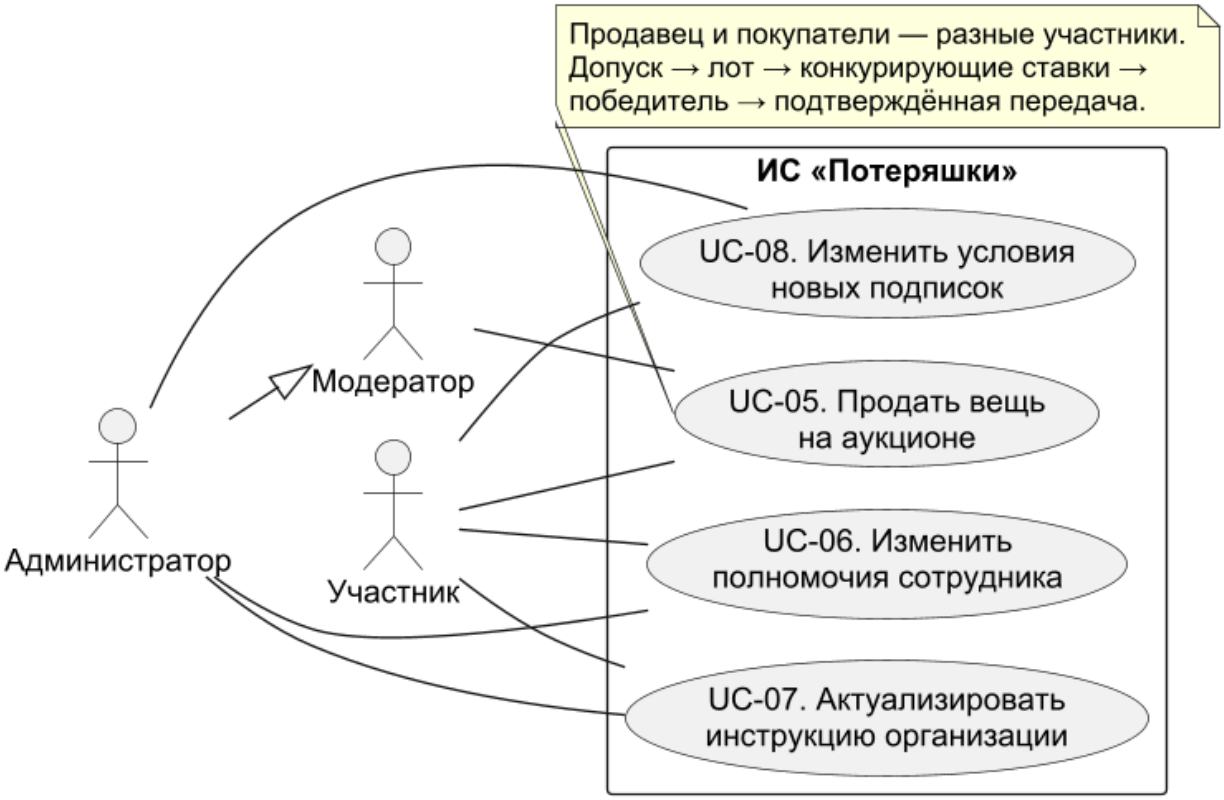


Рисунок 3 — Прецеденты продажи и администрирования

5.2. UC-01. Получить доступ к сервису

Поле	Описание
Цель	Получить проверенную учётную запись и оплаченный доступ к каталогу и пользовательским операциям.
Инициатор	Участник, впервые обращающийся к сервису.
Другие участники	Модератор, почтовая служба, платёжная служба.
Предусловия	Регистрация открыта; существует действующий тариф. Модератор имеет служебный доступ.
Триггер	Участник начинает регистрацию.
Постусловия	Почта подтверждена, профиль одобрен, подписка активна. Сохранены решения, заказ и результат оплаты.
Требования	FR-01–04, FR-14, FR-20; NFR-04, NFR-07, NFR-08.

Шаг	Исполнитель	Действие и реакция системы
1	Участник	Вводит имя, email и пароль. Система проверяет поля и уникальность адреса.

Шаг	Исполнитель	Действие и реакция системы
2	Система, почтовая служба	Создаёт неподтверждённую запись, сохраняет хеш пароля и отправляет одноразовую ссылку.
3	Участник	Открывает ссылку. Система проверяет срок и отмечает подтверждение почты.
4	Участник	Подаёт профиль на проверку и видит состояние «Ожидает рассмотрения».
5	Модератор	Открывает очередь, изучает данные профиля и принимает мотивированное решение.
6	Система	Сохраняет одобрение и уведомляет участника. До одобрения доступ к каталогу остаётся закрытым.
7	Участник	Выбирает тариф. Система создаёт заказ с зафиксированной ценой и длительностью.
8	Участник, платёжная служба	Выполняют оплату. В учебном режиме результат формирует тестовый адаптер без списания средств.
9	Система	Проверяет заказ и событие оплаты, активирует подписку одной транзакцией и сохраняет уведомление.
10	Участник	Открывает каталог. Система проверяет текущие роли, профиль, блокировку и оплаченный период и предоставляет доступ.

Альтернативные и ошибочные потоки:

• А1. Адрес уже зарегистрирован: участник входит в существующую запись; при утрате пароля использует одноразовую ссылку восстановления. Проверка профиля повторно не требуется, если ранее одобренный профиль сохраняет действующий статус.

• А2. Ссылка просрочена либо использована: подтверждение отклоняется, участник запрашивает новую ссылку. Ошибка почтовой доставки сохраняет неподтверждённую запись.

• A3. Модератор отклоняет профиль: участник получает причину, исправляет данные и возвращается к шагу 4. Оплата и каталог до одобрения недоступны.

• A4. Оплата отменена или отклонена: заказ сохраняет результат, оплаченный доступ не открывается. Повтор успешного события не добавляет срок второй раз.

• A5. У проверенного участника закончилась подписка: он выполняет шаги 7–10; продление начинается от более позднего из текущего момента и конца действующего периода.

5.3. UC-02. Опубликовать объявление

Поле	Описание
Цель	Разместить в каталоге проверенное сообщение о потере или находке.
Инициатор	Участник — автор объявления.
Другие участники	Независимый модератор; другие участники просматривают опубликованный результат.
Предусловия	Автор вошёл, профиль одобрен, подписка активна; аккаунт не заблокирован.
Триггер	Автор создаёт новое объявление.
Постусловия	Объявление опубликовано; закрытые признаки и документы отсутствуют в публичной карточке. Решение модератора сохранено.
Требования	FR-04–07, FR-14, FR-20; NFR-05–07, NFR-10.

Шаг	Исполнитель	Действие и реакция системы
1	Автор	Выбирает тип «Потерял» или «Нашёл» и категорию предмета.
2	Автор	Указывает место, дату и время; для потери может задать интервал. Система проверяет заполнение и отсутствие будущего времени.
3	Автор	Вводит публичное описание и отдельно закрытые приметы. Для вещи у организации указывает хранителя.

Шаг	Исполнитель	Действие и реакция системы
4	Система	Сохраняет черновик, доступный автору.
5	Автор	Добавляет фотографии. Система проверяет формат, размер и содержимое, удаляет метаданные и связывает файлы с черновиком.
6	Автор	Отправляет объявление на проверку. Система помещает его в очередь модерации.
7	Модератор	Проверяет описание, место, время и изображения; исключает персональные данные, рекламу и сомнительные материалы.
8	Модератор	Одобряет публикацию. Система сохраняет исполнителя, дату и результат рассмотрения.
9	Система	Переводит объявление в опубликованное состояние и уведомляет автора.
10	Другой участник	Находит объявление в каталоге и получает публичную карточку без закрытых признаков и точного адреса хранения.

Альтернативные и ошибочные потоки:

• А1. Поля или изображение не проходят проверку: черновик сохраняется, автор исправляет указанную ошибку.

• А2. Модератор отклоняет объявление с причиной: оно не появляется в каталоге; после правки автор повторяет шаги 6–9.

• А3. Автор существенно изменяет опубликованное объявление: оно проходит повторную модерацию. Изменение вещи, уже участвующей в передаче или торгах, ограничено правилами её состояния.

• А4. Модератор является автором материала: система запрещает рассмотрение, материал проверяет другой сотрудник.

5.4. UC-03. Вернуть найденную вещь владельцу

Поле	Описание
Цель	Сопоставить находку с обращением владельца и зафиксировать фактическую передачу вещи.
Инициатор	Участник — предполагаемый владелец.
Другие участники	Участник — нашедший; при споре модератор участвует в UC-04.
Предусловия	Находка опубликована; претендент и автор — разные пользователи. Для поиска и новой заявки нужна действующая подписка.
Триггер	Владелец начинает поиск потерянного предмета.
Постусловия	Заявка исполнена, обе стороны подтвердили передачу, находка имеет статус «Возвращено».
Требования	FR-04, FR-08–14, FR-20; NFR-03–06, NFR-10.

Шаг	Исполнитель	Действие и реакция системы
1	Владелец	Задаёт текст, категорию, место, время и публичный статус. Система выдаёт до 20 карточек и сохраняет условия перехода к следующей странице.
2	Владелец	Открывает подходящую находку и сравнивает публичные сведения с обстоятельствами потери.
3	Владелец	Подаёт заявку и описывает непубличные приметы. Система проверяет автора заявки и отсутствие повторной активной заявки.
4	Система	Создаёт дело и личный диалог, сохраняет доказательства и уведомляет нашедшего.
5	Нашедший	Изучает заявку, сопоставляет её с закрытыми признаками и задаёт уточняющий вопрос в диалоге.
6	Владелец	Отвечает. Система сохраняет сообщение и уведомляет другую сторону без раскрытия его текста в общем каталоге.
7	Нашедший	Согласовывает передачу этому претенденту. Система блокирует

Шаг	Исполнитель	Действие и реакция системы
		находку, повторно проверяет доступность и создаёт единственный резерв.
8	Владелец и нашедший	Договариваются в диалоге о времени и способе встречи.
9	Владелец и нашедший	Передают вещь при встрече. Физическая передача происходит вне приложения.
10	Нашедший	Подтверждает передачу. Система сохраняет первое подтверждение; дело остаётся незавершённым.
11	Владелец	Подтверждает получение. Система сохраняет второе подтверждение и завершает заявку и передачу одной транзакцией.
12	Обе стороны	Видят завершённое дело и уведомления; находка исключена из выдачи активных объявлений.

Альтернативные и ошибочные потоки:

• A1. Совпадений нет: владелец меняет условия либо публикует сообщение о потере через UC-02. Автоматический подбор кандидатов является дополнительной функцией поиска.

• A2. Доказательства не совпадают: нашедший отклоняет заявку с объяснением либо продолжает уточнение. Вещь не резервируется автоматически по совпадению описания.

• A3. Другой претендент уже получил резерв: согласование отклоняется, сохраняется одно принятое дело.

• A4. До первого подтверждения встреча отменена: обычная заявка закрывается, резерв снимается. После подтверждения одной стороной разногласие передаётся на UC-04.

• A5. Подписка закончилась после согласования: чтение своего диалога, подтверждение передачи и жалоба по существующему делу доступны без новой оплаты.

• А6. Вещь находится в метро или у другой организации: система показывает контакт, источник, дату проверки и порядок обращения. Владелец связывается с хранителем вне приложения; выдачу определяет организация. Личное резервирование у автора информационного объявления не создаётся, а подтверждение в приложении не заменяет решение хранителя.

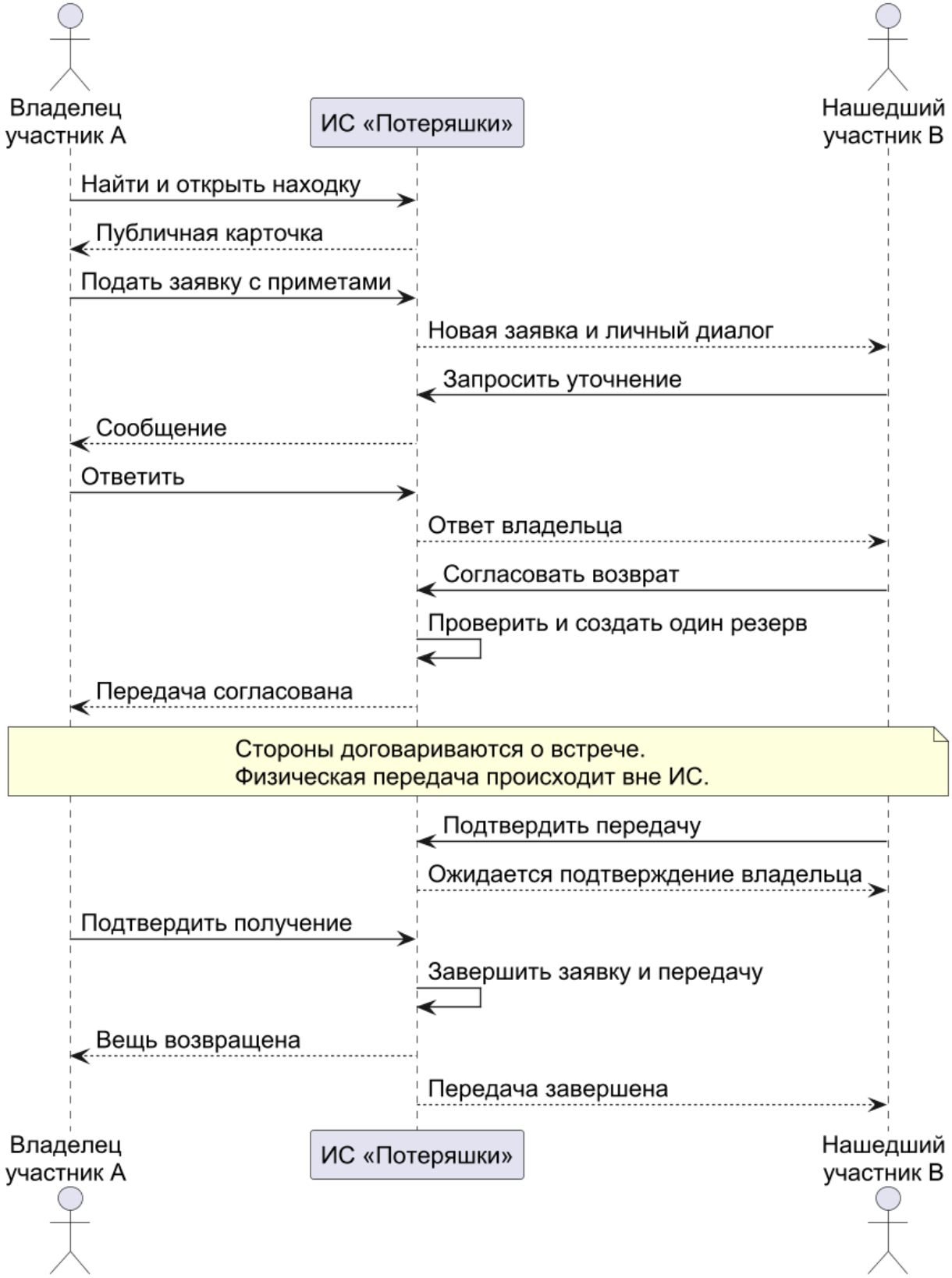


Рисунок 2.1 — Взаимодействие двух участников при возврате вещи

5.5. UC-04. Разрешить спор по находке

Поле	Описание
Цель	Получить мотивированное решение по жалобе и согласовать дальнейшее состояние связанного дела.
Инициатор	Участник — заявитель жалобы.
Другие участники	Участник — другая сторона, независимый модератор.
Предусловия	Есть объявление, пользователь либо дело передачи, на которое направляется жалоба. По своему существующему делу жалоба доступна при истёкшей подписке.
Триггер	Заявитель обнаруживает нарушение или разногласие о передаче.
Постусловия	Жалоба закрыта с причиной решения; допустимые изменения дела и действия сотрудника сохранены в аудите.
Требования	FR-04, FR-12–15, FR-18, FR-20; NFR-04, NFR-05, NFR-13.

Шаг	Исполнитель	Действие и реакция системы
1	Заявитель	Выбирает объект жалобы, описывает обстоятельства и отправляет обращение.
2	Система	Проверяет полномочия и объект, сохраняет жалобу со статусом ожидания. При открытом споре о лоте приостанавливает связанные торги.
3	Модератор	Открывает очередь и выбирает дело, в котором не является заинтересованной стороной.
4	Модератор	Читает материалы и связанный диалог при его наличии; доступ к чужому диалогу разрешён только на время открытого разбора.
5	Модератор	При необходимости запрашивает уточнение обстоятельств в доступном диалоге.
6	Заявитель и другая сторона	Представляют объяснения. Система сохраняет сообщения и историю дела.

Шаг	Исполнитель	Действие и реакция системы
7	Модератор	Оценивает материалы и выбирает решение с обязательным обоснованием.
8	Система	Применяет разрешённое действие: закрывает жалобу; при необходимости скрывает нарушение, блокирует доступ или завершает спорную передачу по служебному решению.
9	Система	Записывает решение в аудит и уведомляет заявителя. После закрытия жалобы служебный доступ к чужому диалогу прекращается.
10	Заявитель и другая сторона	Проверяют решение и состояние связанного объявления или передачи в своих кабинетах.

Альтернативные и ошибочные потоки:

• A1. Материалов недостаточно: модератор запрашивает уточнения, дело остаётся открытым.

• A2. Нарушение не подтверждено: жалоба отклоняется с причиной, необоснованная блокировка не применяется.

• A3. Проверяющий участвует в споре: рассмотрение передаётся независимому сотруднику.

• A4. Вопрос требует компетентной организации: стороны получают официальный маршрут обращения. Система фиксирует информационное решение и не устанавливает право собственности.

• A5. Предлагаемое изменение противоречит состоянию передачи или торгов: система отклоняет его; история подтверждений и ставок сохраняется.

5.6. UC-05. Продать вещь на аукционе

Поле	Описание
Цель	Продать допустимую вещь с проверенным основанием распоряжения и зафиксировать передачу победителю.
Инициатор	Участник — продавец.

Поле	Описание
Другие участники	Модератор и несколько участников — покупателей; один из них становится победителем.
Предусловия	Продавец допущен к сервису; покупатель имеет действующий доступ на момент ставки. Вещь относится к объявлению продавца, опубликована, не хранится у организации и не участвует в согласованном возврате. Основание распоряжения может быть подтверждено.
Триггер	Продавец подаёт материалы на допуск к продаже.
Постусловия	Торги завершены, победитель определён, создано единственное дело передачи; продавец и победитель подтвердили её завершение.
Требования	FR-04, FR-12–18, FR-20; NFR-03–06, NFR-08.

Шаг	Исполнитель	Действие и реакция системы
1	Продавец	Указывает основание продажи и прикладывает подтверждающие материалы по правилам BR-11.
2	Система	Проверяет комплектность, допустимую категорию и отсутствие конфликтующего процесса, создаёт заявку на допуск.
3	Модератор	Проверяет материалы, даты, отсутствие заявлений владельца и принимает мотивированное решение о допуске.
4	Система	Сохраняет разрешение, сотрудника и дату решения и уведомляет продавца.
5	Продавец	Задаёт стартовую цену, шаг и сроки. Система связывает аукцион с разрешением именно на эту вещь.
6	Система	Открывает торги в установленное время. Покупатели видят условия лота и текущую цену.
7	Первый покупатель	Отправляет ставку. Система проверяет доступ, время и цену и принимает её в транзакции.
8	Другой покупатель	Повышает предложение. Система последовательно проверяет

Шаг	Исполнитель	Действие и реакция системы
		конкурирующие ставки одного лота и обновляет текущую цену.
9	Система	По окончании срока закрывает торги и фиксирует наибольшую принятую ставку и победителя.
10	Система	Создаёт для продавца и победителя одну заявку, диалог и передачу, уведомляет участников.
11	Продавец и победитель	Согласовывают встречу и расчёт в диалоге. Расчёт за сам лот и физическая передача происходят вне приложения.
12	Продавец	Подтверждает передачу вещи. Дело ожидает подтверждения победителя.
13	Победитель	Подтверждает получение. Система завершает передачу и сохраняет итог торгов и историю ставок.

Альтернативные и ошибочные потоки:

• A1. Основание либо категория не подходят: модератор отклоняет допуск с причиной, аукцион не создаётся.

• A2. Продавец ставит на собственный лот, цена ниже минимальной или срок закончился: ставка отклоняется, принятые ранее ставки сохраняются.

• A3. Ставки приходят одновременно: после первой принятой ставки второй запрос проверяется относительно новой цены. Повтор с тем же ключом не создаёт дубль.

• A4. Ставок нет: аукцион завершается без победителя; дело передачи не создаётся, вещь остаётся доступной для дальнейшего поиска.

• A5. Появляется заявление владельца, жалоба или блокировка участника: торги приостанавливаются и передаются на разбор UC-04. Приостановленный аукцион новые ставки не принимает.

• A6. У победителя истекла подписка: результат принятой ставки сохраняется; участники завершают существующую передачу без продления. Односторонняя отмена результата торгов запрещена; спор рассматривает модератор.

5.7. UC-06. Изменить полномочия сотрудника

Поле	Описание
Цель	Назначить или отозвать служебную роль конкретного пользователя.
Инициатор	Администратор.
Другие участники	Участник, которому назначается или отзывается роль модератора либо администратора.
Предусловия	Инициатор имеет действующие административные права; целевая запись существует.
Триггер	Возникает необходимость изменить состав сотрудников.
Постусловия	Назначения ролей изменены, причина и исполнитель сохранены. Новые запросы сотрудника проверяются по актуальным полномочиям.
Требования	FR-04, FR-19, FR-20; NFR-04, NFR-13.

Шаг	Исполнитель	Действие и реакция системы
1	Администратор	Открывает пользователей и выбирает целевую запись.
2	Система	Показывает текущие назначения и состояние пользователя.
3	Администратор	Выбирает назначение либо отзыв одной служебной роли и указывает причину.
4	Система	Повторно проверяет права инициатора и защиту последнего действующего администратора.
5	Система	Сохраняет назначение и запись аудита в одной транзакции.
6	Сотрудник	Обращается к служебной странице. Система проверяет текущее назначение роли, включая случаи использования ранее выданного сеанса.
7	Администратор	Проверяет запись изменения в журнале служебных действий.

Альтернативные потоки: повторное назначение не создаёт дубль; отзыв роли последнего администратора запрещён; запрос сотрудника без требуемой роли отклоняется независимо от состояния старого токена.

5.8. UC-07. Актуализировать инструкцию организации

Поле	Описание
Цель	Обновить проверенные контакты и порядок обращения к хранителю найденной вещи.
Инициатор	Администратор.
Другие участники	Участник, которому требуется инструкция. Официальный сайт организации является источником данных.
Предусловия	Организация есть в справочнике; доступен официальный источник для проверки.
Триггер	Истекает срок пересмотра записи либо обнаружены изменившиеся сведения.
Постусловия	В справочнике сохранены проверенные сведения, источник и дата проверки; пользователю показывается новая редакция.
Требования	FR-10, FR-19, FR-20; NFR-15.

Шаг	Исполнитель	Действие и реакция системы
1	Администратор	Находит запись, требующую проверки. Система показывает источник и дату последнего пересмотра.
2	Администратор	Сверяет телефон, адрес и инструкцию с официальным источником вне приложения.
3	Администратор	Вносит сведения и дату проверки, сохраняет ссылку на источник.
4	Система	Проверяет данные и полномочия, обновляет запись и сохраняет действие в аудите.
5	Участник	Открывает инструкцию метро либо карточку вещи у организации.
6	Система	Показывает актуальную запись, дату и официальный источник. Справочная страница доступна без подписки.

Альтернативные потоки: источник недоступен — сведения не помечаются как заново проверенные, остаются прежняя дата и предупреждение; ошибочная ссылка или отсутствующие обязательные поля — обновление отклоняется. Пользователь может уточнить порядок выдачи по официальному телефону.

5.9. UC-08. Изменить условия новых подписок

Поле	Описание
Цель	Ввести новый тариф, сохранив условия уже созданных заказов и оплаченных периодов.
Инициатор	Администратор.
Другие участники	Участник, оформляющий подписку; платёжная служба участвует в оплате через UC-01.
Предусловия	Администратор имеет доступ к тарифам.
Триггер	Принимается решение изменить стоимость или длительность новых подписок.
Постусловия	Новый тариф доступен для новых заказов; старые заказы и подписки сохраняют исходные параметры. Изменение записано в аудит.
Требования	FR-03, FR-19, FR-20; NFR-08.

Шаг	Исполнитель	Действие и реакция системы
1	Администратор	Вводит название, цену и длительность нового тарифа.
2	Система	Проверяет допустимость сумм и срока, сохраняет новый тариф и аудит.
3	Администратор	Закрывает прежний тариф для новых заказов, если его использование прекращается.
4	Участник	Открывает доступные тарифы и выбирает новый вариант.
5	Система	Создаёт заказ со снимком цены и длительности выбранного тарифа.
6	Участник	Продолжает оформление через UC-01. Ранее созданный заказ при

Шаг	Исполнитель	Действие и реакция системы
		оплате использует сохранённые условия.
7	Администратор	Проверяет историю изменения; ранее оплаченные периоды продолжают действовать до своего окончания.

Альтернативные потоки: цена или срок некорректны — тариф не сохраняется; закрытый тариф нельзя выбрать для нового заказа; повтор оплаты уже созданного заказа не увеличивает срок повторно.

6. Предлагаемая архитектура

6.1. Архитектурный подход и технологии

Система проектируется как модульное монолитное приложение на Java 17. Одна JVM принимает HTTP-запросы, формирует страницы и выполняет прикладные операции. PostgreSQL и файловое хранилище отделены от процесса приложения. Модули предметной области разделяют ответственность, но используют общие правила доступа и транзакции.

Уровень	Технологии	Ответственность
Представление	Spring MVC, Thymeleaf, HTML, CSS	Страницы, формы, валидация параметров, отображение результатов и ошибок
Аутентификация	Spring Security, JWT, BCrypt	Проверка сеанса, защита форм CSRF, вход, выход и обновление токенов
Бизнес-логика	Java-сервисы, AccessPolicy, Spring Transactions	Правила процессов, права на объекты, координация действий и транзакционные границы
Хранение	Hibernate/JPA, Spring Data JPA, JdbcTemplate	Проекции сущностей, параметризованные запросы, вызовы функций и процедур
База данных	PostgreSQL, PL/pgSQL	Связи, ограничения, триггеры, блокировки и критичные переходы состояний
Файловый адаптер	Закрытый локальный каталог, Java ImageIO	Проверка фотографий, хранение вложений и выдача после проверки прав

Уровень	Технологии	Ответственность
Внешние адаптеры	Учебная оплата; почтовый интерфейс и SMTP	Подтверждение оплаты подписки, отправка писем и замена интеграций для демонстрации
Сборка и проверка	Gradle Wrapper, JUnit, Spring Test, Java-модуль tooling	Сборка JAR, интеграционные проверки, диаграммы, отчёты и установка на helios

Java, Spring Boot, Hibernate/JPA, PostgreSQL, Lombok и Gradle выбраны с учётом проекта MyFilmList [7]. Spring MVC и Thymeleaf позволяют включить контроллеры и представления в один JAR [8, 9]. Семейство Spring Boot 3.5 поддерживает выбранную версию Java [5]. Версии фиксируются в build.gradle. Изменения схемы представлены SQL-скриптами в Git; Hibernate проверяет соответствие сущностей существующей схеме и не создаёт таблицы при запуске.

На диаграмме компонентов показаны направления вызовов внутри приложения. Развёртывание, сетевые соединения и места хранения вынесены в отдельную схему.

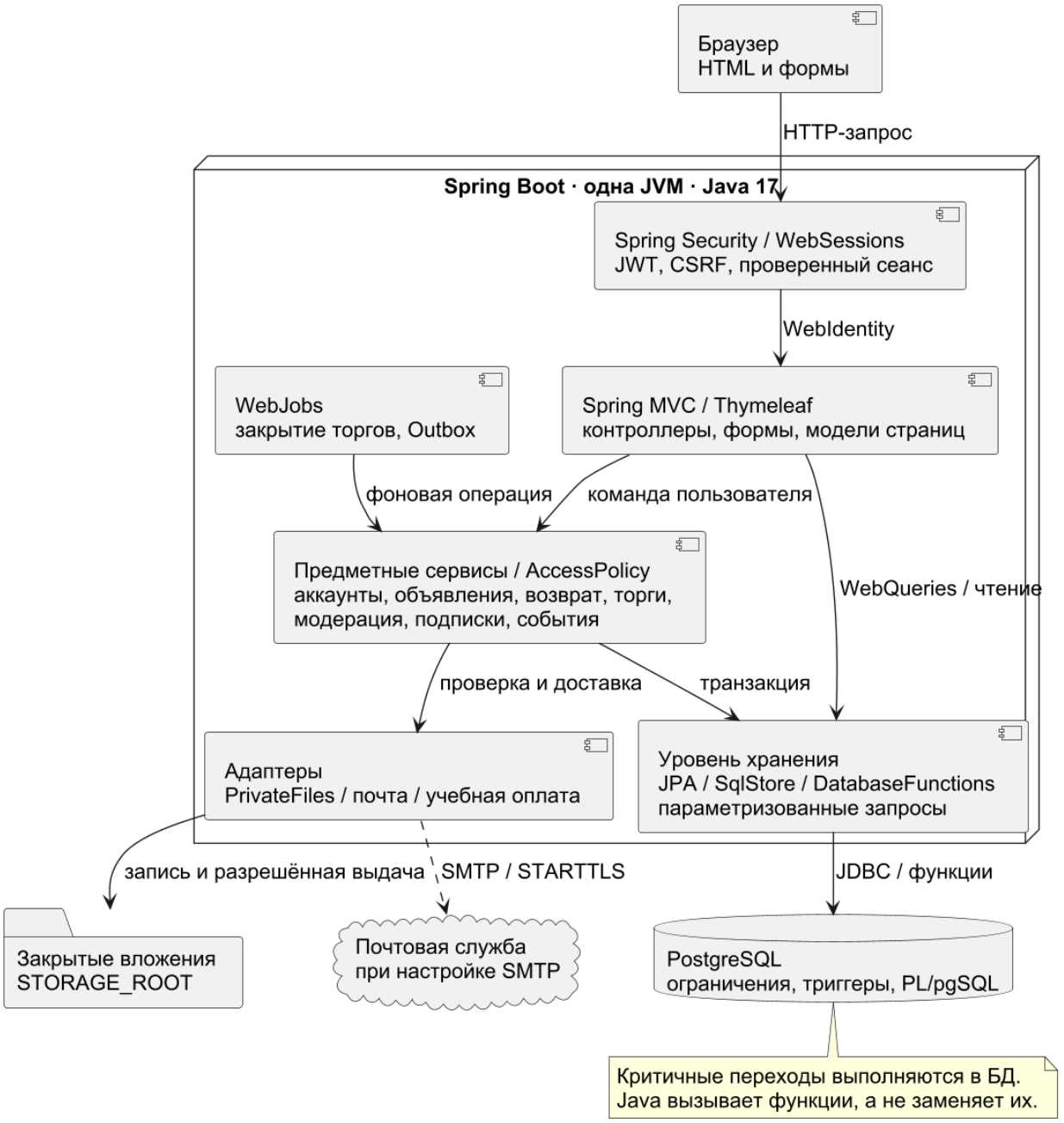


Рисунок 4 — Компоненты и уровни приложения

6.2. Компоненты и распределение ответственности

Компонент	Вход и результат	Основные зависимости
WebSessions, WebIdentity, Spring Security	Проверенный сеанс и идентификатор действующего пользователя	Хеши паролей, версии токенов и отозванные сеансы в PostgreSQL
MVC-контроллеры	Параметры форм; модель страницы либо перенаправление	Сервисы предметной области и Thymeleaf

Компонент	Вход и результат	Основные зависимости
WebQueries	Проекции для каталога, кабинета и очередей	SqlStore и проверки доступа; публичные проекции исключают секреты
AccountService, SubscriptionService	Подтверждённый профиль, заказ и оплаченный период; UC-01	AccessPolicy, почтовый и платёжный адаптеры, DatabaseFunctions
ListingService	Черновик, публикация, фильтры и карточка; UC-02 и начало UC-03	AccessPolicy, SqlStore, поиск PL/ pgSQL и файловый адаптер
ReturnService	Заявка, диалог, резерв и подтверждения; UC-03	Проверка участников и функции резервирования и передачи
AdministrationService	Жалоба, служебное решение, роли, тарифы, организации; UC-04, UC-06–08	AccessPolicy, SqlStore и неизменяемый аудит
AuctionService	Допуск, лот, ставка и итог; UC-05	Функции торгов, запись находки и проверка права продавца
OutboxService, WebJobs	Доставка уведомлений и закрытие истёкших торгов	Очередь событий и процедура PostgreSQL; повторная обработка безопасна
SqlStore, репозитории JPA, DatabaseFunctions	Параметризованные операции хранения	JDBC-пул и схема PostgreSQL
PrivateFiles, MediaPages	Проверенный файл и разрешённая выдача	Закрытый каталог и проверка владельца либо служебных полномочий

Роль инициатора определяется по проверенному сеансу. Контроллер не принимает идентификатор действующего пользователя из скрытого поля формы. AccessPolicy проверяет роль, состояние профиля, подписку и принадлежность объекта в серверном коде; скрытие кнопки не заменяет эту проверку. Для модератора отдельно проверяется отсутствие заинтересованности в рассматриваемом деле.

Модели Thymeleaf содержат только сведения, нужные странице. Пароли, хеши токенов и чужие закрытые признаки в них не включаются. Служебная очередь и публичный каталог используют разные проекции. Шаблоны находятся в src/main/resources/templates, стили — в src/main/resources/static и включаются в JAR.

6.3. Обработка запроса на примере возврата

1. Владелец открывает опубликованную находку. Spring Security проверяет токен и текущий сеанс; контроллер получает идентификатор пользователя через WebIdentity.

2. Контроллер передаёт заявку и доказательства в ReturnService. AccessPolicy проверяет доступ, сервис создаёт заявку и диалог в одной транзакции и сохраняет событие уведомления.

3. Нашедший открывает дело из своего кабинета. Проверка принадлежности даёт доступ только участникам заявки; во время открытого спора допускается независимый модератор.

4. После уточнений нашедший подтверждает согласование. DatabaseFunctions вызывает lf_reserve_found; функция блокирует находку, проверяет отсутствие другого резерва и создаёт передачу.

5. Первое подтверждение вызывает lf_confirm_transfer и сохраняет одну отметку. Второе завершает передачу, заявку и состояние находки той же функцией.

6. Контроллер возвращает обновлённую страницу. OutboxService доставляет уведомления после фиксации транзакции. Ошибка внешнего адаптера не отменяет уже сохранённый результат возврата.

Диаграмма взаимодействия UC-03 показывает разных пользователей одной роли и последовательность их обращений к приложению. Обмен HTTP-запросами идёт через контроллеры; пользователи не обращаются к базе напрямую.

6.4. Хранение и транзакционные границы

В PostgreSQL размещаются учётные записи, назначения ролей, токены, проверки профилей, тарифы и заказы, подписки, объявления, справочники, заявки, сообщения, передачи, жалобы, допуски, аукционы, ставки, уведомления, очередь событий и аудит. Таблица назначений ролей образует связь M:N между пользователями и ролями. Конкретные таблицы, атрибуты и ограничения разрабатываются на втором этапе.

Критичные операции реализуются функциями PL/pgSQL: активация подписки, поиск, резервирование, подтверждение передачи, приём ставки и завершение торгов. Третий этап должен вызывать эти функции из Java, сохраняя правила и блокировки в базе. Прикладной слой проверяет пользователя, подготавливает параметры и обрабатывает результат.

Операция	**Транзакционная граница**	**Защита от конфликта**
Оплата подписки	Событие оплаты и один оплаченный период	Ключ события, блокировки пользователя и заказа, отсутствие повторного продления
Согласование возврата	Принятая заявка, резерв, передача и событие	Блокировка находки, один действующий резерв и отсутствие незавершённых торгов
Подтверждение передачи	Отметка стороны либо завершение всего дела	Блокировка состояния дела и проверка участников
Приём ставки	Проверенная ставка и событие уведомления	Единый порядок блокировок: находка, затем аукцион; повторная проверка времени и цены
Завершение торгов	Победитель, итог и единственное дело передачи	Те же блокировки, запрет ставки после окончания, идемпотентный повтор
Служебное решение	Изменение состояния и аудит	Проверка независимости, полномочий и допустимого перехода

Для денежных значений применяется NUMERIC(12,2), для моментов времени — timestamptz. Ввод и отображение используют Europe/Moscow. Поиск выполняется средствами PostgreSQL: индексы по месту, категории, типу и состоянию, полнотекстовый индекс по русскому описанию. Состав и полезность индексов обосновываются на втором этапе по прецедентам.

Outbox хранит событие вместе с бизнес-изменением в одной транзакции. Отдельный обработчик выбирает недоставленные записи, создаёт внутренние уведомления и отмечает доставку. Аудит фиксирует исполнителя, объект, действие, время и результат; пользовательские операции не редактируют журнал.

6.5. Файлы и внешние интеграции

Фотографии и подтверждающие материалы хранятся отдельно от статических ресурсов. В базе находятся ключ файла, тип, размер и связь с объявлением либо допуском. Фотография декодируется, проверяется и пересохраняется без исходных метаданных. Выдача файла идёт через контроллер с проверкой прав. Документ допуска доступен продавцу и модератору; публичный каталог его не раскрывает.

Для учебной установки выбран закрытый локальный каталог STORAGE_ROOT. MinIO может заменить файловый адаптер при дальнейшем развитии; такой вариант требует настройки закрытого бакета и совместного резервного копирования базы и объектов. Он не является зависимостью первоначального запуска на helios.

Платёжный адаптер подтверждает оплату подписки, а не расчёт между продавцом и покупателем лота. Учебный режим не списывает средства и проверяет повторные результаты. При подключении внешнего провайдера результат должен проверяться адаптером до вызова функции активации подписки.

Почтовый интерфейс используется для подтверждения адреса и восстановления доступа. В учебном режиме письма доступны получателю в его кабинете; для внешней доставки предусмотрены параметры SMTP и STARTTLS. Внутренние уведомления сохраняются в PostgreSQL независимо от почтовой доставки.

Справочная интеграция метро читает данные из справочника организаций. Администратор обновляет запись по официальному источнику через UC-07. Запрос к каталогу или инструкции не зависит от доступности сайта метрополитена; прямой обмен реестрами не предусматривается.

6.6. Развёртывание на helios

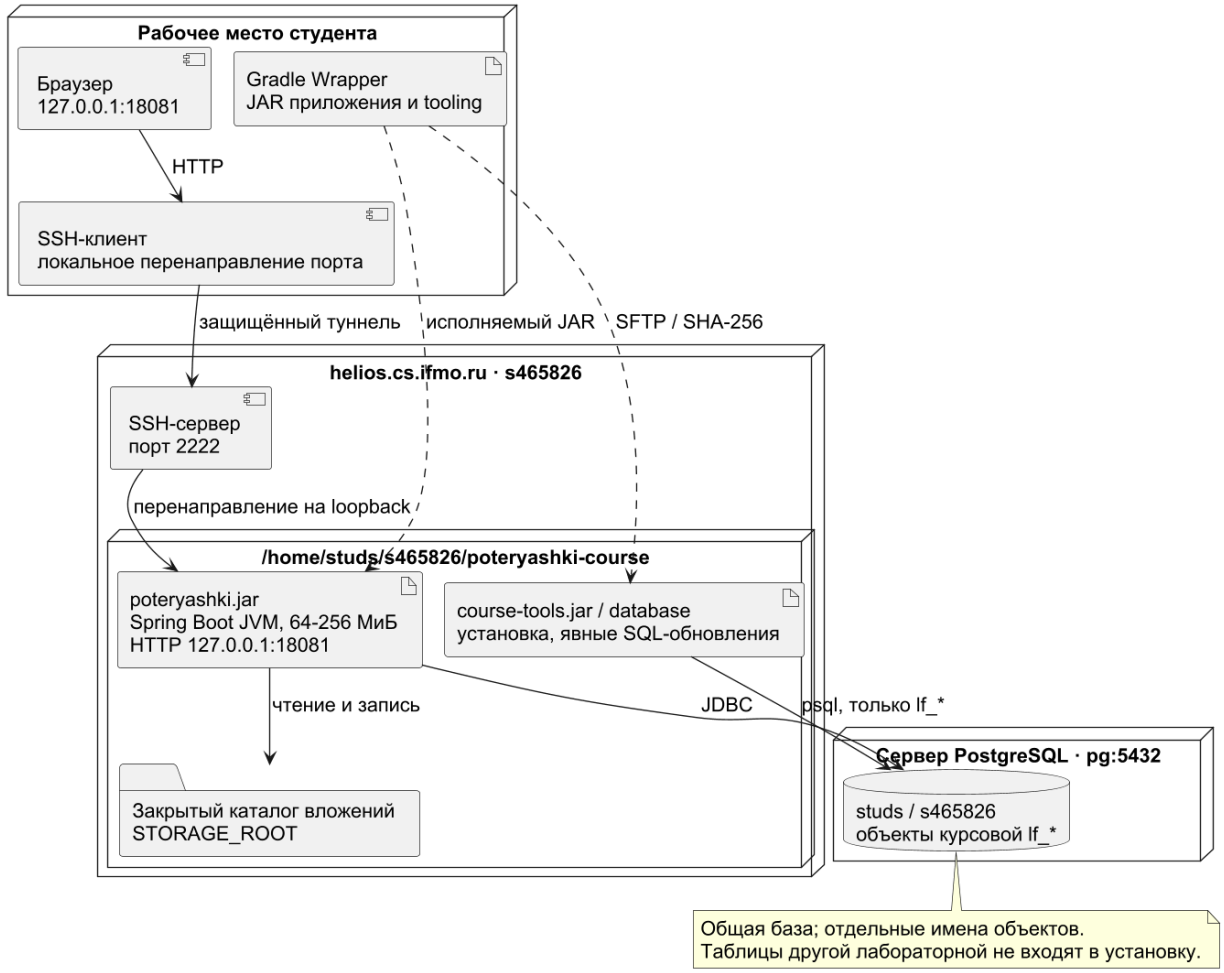


Рисунок 4.1 — Узлы развёртывания и сетевые соединения

Узел или ресурс	Размещение и доступ
Браузер	Рабочее место студента, http://127.0.0.1:18081
SSH-туннель	Соединение с helios.cs.ifmo.ru:2222, перенаправление локального 18081 на loopback-порт приложения
Spring Boot JVM	~/poteryashki-course, HTTP только на 127.0.0.1:18081, память 64–256 МиБ
PostgreSQL	Сервер pg:5432, база studs, схема s465826, таблицы и функции с префиксом lf_
Файлы приложения	Отдельный каталог STORAGE_ROOT внутри рабочей области курсовой, без публичной раздачи
SQL и средства сопровождения	database/ и Java-модуль tooling; установка и проверки ограничены объектами курсовой
Внешняя почта и оплата	Учебные адаптеры; реальные соединения требуют отдельной настройки

Учётная запись на helios создаёт объекты в своей схеме общей базы studs; отдельная база или схема не требуется. Префикс lf_ отделяет курсовую от другой лабораторной. Изменения схемы применяются явно собственными SQL-скриптами; приложение запускается с Hibernate validate. Общий журнал миграций другой работы не используется.

Приложение и tooling собираются локально Gradle Wrapper. Установка передаёт JAR, ресурсы, SQL и документацию в ~/poteryashki-course и проверяет контрольные суммы. Обновление останавливает только процесс курсовой, применяет изменения схемы, выполняет демонстрационный запуск и открывает HTTP-сервер. Пароли БД и почты, секрет JWT и SSH-пароль получают из окружения либо защищённых локальных настроек; в репозиторий они не включаются.

Параметры DB_URL, DB_USER, DB_PASSWORD и DB_PREFIX задают подключение к данным. APP_PORT и STORAGE_ROOT задают порт и каталог файлов; MAIL_MODE, MAIL_HOST и связанные параметры — почту. В учебном режиме включены демонстрационная оплата и планировщик. Для публичного размещения потребуется HTTPS и Cookie Secure; демонстрация на helios использует защищённый SSH-туннель.

Подключение с рабочей машины:

```
ssh -p 2222 -L 18081:127.0.0.1:18081 s465826@helios.cs.ifmo.ru
```

Запуск из ~/poteryashki-course:

```
java -Xms64m -Xmx256m -jar tooling/build/libs/course-tools.jar web start
```

После запуска браузер открывает http://127.0.0.1:18081. Перед демонстрацией проверяются доступ к PostgreSQL, свободный порт, каталог вложений и учебные данные. Проверки создают временные объекты lfcheck_* или lf3check_*, отказываются использовать уже существующий набор и удаляют только свой набор с проверкой исходных объектов и последовательностей.

6.7. План демонстрации последующих этапов

1. Показать три роли и восемь прецедентов, затем компоненты и узлы размещения.

2. Пройти UC-01: новый участник подтверждает почту, модератор одобряет профиль, участник оплачивает подписку.

3. Пройти UC-02: участник готовит находку с фотографией и закрытой приметой, независимый модератор публикует её.

4. Пройти UC-03 в двух учётных записях: владелец находит вещь, нашедший проверяет заявку, стороны общаются, резервируют и подтверждают передачу.

5. Показать альтернативу метро: официальный контакт, хранитель и порядок обращения вместо личного резервирования.

6. Пройти UC-04: участник направляет жалобу, модератор изучает материалы, стороны дают пояснения и получают решение.

7. Пройти UC-05: допуск, создание лота, предложения двух покупателей, завершение и дело передачи победителю.

8. Показать UC-06–08: права сотрудника, актуальная инструкция и новый тариф с сохранением старых заказов.

9. Показать контроль конкуренции, повтор оплаты, истечение подписки, историю решений и протоколы проверок.

Вывод

Определены предметная область, три роли доступа и восемь прецедентов с законченным результатом. Описаны взаимодействия участников, модератора и администратора, основные и альтернативные потоки. Уточнены функциональные и нефункциональные требования. Архитектура разделяет представление, проверку сеанса, предметные сервисы, хранение и адаптеры. Java-приложение на Spring MVC и Thymeleaf размещается на helios, данные — в PostgreSQL, вложения — в закрытом каталоге. MinIO рассматривается как расширение файлового адаптера.

Второй этап включает ER-модель, даталогическую модель, схему PostgreSQL, ограничения и триггеры, тестовые данные, функции PL/pgSQL и индексы. Реализация интерфейса и серверного приложения относится к последующим этапам.

Список использованных источников

Дата проверки источников: 03.10.2026. Контакты и условия организации подлежат повторной проверке перед эксплуатацией. Все проектные требования и численные цели сформулированы для данной курсовой; они не являются требованиями метрополитена.

1. Петербургский метрополитен. Справочная о забытых вещах. Официальный источник порядка хранения и получения: https://metro.spb.ru/zabitieveshy.html?v=1

2. Петербургский метрополитен. Контакты. Проверка телефона информационно-справочного центра: https://metro.spb.ru/contacts.html

3. Гражданский кодекс Российской Федерации, часть первая, статья 227 «Находка». Текст нормы в СПС «КонсультантПлюс»: https://www.consultant.ru/document/cons_doc_LAW_5142/b3526f37d7093a56bf084817caeee175f032a3d7/

4. Гражданский кодекс Российской Федерации, часть первая, статья 228 «Приобретение права собственности на находку». Текст нормы в СПС «КонсультантПлюс»: https://www.consultant.ru/document/cons_doc_LAW_5142/104dea7aa78d4c2e927d3baed4e96d5ace10e692/

5. Spring. Spring Boot 3.5 — System Requirements. Официальная документация: https://docs.spring.io/spring-boot/3.5/system-requirements.html

6. Кафедра ВТ, Университет ИТМО. Программирование. Публичные задания с демонстрацией Java-программ на helios: https://se.ifmo.ru/courses/programming

7. MyFilmList. Backend, build.gradle, версия 74d1928973cae329d6ca651bac2ffde445a43de0: https://github.com/maxbarsukov/MyFilmList/blob/74d1928973cae329d6ca651bac2ffde445a43de0/build.gradle

8. Spring Framework. Thymeleaf: https://docs.spring.io/spring-framework/reference/web/webmvc-view/mvc-thymeleaf.html

9. Thymeleaf. Интеграция со Spring: https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html

Приложение А. Термины

Термин	Определение
Верификация профиля	Подтверждение почты и решение модератора о допуске аккаунта; не паспортная идентификация
Подписка	Оплаченный период доступа к пользовательским функциям
Контрольный признак	Непубличная характеристика вещи для проверки претендента
Резервирование	Согласование выдачи одной находки одному претенденту
Справочная интеграция	Подбор проверенных контактов и инструкций организации без обмена её внутренними данными
Допуск аукциона	Решение модератора по материалам основания распоряжения вещью
Идемпотентность	Повтор запроса или события не создаёт повторный результат операции
RPO	Допустимая потеря данных по времени относительно сбоя
RTO	Целевое время восстановления работы
Outbox	Таблица событий, записываемых вместе с бизнес-операцией для последующей надёжной доставки

Приложение Б. План проверки требований

Пункт задания	Раздел отчёта	Результат
1. Выбор предметной области	1–2	Поиск и возврат потерянных вещей в городской среде
2. Подробное текстовое описание	2	Участники, процессы, данные, статусы, бизнес-правила
3. Зачем нужна система и какие задачи решает	3	Цель, проблемы и задачи автоматизации
4. Функциональные/ нефункциональные требования	4	FR-01–FR-20, NFR-01–NFR-16 и критерии проверки
5. Модели и описания прецедентов	5	Три роли, восемь описаний, три диаграммы прецедентов и взаимодействие двух участников
6. Архитектура, технологии и helios	6	Компоненты, развёртывание, связи, транзакции и адаптеры Java/Spring MVC/PostgreSQL

Пункт задания	Раздел отчёта	Результат
7. Отчёт	Весь документ	Результаты анализа предметной области и проектирования системы

Группа требований	Прецеденты	Проверка на этапе реализации
Аккаунты и подписка: FR-01–04	UC-01, UC-06	Проверка почты, решение модератора, оплата/отказ/повтор, прямой запрос без доступа
Объявления и поиск: FR-05–09	UC-02, UC-03	Обязательные поля, файлы, закрытые признаки, повторная модерация и комбинированный поиск
Метро: FR-10	UC-03 (A6), UC-07	Хранитель, официальный контакт и источник, дата проверки, отсутствие записи
Возврат: FR-11–13	UC-03, UC-04, UC-05	Заявки, права на диалог, конкурирующие резервы, два подтверждения обычной и аукционной передачи
Уведомления и споры: FR-14–15	UC-01–05	Решения и сообщения в кабинете, сбой доставки, жалоба с причиной и результатом
Аукцион: FR-16–18	UC-05; спор — UC-04	Допуск, продавец не ставит, конкуренция, срок, отсутствие ставок и приостановление
Управление и аудит: FR-19–20	UC-06–08; аудит всех процессов	Права сотрудников, новый тариф, инструкция, отсутствие секретов и запрет изменения аудита

Для NFR-01/02 предусматривается нагрузочная проверка; для NFR-03 — интеграционные тесты конкурентных транзакций на PostgreSQL; для NFR-04–08 и NFR-13 — проверки доступа, ввода и отказов адаптеров. NFR-09 проверяется восстановлением, NFR-10/11 — пользовательскими сценариями и размерами экрана, NFR-12/14 — сборкой и запуском, NFR-15 — просроченной записью справочника, NFR-16 — сравнением исходных объектов и последовательностей до и после временного набора, проверкой синтетических адресов и закрытого каталога. Проверки проводятся после реализации соответствующих функций.

7. ER-модель базы данных

7.1. Сущности и связи

Модель построена по процессам и прецедентам первого этапа и содержит 27 сущностей: пользователи, роли, назначения ролей, проверки профилей, одноразовые токены, токены обновления, тарифы, заказы оплаты, платёжные события, периоды подписки, организации, места, категории, объявления, изображения, контрольные признаки, заявки на передачу, диалоги, сообщения, передачи, жалобы, допуски к продаже, аукционы, ставки, уведомления, события доставки и записи аудита.

Пользователь может иметь несколько ролей, а одна роль назначается нескольким пользователям. Отношение «многие ко многим» реализовано ассоциативной сущностью «Назначение роли»: lf_user_roles(user_id, role_code). Составной первичный ключ исключает повторное назначение одной роли пользователю.

Объявление принадлежит одному автору, одной категории и одному месту. Место может быть связано с организацией, предоставляющей справочную информацию. Организация-хранитель находки задаётся отдельно: событие могло произойти в одном месте, а вещь передана другой организации. Поле custodian_org_id не означает партнёрство организации с приложением.

Заявка связывает находку с претендентом. У находки может быть несколько заявок, но один действующий резерв. Диалог и передача относятся к конкретной заявке; участники определяются через автора объявления и заявителя. Необязательный auction_id с внешним ключом и UNIQUE связывает дело передачи с завершённым аукционом: одному аукциону соответствует не более одного такого дела. Допуск к продаже относится к находке и продавцу; каждый аукцион использует конкретный допуск. Это исключает подстановку разрешения для другой вещи. Всего модель содержит 42 внешних ключа.

Заказ оплаты фиксирует стоимость и длительность тарифа на момент создания. Изменение справочника тарифов не меняет уже созданный заказ. Внешние события оплаты отделены от периодов подписки, поскольку платёжный сервис может повторить событие. Одному оплаченному заказу соответствует не более одного периода.

Уведомление адресовано пользователю. Событие доставки содержит тип агрегата, его идентификатор и JSON-параметры; эта техническая очередь не имеет универсального внешнего ключа на разные таблицы. Аудит хранит таблицу и идентификатор изменённой сущности. Его actor_id может быть NULL для фоновой операции.

7.2. Диаграммы

Модель разделена на шесть листов для чтения в формате A4. Листы показывают основные связи и их мощности; повтор сущности на разных листах не означает дополнительную таблицу. Полный перечень внешних ключей и атрибутов приведён в даталогической модели и словаре отношений. Обозначения: 1 — ровно один, 0..1 — необязательный экземпляр, 0..N — множество.

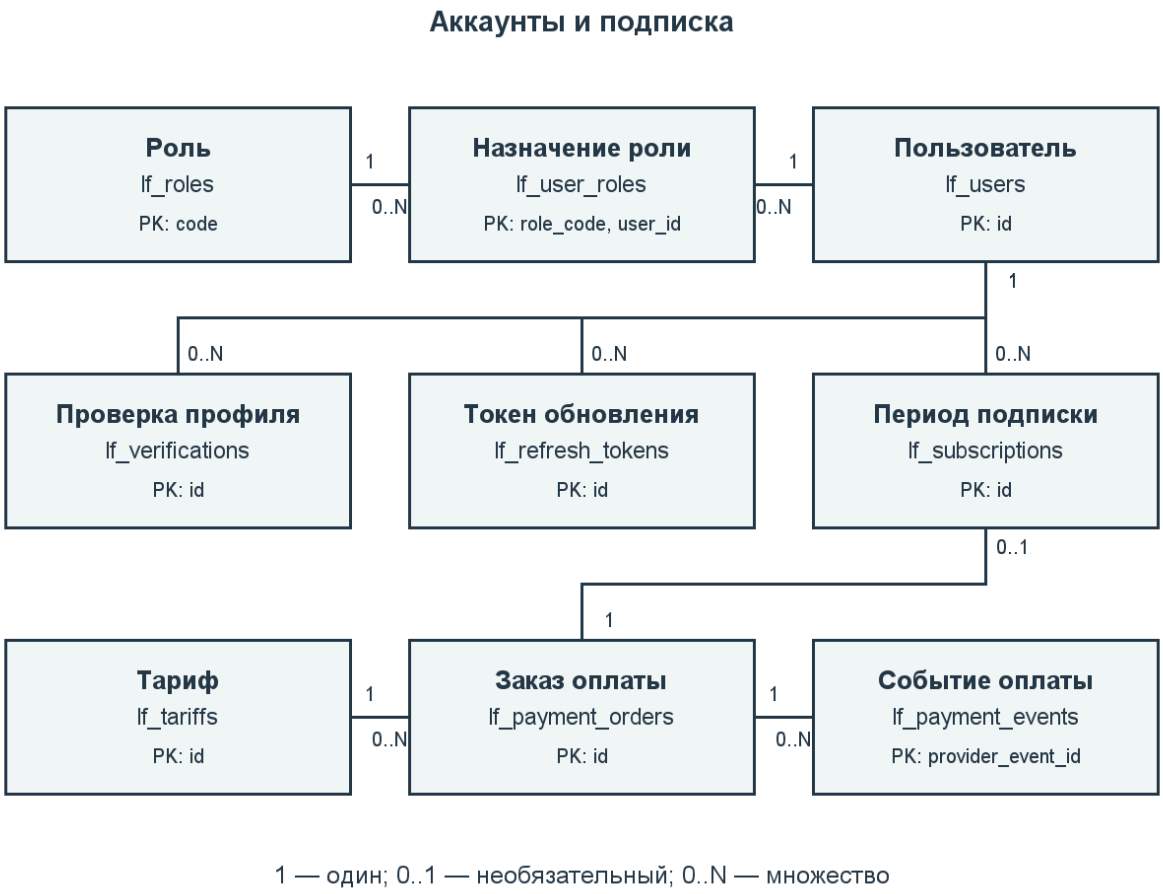


Рисунок 5 — ER-модель: аккаунты и подписка

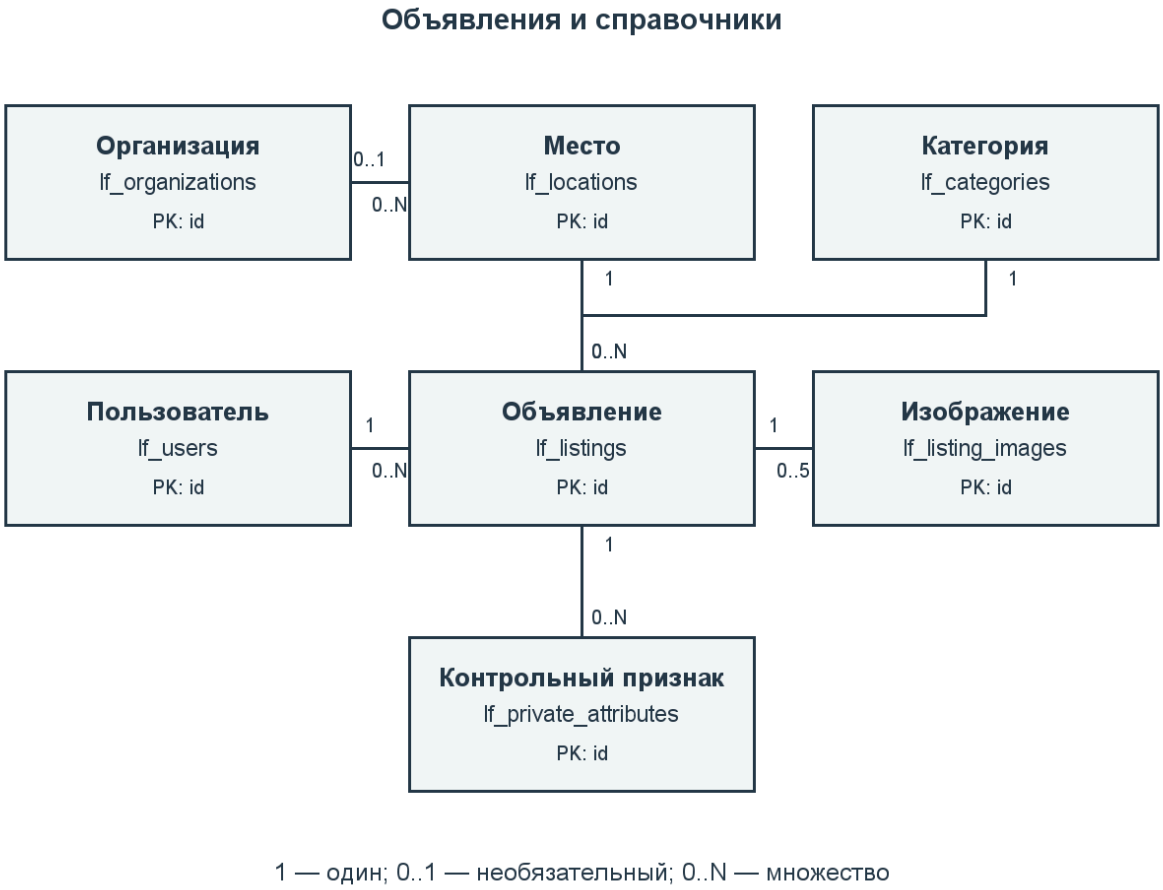


Рисунок 6 — ER-модель: объявления и справочники

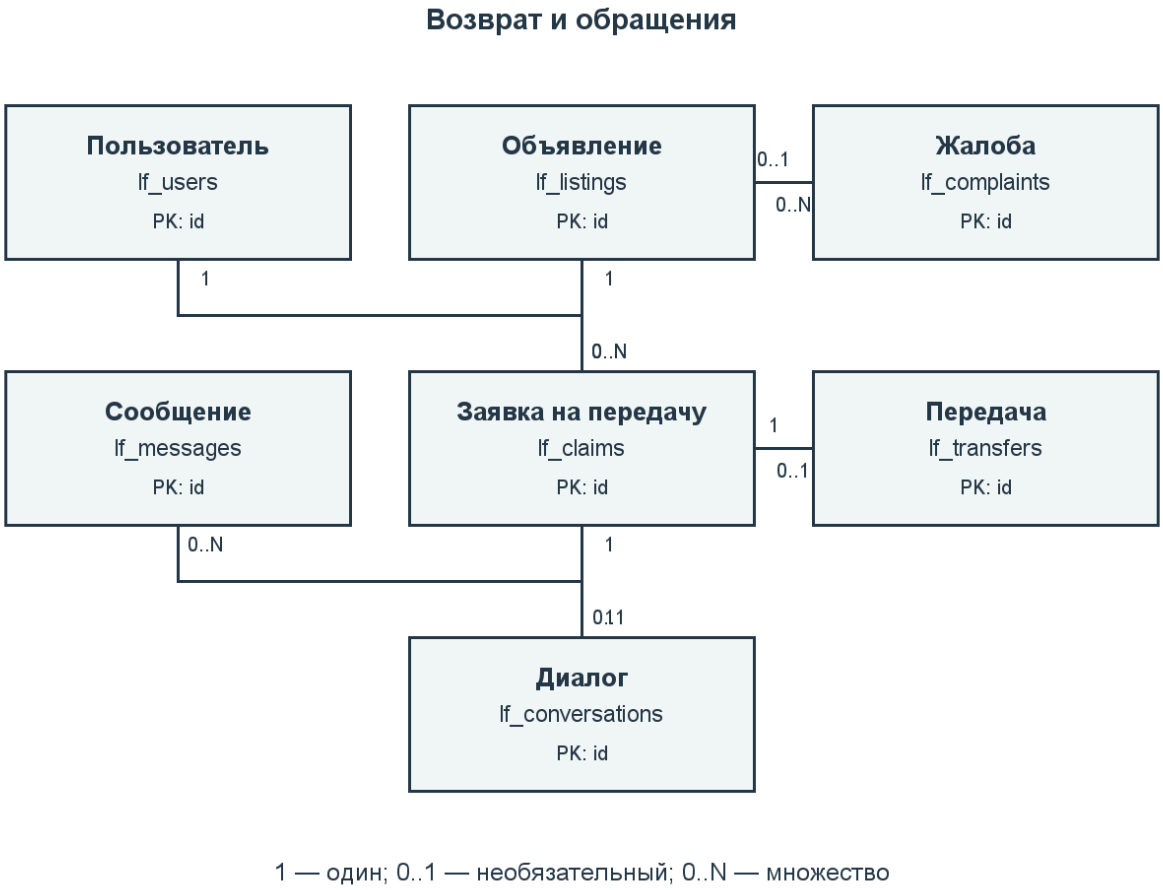


Рисунок 7 — ER-модель: возврат и обращения

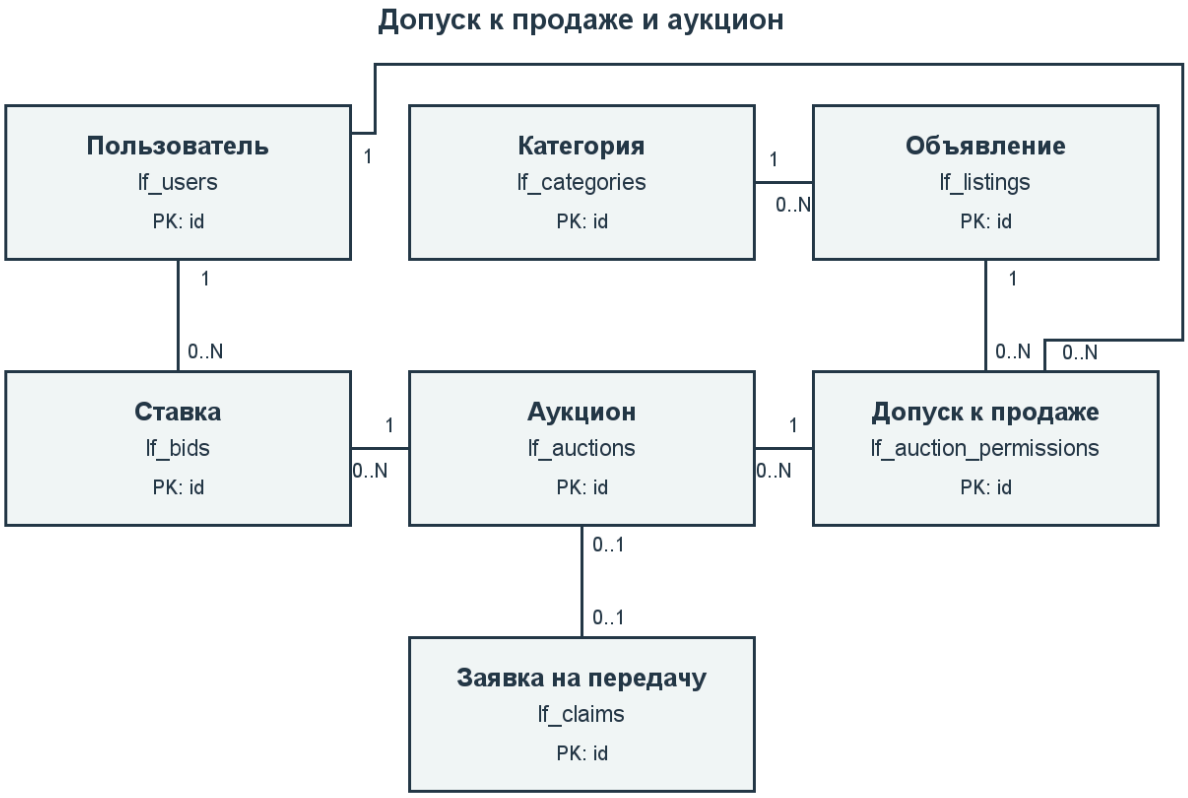


Рисунок 8 — ER-модель: аукционы

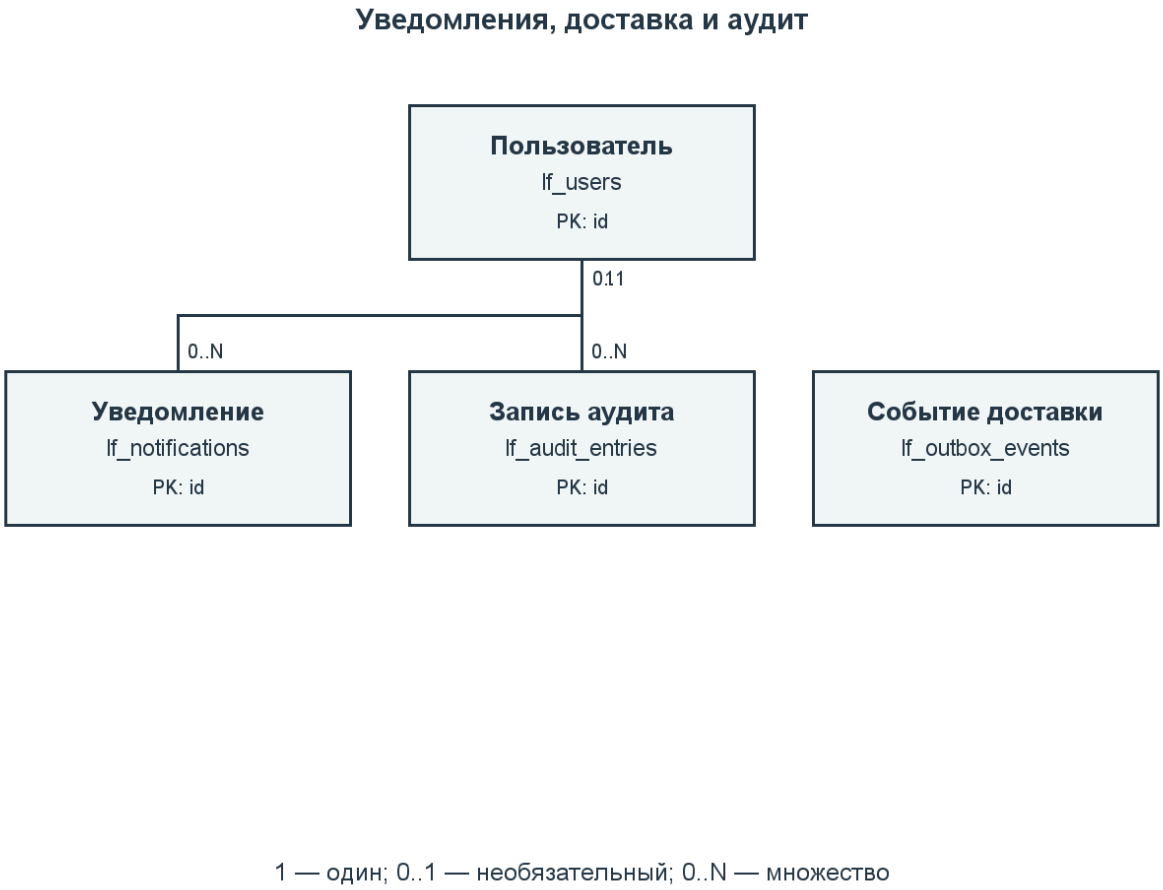


Рисунок 9 — ER-модель: уведомления, доставка и аудит



Рисунок 10 — ER-модель: верификация и токены

Редактируемая концептуальная модель находится в docs/part2/model/er.mmd, полная даталогическая схема — в logical.mmd. Формат Mermaid поддерживает отображение ER-диаграмм и связей с ключами. Изображения каждого листа предоставлены в PNG и SVG.

8. Даталогическая модель

8.1. Преобразование ER-модели

Каждая сущность преобразована в таблицу PostgreSQL. Основные идентификаторы — bigint GENERATED BY DEFAULT AS IDENTITY; код роли является естественным ключом. Для ассоциации пользователей и ролей выбран составной ключ. Связи 1:N реализованы внешними ключами; для связей 1:0..1 добавлено UNIQUE: заявка — диалог, заявка — передача и заказ оплаты — период подписки.

Модель приведена к третьей нормальной форме. Атрибуты пользователя не копируются в объявления, диалоги и ставки. Сведения о категории, месте,

организации и тарифе вынесены в справочники. Составной ключ назначения роли не содержит атрибутов, зависящих только от одной его части. Снимок суммы и длительности в заказе относится к самому заказу и необходим для сохранения условий оплаты, поэтому не заменяется ссылкой на текущие значения тарифа.

Время хранится в timestamptz. Начало события обязательно; event_until задаётся только для интервала потери. Денежные значения используют numeric(12,2), без float. Закрытые признаки хранятся в lf_private_attributes и отсутствуют в публичном представлении. Файлы представлены ключами объектов MinIO; бинарные данные не записываются в PostgreSQL. Технические JSON-поля применяются только в очереди событий и аудите, а связи предметной области остаются реляционными.

Циклическая связь аукциона и выигравшей ставки реализована составным внешним ключом (id, winner_bid_id) → lf_bids(auction_id, id), отложенным до завершения транзакции. Победитель не может относиться к другому аукциону. Отложенная проверка резерва позволяет в одной транзакции согласовать заявку и изменить статус объявления. Физический столбец search_vector является контролируемой избыточностью для поиска: PostgreSQL вычисляет его из текста объявления автоматически; в концептуальной нормализованной модели этот атрибут не требуется.

Словарь отношений сформирован из системного каталога PostgreSQL после установки на helios. Он отражает реально созданные типы, возможность NULL, первичные и внешние ключи. Отдельные частичные уникальные индексы описаны в разделе 13.

8.2. Словарь отношений

1. Допуск к продаже: lf_auction_permissions

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
listing_id	bigint	Нет	FK
applicant_id	bigint	Нет	FK
basis	varchar(20)	Нет	—

Поле	Тип PostgreSQL	NULL	Ключ
evidence_object_key	varchar(240)	Нет	—
state	varchar(12)	Нет	—
moderator_id	bigint	Да	FK
reviewed_at	timestamptz	Да	—
reason	text	Да	—
created_at	timestamptz	Нет	—

FOREIGN KEY (applicant_id) REFERENCES lf_users(id).

UNIQUE (id, listing_id, applicant_id).

FOREIGN KEY (listing_id) REFERENCES lf_listings(id).

FOREIGN KEY (moderator_id) REFERENCES lf_users(id).

PRIMARY KEY (id).

2. Аукцион: lf_auctions

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK, FK
listing_id	bigint	Нет	FK
seller_id	bigint	Нет	FK
permission_id	bigint	Нет	FK
starts_at	timestamptz	Нет	—
ends_at	timestamptz	Нет	—
start_price	numeric(12,2)	Нет	—
bid_step	numeric(12,2)	Нет	—
state	varchar(12)	Нет	—
winner_bid_id	bigint	Да	FK
closed_at	timestamptz	Да	—

FOREIGN KEY (listing_id) REFERENCES lf_listings(id).

FOREIGN KEY (permission_id, listing_id, seller_id) REFERENCES lf_auction_permissions(id, listing_id, applicant_id).

PRIMARY KEY (id).

FOREIGN KEY (seller_id) REFERENCES lf_users(id).

FOREIGN KEY (id, winner_bid_id) REFERENCES lf_bids(auction_id, id) DEFERRABLE INITIALLY DEFERRED.

3. Запись аудита: lf_audit_entries

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
actor_id	bigint	Да	FK
action	varchar(80)	Нет	—
entity_table	varchar(80)	Нет	—
entity_id	bigint	Нет	—
details	jsonb	Нет	—
created_at	timestamptz	Нет	—

FOREIGN KEY (actor_id) REFERENCES lf_users(id).

PRIMARY KEY (id).

4. Ставка: lf_bids

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
auction_id	bigint	Нет	FK
bidder_id	bigint	Нет	FK
request_key	uuid	Нет	—
amount	numeric(12,2)	Нет	—
placed_at	timestamptz	Нет	—

FOREIGN KEY (auction_id) REFERENCES lf_auctions(id).

UNIQUE (auction_id, id).

FOREIGN KEY (bidder_id) REFERENCES lf_users(id).

PRIMARY KEY (id).

UNIQUE (request_key).

5. Категория: lf_categories

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
name	varchar(80)	Нет	—
auction_allowed	boolean	Нет	—

UNIQUE (name).

PRIMARY KEY (id).

6. Заявка на передачу: lf_claims

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
listing_id	bigint	Нет	FK
claimant_id	bigint	Нет	FK
evidence	text	Нет	—
state	varchar(12)	Нет	—
created_at	timestamptz	Нет	—
auction_id	bigint	Да	FK

FOREIGN KEY (auction_id) REFERENCES lf_auctions(id).

UNIQUE (auction_id).

FOREIGN KEY (claimant_id) REFERENCES lf_users(id).

UNIQUE (id, listing_id).

FOREIGN KEY (listing_id) REFERENCES lf_listings(id).

PRIMARY KEY (id).

7. Жалоба: lf_complaints

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
reporter_id	bigint	Нет	FK
listing_id	bigint	Да	FK
reported_user_id	bigint	Да	FK
reason	text	Нет	—
state	varchar(12)	Нет	—
moderator_id	bigint	Да	FK
resolution	text	Да	—
created_at	timestamptz	Нет	—

FOREIGN KEY (listing_id) REFERENCES lf_listings(id).

FOREIGN KEY (moderator_id) REFERENCES lf_users(id).

PRIMARY KEY (id).

FOREIGN KEY (reported_user_id) REFERENCES lf_users(id).

FOREIGN KEY (reporter_id) REFERENCES lf_users(id).

8. Диалог: lf_conversations

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
claim_id	bigint	Нет	FK
created_at	timestamptz	Нет	—

FOREIGN KEY (claim_id) REFERENCES lf_claims(id).

UNIQUE (claim_id).

PRIMARY KEY (id).

9. Изображение: lf_listing_images

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
listing_id	bigint	Нет	FK
object_key	varchar(240)	Нет	—
media_type	varchar(20)	Нет	—
size_bytes	integer	Нет	—
position	smallint	Нет	—

FOREIGN KEY (listing_id) REFERENCES lf_listings(id) ON DELETE CASCADE.

UNIQUE (listing_id, "position").

UNIQUE (object_key).

PRIMARY KEY (id).

10. Объявление: lf_listings

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
author_id	bigint	Нет	FK
category_id	bigint	Нет	FK
location_id	bigint	Нет	FK
kind	varchar(5)	Нет	—
title	varchar(120)	Нет	—
description	text	Нет	—
event_at	timestamptz	Нет	—
event_until	timestamptz	Да	—
state	varchar(12)	Нет	—
custodian_org_id	bigint	Да	FK
official_reported_at	timestamptz	Да	—
moderator_id	bigint	Да	FK

Поле	Тип PostgreSQL	NULL	Ключ
moderated_at	timestamptz	Да	—
created_at	timestamptz	Нет	—
search_vector	tsvector	Да	—

FOREIGN KEY (author_id) REFERENCES lf_users(id).

FOREIGN KEY (category_id) REFERENCES lf_categories(id).

FOREIGN KEY (custodian_org_id) REFERENCES lf_organizations(id).

FOREIGN KEY (location_id) REFERENCES lf_locations(id).

FOREIGN KEY (moderator_id) REFERENCES lf_users(id).

PRIMARY KEY (id).

11. Место: lf_locations

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
city	varchar(80)	Нет	—
description	varchar(240)	Нет	—
latitude	numeric(9,6)	Да	—
longitude	numeric(9,6)	Да	—
metro_station	varchar(100)	Да	—
organization_id	bigint	Да	FK

FOREIGN KEY (organization_id) REFERENCES lf_organizations(id).

PRIMARY KEY (id).

12. Сообщение: lf_messages

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
conversation_id	bigint	Нет	FK
sender_id	bigint	Нет	FK

Поле	Тип PostgreSQL	NULL	Ключ
body	text	Нет	—
sent_at	timestamptz	Нет	—

FOREIGN KEY (conversation_id) REFERENCES lf_conversations(id).

PRIMARY KEY (id).

FOREIGN KEY (sender_id) REFERENCES lf_users(id).

13. Уведомление: lf_notifications

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
user_id	bigint	Нет	FK
kind	varchar(40)	Нет	—
body	text	Нет	—
read_at	timestamptz	Да	—
created_at	timestamptz	Нет	—

PRIMARY KEY (id).

FOREIGN KEY (user_id) REFERENCES lf_users(id).

14. Организация: lf_organizations

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
name	varchar(160)	Нет	—
phone	varchar(40)	Нет	—
address	text	Да	—
instructions	text	Нет	—
source_url	text	Нет	—
verified_on	date	Нет	—

UNIQUE (name).

PRIMARY KEY (id).

15. Событие доставки: lf_outbox_events

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
kind	varchar(40)	Нет	—
aggregate_id	bigint	Нет	—
payload	jsonb	Нет	—
attempts	integer	Нет	—
next_attempt_at	timestamptz	Нет	—
delivered_at	timestamptz	Да	—
created_at	timestamptz	Нет	—

PRIMARY KEY (id).

16. Событие оплаты: lf_payment_events

Поле	Тип PostgreSQL	NULL	Ключ
provider_event_id	varchar(100)	Нет	PK
order_id	bigint	Нет	FK
amount	numeric(12,2)	Нет	—
currency	char(3)	Нет	—
received_at	timestamptz	Нет	—

FOREIGN KEY (order_id) REFERENCES lf_payment_orders(id).

PRIMARY KEY (provider_event_id).

17. Заказ оплаты: lf_payment_orders

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
user_id	bigint	Нет	FK
tariff_id	bigint	Нет	FK

Поле	Тип PostgreSQL	NULL	Ключ
request_key	uuid	Нет	—
amount	numeric(12,2)	Нет	—
duration_days	integer	Нет	—
currency	char(3)	Нет	—
state	varchar(12)	Нет	—
created_at	timestamptz	Нет	—
paid_at	timestamptz	Да	—

PRIMARY KEY (id).

UNIQUE (request_key).

FOREIGN KEY (tariff_id) REFERENCES lf_tariffs(id).

FOREIGN KEY (user_id) REFERENCES lf_users(id).

18. Контрольный признак: lf_private_attributes

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
listing_id	bigint	Нет	FK
name	varchar(80)	Нет	—
value	text	Нет	—

FOREIGN KEY (listing_id) REFERENCES lf_listings(id) ON DELETE CASCADE.

UNIQUE (listing_id, name).

PRIMARY KEY (id).

19. Токен обновления: lf_refresh_tokens

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
user_id	bigint	Нет	FK
token_hash	varchar(64)	Нет	—

Поле	Тип PostgreSQL	NULL	Ключ
expires_at	timestamptz	Нет	—
revoked_at	timestamptz	Да	—
created_at	timestamptz	Нет	—

PRIMARY KEY (id).

UNIQUE (token_hash).

FOREIGN KEY (user_id) REFERENCES lf_users(id).

20. Роль: lf_roles

Поле	Тип PostgreSQL	NULL	Ключ
code	varchar(16)	Нет	PK
title	varchar(80)	Нет	—

PRIMARY KEY (code).

21. Период подписки: lf_subscriptions

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
user_id	bigint	Нет	FK
payment_order_id	bigint	Нет	FK
starts_at	timestamptz	Нет	—
ends_at	timestamptz	Нет	—

FOREIGN KEY (payment_order_id) REFERENCES lf_payment_orders(id).

UNIQUE (payment_order_id).

PRIMARY KEY (id).

FOREIGN KEY (user_id) REFERENCES lf_users(id).

22. Тариф: lf_tariffs

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
title	varchar(80)	Нет	—
amount	numeric(12,2)	Нет	—
duration_days	integer	Нет	—
active	boolean	Нет	—

PRIMARY KEY (id).

23. Передача: lf_transfers

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
claim_id	bigint	Нет	FK
finder_confirmed_at	timestamptz	Да	—
owner_confirmed_at	timestamptz	Да	—
moderator_id	bigint	Да	FK
override_reason	text	Да	—
completed_at	timestamptz	Да	—

FOREIGN KEY (claim_id) REFERENCES lf_claims(id).

UNIQUE (claim_id).

FOREIGN KEY (moderator_id) REFERENCES lf_users(id).

PRIMARY KEY (id).

24. Назначение роли: lf_user_roles

Поле	Тип PostgreSQL	NULL	Ключ
user_id	bigint	Нет	PK, FK
role_code	varchar(16)	Нет	PK, FK

PRIMARY KEY (user_id, role_code).

FOREIGN KEY (role_code) REFERENCES lf_roles(code) ON DELETE RESTRICT.

FOREIGN KEY (user_id) REFERENCES lf_users(id) ON DELETE RESTRICT.

25. Пользователь: lf_users

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
email	varchar(254)	Нет	—
display_name	varchar(80)	Нет	—
password_hash	varchar(100)	Нет	—
state	varchar(12)	Нет	—
email_confirmed_at	timestamptz	Да	—
token_version	integer	Нет	—
created_at	timestamptz	Нет	—

UNIQUE (email).

PRIMARY KEY (id).

26. Одноразовый токен: lf_verification_tokens

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
user_id	bigint	Нет	FK
purpose	varchar(16)	Нет	—
token_hash	varchar(64)	Нет	—
expires_at	timestamptz	Нет	—
consumed_at	timestamptz	Да	—
created_at	timestamptz	Нет	—

PRIMARY KEY (id).

UNIQUE (token_hash).

FOREIGN KEY (user_id) REFERENCES lf_users(id).

27. Проверка профиля: lf_verifications

Поле	Тип PostgreSQL	NULL	Ключ
id	bigint	Нет	PK
user_id	bigint	Нет	FK
state	varchar(12)	Нет	—
reviewer_id	bigint	Да	FK
reason	text	Да	—
created_at	timestamptz	Нет	—
reviewed_at	timestamptz	Да	—

PRIMARY KEY (id).

FOREIGN KEY (reviewer_id) REFERENCES lf_users(id).

FOREIGN KEY (user_id) REFERENCES lf_users(id).

9. Реализация в PostgreSQL

9.1. Окружение и размещение

Работа установлена 03.10.2026 на helios.cs.ifmo.ru, SSH-порт 2222, учётная запись s465826. Сервер работает под FreeBSD 14.5-STABLE. Доступны PostgreSQL 18.3 и OpenJDK 21.0.4. Java понадобится для следующего этапа; на данном этапе выполняются SQL-скрипты и проверки модели данных.

Подключение к PostgreSQL: хост pg, база studs, схема s465826. У аккаунта есть CREATE в своей схеме, но нет CREATE для базы studs. Поэтому новая база или схема на helios не создаётся. Все таблицы, функции, индексы, триггеры и представление курсовой имеют префикс lf_. Данные другой лабораторной остаются в её существующих объектах.

Каталог на сервере — /home/studs/s465826/poteryashki-course. При создании схемы скрипт использует отдельную транзакцию и ON_ERROR_STOP; повторный запуск на уже созданных объектах останавливается. Удаление проверяет маркер курсовой у таблицы lf_users и

перечисляет объекты явно, без DROP SCHEMA и без CASCADE. Внешняя зависимость вызовет ошибку и откат удаления.

9.2. Средства DDL

Первичные ключи обеспечивают идентификацию строк. Внешние ключи исключают ссылки на несуществующих пользователей, объявления и заявки. Большинство связей используют RESTRICT/NO ACTION; физическое удаление сущности с историей запрещено. Для изображений и контрольных признаков допускается каскадное удаление вместе с черновиком объявления, если на объявление нет препятствующих внешних ссылок.

CHECK ограничивает допустимые статусы, положительные цены и шаг ставок, интервал подписки и аукциона, размер изображения до 5 МиБ, позицию фотографии от 1 до 5, координаты и совместное наличие широты и долготы. При отклонении проверки или допуска обязательна непустая причина. У жалобы ровно один объект: объявление либо пользователь.

Пример связи допуска с лотом:

```
FOREIGN KEY (permission_id, listing_id, seller_id)
REFERENCES lf_auction_permissions(id, listing_id, applicant_id)
```

Пример ограничения ставки-победителя:

```
FOREIGN KEY (id, winner_bid_id)
REFERENCES lf_bids(auction_id, id)
DEFERRABLE INITIALLY DEFERRED
```

Проверки, зависящие от текущего времени или других строк, вынесены в триггеры и функции. Например, CHECK не применяется для запрета ставки после окончания торгов: такая проверка должна выполняться при каждой операции под блокировкой.

10. Триггеры и целостность бизнес-процессов

Группа триггеров	Что проверяется	Связанные прецеденты
lf_user_guard, lf_user_block_auctions	Изменение пароля или статуса увеличивает token_version; блокировка продавца либо участника приостанавливает незавершённые торги	UC-01, UC-04, UC-06

Группа триггеров	Что проверяется	Связанные прецеденты
lf_verification_guard	Роль проверяющего, подтверждённая почта; одобрение переводит профиль в verified	UC-01
lf_payment_guard, lf_subscription_guard	Снимок тарифа, неизменность заказа; период только по оплаченному заказу; отсутствие пересечения периодов пользователя	UC-01, UC-08
lf_listing_guard	Допуск автора, отсутствие будущего события, права модератора, запрет самомодерации и повторная модерация существенных изменений	UC-02
lf_claim_guard	Находка опубликована, заявитель не автор, допустимые переходы статуса; новая претензия приостанавливает торги	UC-03, UC-05
lf_listing_reserved, lf_claim_reserved	В конце транзакции резерв соответствует ровно одной принятой заявке; возврат подтверждён передачей, продажа — завершённым аукционом	UC-03, UC-05
lf_message_guard, lf_transfer_guard	Сообщение от участника либо служебной роли при открытой жалобе на эту находку; передача относится к принятой заявке; закрытая передача неизменяема; служебное решение требует модератора	UC-03, UC-04
lf_permission_guard	Нет документов и вещей организации; проверка основания, официальной даты и роли; реквизиты рассмотренного допуска зафиксированы	UC-05
lf_auction_guard, lf_bid_guard	Лот соответствует допуску и актуальному объявлению; допустимые статусы, время	UC-05

Группа триггеров	Что проверяется	Связанные прецеденты
	торгов, шаг, подписка; ставка продавца запрещена	
lf_*_audit и lf_audit_ immutable	Решения фиксируются без паролей, токенов и закрытых признаков; UPDATE, DELETE и TRUNCATE аудита запрещены	UC-01–08

Триггеры стоят на таблицах и действуют при прямом INSERT/UPDATE, а не только при вызове прикладных функций. Отложенные проверки выполняются при COMMIT. Отдельный частичный уникальный индекс немедленно исключает второй действующий резерв и второй незавершённый аукцион.

Операции возврата, ставок и закрытия сначала блокируют строку объявления, затем строку конкретного процесса. Активация подписки блокирует пользователя и заказ. Это сериализует конкурирующие изменения. После ожидания блокировки используется clock_timestamp(), поэтому истёкшее за время ожидания окно торгов не позволяет принять ставку.

Блокировка пользователя затрагивает несколько агрегатов. При столкновении с другой транзакцией PostgreSQL может вернуть SQLSTATE 40P01; прикладной слой должен откатить и ограниченно повторить всю транзакцию. Ошибка блокировки не обходится частичным повтором отдельных SQL-команд. Ошибки P0001 отражают бизнес-условие, 23505 — нарушение уникальности, 23514 — CHECK, 23503 — внешний ключ.

Роль владельца схемы технически может отключить триггеры или удалить объекты. Проверки рассчитаны на штатные DML-операции. В следующем этапе Spring Security удостоверяет пользователя и передаёт его идентификатор в функции; сам параметр p_user не является средством аутентификации БД. Функции используют SECURITY INVOKER и закреплённый search_path, не повышают права вызывающего.

11. Скрипты создания, удаления и заполнения

Файл	Назначение
database/create.sql	Транзакционная установка таблиц, логики и индексов в указанную схему
database/01_tables.sql	27 таблиц, типы столбцов, ключи и CHECK
database/02_logic.sql	PL/pgSQL, триггеры и публичное представление
database/03_indexes.sql	Индексы для бизнес-операций и ограничений
database/04_handover.sql	Связь дела передачи с аукционом, функции передачи победителю, ограничения отмены и доступа к диалогу; применяется также при обновлении
database/05_search.sql	Единый поиск по доступным публичным статусам и совместимый поиск активных объявлений
database/upgrade_web.sql	Транзакционное обновление существующего набора lf_ с проверкой маркера; восстановление недостающих дел передачи без удаления данных
database/seed.sql	Синтетические пользователи, находки, потеря, метро, возврат, подписки, аукцион и ставки
database/drop.sql	Удаление исключительно объектов курсовой с проверкой маркера и откатом при ошибке
database/create_database.sql	Создание отдельной локальной БД poteryashki_course и схемы poteryashki
database/drop_database.sql	Удаление только отдельной локальной БД poteryashki_course
database/test.sql	50 проверок целостности в транзакции с ROLLBACK
database/explain.sql	Временный набор из 10 000 объявлений и EXPLAIN ANALYZE
tooling: команда db	Общая команда запуска SQL для helios
tooling: команда verify-db	Изолированный полный цикл установки/удаления и проверка двух соединений

Команды psql запускаются на helios из каталога курсовой. Проверка verify-db запускается локально через SSH; пароль вводится скрыто:

```
java -Xmx256m -jar tooling/build/libs/course-tools.jar db create
java -Xmx256m -jar tooling/build/libs/course-tools.jar db seed
java -Xmx256m -jar tooling/build/libs/course-tools.jar db test
java -Xmx256m -jar tooling/build/libs/course-tools.jar verify-db
java -Xmx256m -jar tooling/build/libs/course-tools.jar db explain
```

Команда java -Xmx256m -jar tooling/build/libs/course-tools.jar db drop применяется только для сброса собственной курсовой; перед последующим create повторно запускается seed. Она не удаляет общую базу studs. Локальная установка требует роли, имеющей CREATE DATABASE:

```
psql -X -v ON_ERROR_STOP=1 -d postgres -f database/create_database.sql
psql -X -d poteryashki_course -v schema=poteryashki -f database/seed.sql
psql -X -d postgres -f database/drop_database.sql
```

Учебные email используют зарезервированный домен example.invalid. Значения цен, признаки и фотографии демонстрационные. В базе есть роли user/moderator/admin, подтверждённые и недопущенные профили, оплаченные периоды, семь объявлений, справочник метро, контрольные признаки, обращения и диалог, завершённая передача, жалоба, два допуска, один активный аукцион и две ставки. Пароли хранятся как BCrypt-хеши учебного значения; одноразовые и refresh-токены представлены синтетическими хешами. Реальные платежи, письма и файлы MinIO на втором этапе не выполняются.

12. Функции и процедуры PL/pgSQL

Подпрограмма	Параметры и результат	Алгоритм и назначение
lf_consume_token	Хеш, назначение; user_id	Блокирует одноразовый токен, проверяет срок и повтор использования; подтверждает email либо разрешает продолжить смену пароля в той же транзакции
lf_activate_subscription	Заказ, ID события, сумма, валюта; subscription_id	Сверяет заказ и callback, исключает повтор, добавляет период от максимума текущего времени и конца оплаченного срока; сохраняет уведомление и outbox
lf_reserve_found	Заявка, нашедший; transfer_ id	Блокирует находку, проверяет авторство, подписку и отсутствие торгов, принимает заявку, создаёт резерв и передачу; вещь организации выдаёт сама организация
lf_confirm_transfer	Передача, участник; boolean	Участник подтверждает свою сторону; два подтверждения переводят заявку в fulfilled, находку в returned. Для согласованной

Подпрограмма	Параметры и результат	Алгоритм и назначение
		передачи продление подписки не требуется
lf_resolve_return	Передача, модератор, причина; void	Завершает согласованный возврат мотивированным решением сотрудника, фиксирует основание в передаче и событие аудита
lf_place_bid	Лот, участник, сумма, UUID; bid_id	Блокирует находку и лот, проверяет повтор ключа, время, подписку и минимальную цену; вставляет ставку и событие доставки
lf_finalize_auction	Лот; winner_bid_id либо NULL	Выбирает победителя, завершает лот и вызывает создание дела передачи; повтор возвращает сохранённого победителя; без ставок вещь остаётся опубликованной
lf_prepare_auction_ handover	Завершённый лот; claim_id либо NULL	Создаёт одну принятую заявку победителя, диалог и передачу; резервирует вещь. Повтор возвращает прежнее дело и не меняет подтверждения. Истечение подписки после ставки не отменяет результат торгов
lf_close_due_auctions	Размер пакета от 1 до 1000	Процедура обработки просроченных торгов. Приостановленные лоты не закрывает автоматически
lf_search_listings	Текст, тип, категория, место, время, курсор, лимит	Русский полнотекстовый поиск только опубликованных объявлений; лимит 1–100; порядок id DESC и курсор без OFFSET
lf_search_listings_state	Те же параметры и статус	Фильтр опубликованных, зарезервированных, переданных либо проданных вещей; черновики, отклонённые и непроверенные объявления никогда не возвращаются

Вспомогательные lf_assert_user и lf_assert_moderator проверяют допуск профиля, оплаченный период и служебную роль. UUID ключа запроса ставки исключает повторное создание при повторной отправке клиентом. Повтор с тем же ключом и другим содержимым отклоняется. Событие оплаты также сверяется с исходным заказом, суммой и валютой.

Пример вызова ставок в одной транзакции:

```
BEGIN;
SET LOCAL search_path TO s465826, pg_catalog;
SELECT lf_place_bid(1, 4, 1200.00,
'50000000-0000-0000-0000-000000000001');
COMMIT;
```

Идентификаторы примерные; повторять этот пример на учебном наборе можно только при доступном лоте и допустимой цене. В приложении параметры задаются через JDBC PreparedStatement или нативный запрос JPA. Для аудита прикладная транзакция задаёт set_config('lf.actor_id', user_id::text, true); фоновая операция оставляет actor_id пустым. Обработчик платежей вызывается после проверки подлинности серверного события выбранного провайдера.

13. Индексы и обоснование

13.1. Индексы по сценариям

Первичные ключи и UNIQUE создают индексы автоматически: email, хеши токенов, ключи запросов, ID внешнего события, соответствие передачи и диалога заявке. Внешний ключ PostgreSQL сам по себе не создаёт индекс на ссылающемся столбце; нужные индексы добавлены отдельно.

Сценарий	Индексы из 03_indexes.sql	Польза
Проверка профиля и ролей: UC-01, UC-06	lf_verification_pending_uq, lf_roles_reverse_idx	Один открытый запрос на пользователя; выбор сотрудников по роли
Авторизация и токены: UC-01	lf_refresh_user_idx, lf_verification_token_user_idx	Выбор действующих сессий и неиспользованных токенов для отзыва/проверки
Подписка: UC-01	lf_subscription_access_idx, lf_payment_user_idx, lf_payment_event_order_idx	Проверка срока, поиск конца оплаченного периода, кабинет платежей
Кабинет и модерация: UC-02	lf_listing_author_idx, lf_listing_moderation_idx	Объявления автора и небольшая очередь ожидающих записей
Поиск: UC-03	lf_listing_search_idx, lf_listing_place_idx, lf_listing_feed_idx, lf_listing_text_idx	Фильтры типа/категории/времени; место и диапазон; курсор ленты; GIN по русскому tsvector
Связи объявлений: UC-02, UC-03, UC-07/15	lf_location_organization_idx, lf_listing_category_idx, lf_listing_location_idx, lf_listing_custodian_idx	Проверка ссылок и выбор связанных записей при работе со справочниками

Сценарий	Индексы из 03_indexes.sql	Польза
Возврат и диалог: UC-03	lf_claim_reserve_uq, lf_claim_open_uq, lf_claim_claimant_idx, lf_claim_listing_idx, lf_message_history_idx, lf_message_sender_idx	Один резерв; отсутствие повторной открытой заявки; обращения пользователя и последовательная история сообщений
Передача результата торгов: UC-05	lf_claim_auction_unique	Не более одного дела передачи на аукцион; быстрый поиск для повторного завершения
Окончание подписки: UC-01	lf_expiry_event_once	Частичный UNIQUE исключает повторное событие при параллельных обработчиках
Жалобы: UC-04	lf_complaint_queue_idx, lf_complaint_reporter_idx, lf_complaint_listing_idx, lf_complaint_user_idx	Служебная очередь и разбор обращений по объекту
Допуск: UC-05	lf_permission_listing_idx, lf_permission_applicant_idx	История оснований для вещи и продавца
Торги: UC-05	lf_auction_open_uq, lf_auction_due_idx, lf_auction_seller_idx, lf_auction_permission_idx, lf_bid_winner_idx, lf_bid_user_idx	Один незавершённый лот; выбор просроченных торгов; максимальная ставка по auction_id/amount DESC/id; история участника
Уведомления и аудит: UC-01–08	lf_notification_unread_idx, lf_outbox_pending_idx, lf_audit_entity_idx, lf_audit_actor_idx	Непрочитанные сообщения, очередь доставки, история сущности и действия сотрудника

Частичные индексы включают только востребованные статусы. Они занимают меньше места и обслуживают очередь без просмотра завершённых записей. Уникальные частичные индексы являются также средством целостности. Составной индекс располагает поля равенства перед полем диапазона; порядок полей согласован с запросом.

13.2. Проверка планов

На helios выполнен EXPLAIN (ANALYZE, BUFFERS) на семи исходных и 10 000 временных объявлениях. Из временных 20% опубликованы, остальные архивные. Термин «термос» встречается в десяти опубликованных строках. После вставки очищается pending list GIN; в рабочей базе этим также

занимается обслуживание индекса и autovacuum. Это проверка конкретных запросов, а не нагрузочный тест интерфейса или гарантия времени отклика.

С индексом полнотекстовый запрос использует Bitmap Index Scan по lf_listing_text_idx. После временного удаления GIN тот же запрос просматривает опубликованные объявления через B-tree и отбрасывает неподходящие строки. Поиск места и временного интервала использует lf_listing_place_idx; курсор ленты — lf_listing_feed_idx. Для двух исходных ставок PostgreSQL выбирает Seq Scan и сортировку: на столь малой таблице индекс победителя не обязательно выгоднее.

Полные планы, число строк, буферы и фактическое время сохранены в docs/part2/validation/indexes.txt. Временное удаление индекса и данные откатываются ROLLBACK; после отката обновляется статистика исходного набора. Выбор индекса зависит от объёма, распределения данных и конкретных параметров запроса.

14. Проверка реализации и демонстрация

14.1. Проверки целостности

Файл test.sql выполнил 50 проверок с ожидаемыми успехами и ошибками. Проверены уникальность почты и ролей, допуск и блокировка, условия модерации, сумма заказа, повтор события оплаты, сохранение оплаченного остатка, пересечение подписок, время события и интервал потери, лимит файлов, чужой диалог, дубликат заявки, второй резерв, подтверждения возврата и решение модератора.

Для аукционов проверены запрет продажи документов и вещей организации, официальная дата шестимесячного срока, запрет ставки продавца, шаг ставки, повтор ключа с другим содержимым, второй открытый лот, преждевременное завершение и приостановка после заявления владельца или блокировки участника. Также проверены неизменность истории и аудита, повтор одноразового токена и отсутствие закрытых полей в публичном представлении.

Проверки выполняются в транзакции и откатывают свои изменения. Последовательности identity могут получить пропуски: их значения не

являются счётчиком строк и не откатываются вместе с транзакцией. Результат 50 PASS и ROLLBACK сохранён в docs/part2/validation/integrity.txt.

14.2. Конкурентные операции и жизненный цикл

Скрипт validate_database.py создаёт отдельный проверочный набор lfcheck_* в той же доступной схеме, применяя те же определения с другим префиксом. Если такие объекты уже существуют, скрипт останавливается и не удаляет их автоматически. После успешного создания он гарантированно запускает собственное удаление в finally. Основные lf_* и таблицы лабораторной в испытании не изменяются.

Испытание	Результат
Две заявки одновременно получают резерв одной находки	Один резерв принят, второе соединение получает ошибку недоступности
Два участника одновременно предлагают одинаковую допустимую цену	Принята одна ставка, вторая отклонена после повторной проверки минимума
Два соединения обрабатывают одно событие оплаты нового заказа	Оба получают успешный результат; создан один период подписки
Короткий аукцион достигает времени окончания	Процедура определяет победителя; повторное завершение возвращает тот же ID
Создание, заполнение, проверка и удаление тестового набора	Полный цикл выполнен; объекты lfcheck_* удалены
Сравнение состава внешних объектов схемы до и после	Состав lf_* и объектов лабораторной сохранён

Протокол находится в docs/part2/validation/concurrency.txt. Сведения об окружении и исходные объекты другой работы перечислены в environment.txt. Проверка касается сохранности структуры; операции скриптов не обращаются к данным другой лабораторной. Реализация Spring MVC/Thymeleaf, SMTP, MinIO и внешнего платёжного адаптера на втором этапе ещё не выполнялась, поэтому проверки этих компонентов не представлены как успешные.

14.3. Порядок показа этапа 2

1. Показать ER-модель, связь пользователей и ролей M:N и даталогический словарь.

2. Подключиться к helios и открыть ~/poteryashki-course. Показать текущую БД и список lf_*.

3. Выполнить java -Xmx256m -jar tooling/build/libs/course-tools.jar db test и показать 50 результатов PASS.

4. Выполнить java -Xmx256m -jar tooling/build/libs/course-tools.jar verify-db и показать результаты двух соединений и удаления проверочного набора.

5. Выполнить java -Xmx256m -jar tooling/build/libs/course-tools.jar db explain и показать выбранные индексы.

6. Объяснить функции резервирования, оплаты и ставок, ограничения и повторную обработку запроса.

15. Вывод по второму этапу

Разработана модель хранения данных информационной системы «Потеряшки». Созданы 27 таблиц, отношение пользователей и ролей M:N, внешние ключи, CHECK, уникальные и частичные индексы, триггеры и функции PL/pgSQL. Модель реализована на PostgreSQL сервера helios и заполнена синтетическими данными. Проверены 50 условий целостности, конкурентные операции, завершение торгов и полный цикл создания/удаления изолированного набора.

Следующий этап — реализация слоя хранения и предметных сервисов Spring Boot с вызовом подготовленных функций БД. Пользовательские страницы Spring MVC/Thymeleaf добавляются после бизнес-логики. Подписка, модерация и проверка принадлежности вещи остаются раздельными процессами. Справочник метро хранит официальные инструкции, а наличие записи не подтверждает физическое хранение вещи в организации.

Дополнительные источники к этапу 2

Дата обращения: 03.10.2026.

1. PostgreSQL 18. Constraints: https://www.postgresql.org/docs/18/ddl-constraints.html

2. PostgreSQL 18. Trigger Functions: https://www.postgresql.org/docs/18/plpgsql-trigger.html

3. PostgreSQL 18. Explicit Locking: https://www.postgresql.org/docs/18/explicit-locking.html

4. PostgreSQL 18. Partial Indexes: https://www.postgresql.org/docs/18/indexes-partial.html

5. PostgreSQL 18. EXPLAIN: https://www.postgresql.org/docs/18/using-explain.html

6. MyFilmList. Отчёт по второму этапу, версия 74d1928973cae329d6ca651bac2ffde445a43de0: https://github.com/maxbarsukov/MyFilmList/blob/74d1928973cae329d6ca651bac2ffde445a43de0/docs/part2/report.pdf

Приложение В. Соответствие заданию этапа 2

Пункт задания	Раздел / файл	Выполненный результат
1. ER-модель, не менее 10 сущностей и M:N	7, model/er.mmd	27 сущностей и отношений; users ↔ roles через user_roles
2. Даталогическая модель	8, model/logical.mmd, model/catalog.json	Типы столбцов, NULL, PK, FK, UNIQUE; словарь реальной установки
3. Реализация PostgreSQL	9, database/create.sql	Установка в studs/s465826 на helios, префикс lf_
4. DDL и триггеры целостности	9–10, 01_tables.sql, 02_logic.sql	Декларативные ограничения и проверки межтабличных условий
5. Создание, удаление, тестовые данные	11, create.sql, drop.sql, seed.sql	Скрипты для общей схемы; отдельные create/drop_database.sql для локальной БД
6. PL/pgSQL для критических запросов	12, 02_logic.sql	Оплата, резерв, возврат, решение спора, ставки, завершение, поиск, токены
7. Индексы и их обоснование	13, 03_indexes.sql, explain.sql	Индексы по прецедентам и реальные планы запросов

Пункт задания	Раздел / файл	Выполненный результат
8. Отчёт	Совместный отчёт этапов 1–2	Текст, схемы, словарь и результаты проверок

Задание третьего этапа

1. Изобразить диаграмму классов, представляющую общую архитектуру системы.

2. Реализовать уровень хранения на основе базы данных второго этапа.

3. Использовать на уровне хранения функции и процедуры PL/pgSQL второго этапа. Не заменять их аналогичными запросами или алгоритмами приложения.

4. Реализовать уровень бизнес-логики на основе процессов первого этапа и уровня хранения.

5. Составить отчёт.

16. Архитектура реализации

16.1. Состав приложения

Реализация третьего этапа находится в каталоге backend. Приложение представляет собой модульный монолит на Java 17 и Spring Boot 3.5.16. Для доступа к данным используются Spring Data JPA, Hibernate и JdbcTemplate. Lombok формирует конструкторы и методы чтения свойств; Jakarta Validation проверяет входные данные объявления. Пароли обрабатываются BCrypt из Spring Security Crypto. Сборка выполняется Gradle Wrapper 8.14.3.

Основные технологические решения соответствуют проекту MyFilmList: Java, Spring Boot, Hibernate/JPA, PostgreSQL, Lombok и Gradle. Версия Spring Boot обновлена в пределах ветки 3.x. Пользовательский интерфейс на Thymeleaf и транспортный слой Spring MVC относятся к следующему этапу. В третьем этапе сервисы вызываются из Java-кода, интеграционных тестов и демонстрационного ApplicationRunner. Приложение не открывает HTTP-порт.

Приложение разделено на пакеты domain, persistence и service. Классы domain отображают таблицы второго этапа. Репозитории persistence выполняют чтение, обычные операции записи и вызовы функций PostgreSQL. Классы service проверяют полномочия участников и управляют транзакциями бизнес-процессов. PoteryashkiApplication собирает Spring-контекст, а DemoRunner запускает демонстрацию на установленных данных.

Таблица 17 — Основные классы реализации

Класс	Назначение
PoteryashkiApplication	Запуск приложения и конфигурация BCrypt
DemoRunner	Демонстрация входа, поиска, справочника и уведомлений через сервисы
AccountService	Регистрация, подтверждение почты, проверка профиля, вход и восстановление пароля
SubscriptionService	Создание платёжного заказа, учебная оплата и периоды подписки
ListingService	Объявления, модерация, поиск, закрытые признаки, метаданные фотографий и справочник организаций
ReturnService	Заявки на находку, сообщения, резервирование и подтверждение передачи
AuctionService	Правовое основание продажи, создание лота, ставки и завершение торгов
AdministrationService	Жалобы, блокировки, роли, тарифы, справочник, уведомления и аудит
OutboxService	Транзакционная доставка событий во внутренние уведомления
AccessPolicy	Проверка пользователя, подписки и служебных ролей; контекст аудита
DatabaseFunctions	Вызовы существующих функций и процедуры PL/pgSQL
SqlStore	Параметризованный CRUD и явные проекции записей
UserRepository, ListingRepository	Чтение неизменяемых JPA-сущностей
SqlNames, TableNamingStrategy	Общий префикс таблиц и изоляция интеграционных проверок

16.2. Диаграммы классов

Общая диаграмма показывает предметные сервисы, общий контроль доступа и классы хранения. На детальных диаграммах показаны репозитории, сущности и процессы возврата и аукциона. Исходники PlantUML позволяют менять состав классов и зависимостей; обзорное изображение построено непосредственно по этому исходнику.

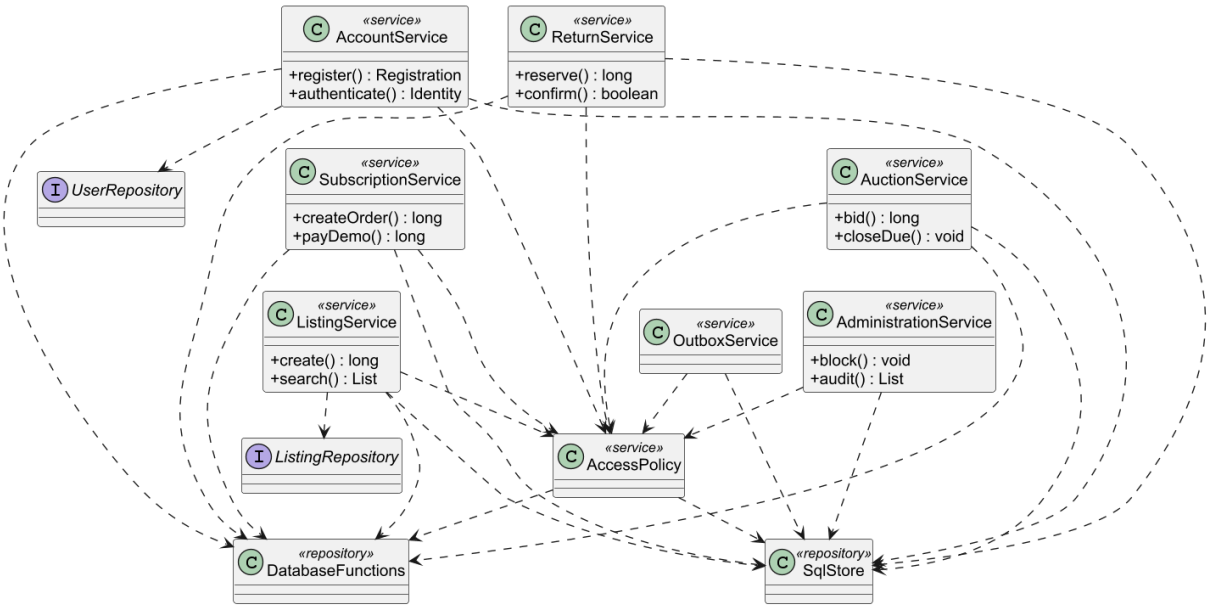


Рисунок 11 — Общая диаграмма классов

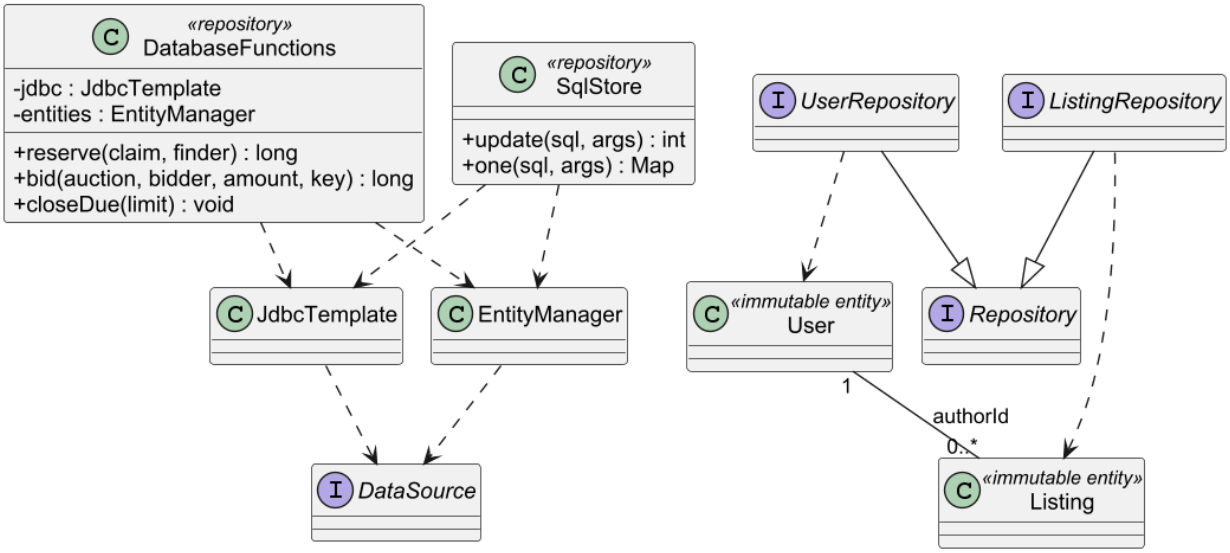


Рисунок 12 — Классы уровня хранения

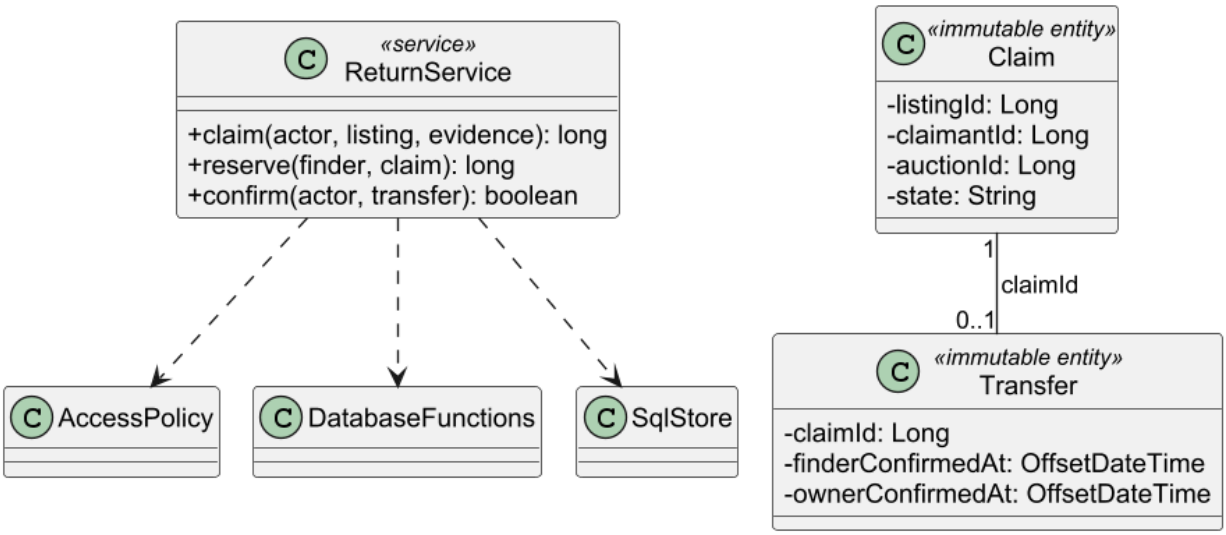


Рисунок 13 — Классы процесса возврата

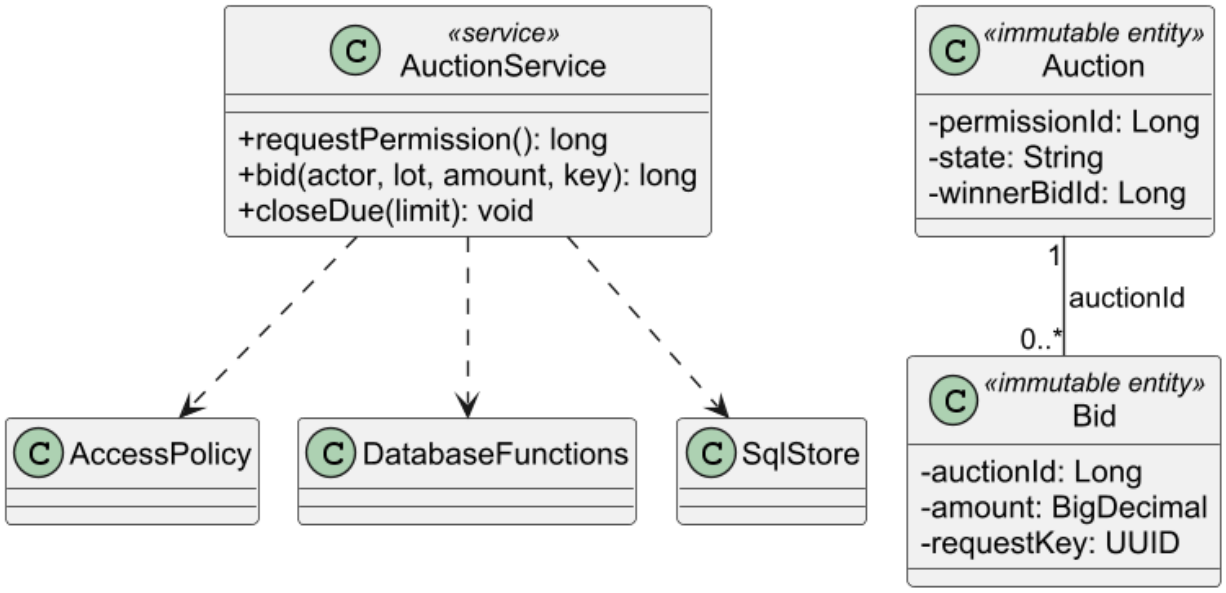


Рисунок 14 — Классы процесса аукциона

Стрелка зависимости означает использование класса. Пустой треугольник в исходниках PlantUML показывает наследование интерфейса Repository. Связи между сущностями обозначают внешние ключи базы данных, а не автоматическую загрузку графа объектов Hibernate. Например, Claim связан с Transfer как 1:0..1, а Auction с Bid как 1:0..*. Составной ключ UserRole сохраняет отношение M:N пользователей и ролей.

17. Уровень хранения

17.1. Отображение базы данных

В пакете domain определены 27 JPA-сущностей, соответствующих всем таблицам второго этапа. UserRole использует составной идентификатор UserRoleKey. Для bigint применяется Long, для integer — Integer, для smallint — Short, для денежных значений numeric — BigDecimal. Моменты времени представлены OffsetDateTime; обмен выполняется в UTC. Для jsonb задан JDBC-тип JSON. Вычисляемый поисковый вектор формируется PostgreSQL и не используется как изменяемое поле JPA.

Все сущности отмечены Hibernate-аннотацией Immutable. UserRepository и ListingRepository предоставляют только операции чтения и не имеют save или delete. Запись выполняется явно через SqlStore и DatabaseFunctions. Это отделяет объектные проекции от переходов состояний и исключает сохранение критичных изменений через обычный JPA-репозиторий.

Таблица 18 — Группы JPA-сущностей

Группа	Классы
Аккаунты и полномочия	User, Role, UserRole, RefreshToken, Verification, VerificationToken
Подписки и платежи	Tariff, PaymentOrder, PaymentEvent, Subscription
Справочники и объявления	Organization, Location, Category, Listing, ListingImage, PrivateAttribute
Возврат вещи	Claim, Conversation, Message, Transfer, Complaint
Аукцион	AuctionPermission, Auction, Bid
События и аудит	Notification, OutboxEvent, AuditEntry

SqlStore использует JdbcTemplate и параметры PreparedStatement. Наименование префикса проверяется по ограниченному шаблону; ввод пользователя не включается в SQL-идентификаторы. На helios применяется префикс lf_, а в проверочном наборе — lf3check_. JPA и JDBC получают одинаковые значения префикса и работают с одним DataSource.

После записи через JDBC контекст EntityManager очищается. Поэтому JPA-чтение в той же транзакции не возвращает устаревшее состояние объекта, изменённого SQL-функцией или триггером. Денежные значения не

переводятся в double и проверяются на положительность, размер и число десятичных знаков.

17.2. Использование PL/pgSQL второго этапа

Для критичных операций DatabaseFunctions вызывает функции PostgreSQL второго этапа. Продление подписки, резервирование, подтверждение передачи, приём ставки и определение победителя выполняются в базе данных в одной транзакции.

Таблица 19 — Соответствие Java-методов операциям PostgreSQL

Метод DatabaseFunctions	**Операция PostgreSQL**	**Где используется**
assertUser	lf_assert_user	AccessPolicy: подтверждение, состояние и подписка
assertModerator	lf_assert_moderator	AccessPolicy: служебные операции
consumeToken	lf_consume_token	AccountService: подтверждение почты и восстановление
activateSubscription	lf_activate_subscription	SubscriptionService: учебная оплата
reserve	lf_reserve_found	ReturnService: согласование возврата
confirm	lf_confirm_transfer	ReturnService: подтверждение передачи
resolveReturn	lf_resolve_return	ReturnService: решение модератора
bid	lf_place_bid	AuctionService: ставка и повтор запроса
finalizeAuction	lf_finalize_auction	AuctionService: завершение конкретного лота
closeDue	lf_close_due_auctions	AuctionService: пакетное завершение торгов
search	lf_search_listings	ListingService: поиск и фильтрация
searchByState	lf_search_listings_state	ListingService: поиск с выбранным публичным статусом

Вызов резервирования содержит только обращение к функции:

```
public long reserve(long claim, long finder) {
return number("select lf_reserve_found(?,?)", claim, finder);
}
```

Ставка также выполняется через функцию:

```
public long bid(long auction, long bidder,
BigDecimal amount, UUID key) {
return number("select lf_place_bid(?,?,?,?)",
auction, bidder, amount, key);
}
```

Процедура завершения вызывается командой CALL. Предел пакета проверяет сама процедура. На уровне сервисов реализованы полномочия инициатора и типы входных значений; расчёт победителя и изменения связанных записей остаются внутри PostgreSQL.

17.3. Транзакции и целостность

Публичные методы сервисов выполняются с Transactional. JpaTransactionManager объединяет JPA и JDBC в одной транзакции. Вызов функции, изменение обычных записей, события outbox и аудит завершаются совместно. Ошибка ограничения или функции вызывает откат. Deferred-ограничения второго этапа проверяются при завершении транзакции; в положительных интеграционных сценариях дополнительно выполняется SET CONSTRAINTS ALL IMMEDIATE.

Автор действия передаётся командой set_config с третьим параметром true. Значение lf.actor_id существует только внутри текущей транзакции и не переносится другому пользователю через пул соединений. Для фоновой задачи автор отсутствует. Триггеры второго этапа сохраняют изменения в неизменяемом журнале.

Для резервирования и ставок сохраняется порядок блокировок SQL-функций: объявление, затем связанная заявка или аукцион. При оплате сначала блокируется пользователь, после чего проверяются его состояние и заказ. Это исключает взаимную блокировку двух подтверждений оплаты при попытке повысить общую блокировку пользователя до исключительной.

Повтор принятой ставки дополнительно проходит проверку актуальной подписки и состояния пользователя. Поэтому заблокированный участник не получает успешный результат повторного запроса только из-за ранее сохранённого ключа. Ключи заявок на оплату и ставок представлены UUID; повтор с другим содержимым отклоняется.

18. Уровень бизнес-логики

18.1. Учётные записи и подписка

При регистрации AccountService нормализует адрес, проверяет имя и пароль, сохраняет BCrypt-хеш и роль user. Одновременно создаются заявка на проверку профиля и одноразовый токен подтверждения почты. В базе хранится SHA-256-хеш токена. Исходное значение является внутренним результатом сервиса для почтового адаптера и не публикуется в каталоге или журнале.

После подтверждения почты профиль рассматривает независимый модератор. Одного подтверждения адреса недостаточно для платных операций: требуется состояние verified. После отказа пользователь может повторно подать профиль, если новая заявка ещё не ожидает рассмотрения.

authenticate проверяет BCrypt и возвращает Identity с идентификатором пользователя и версией сессии. checkIdentity повторно проверяет блокировку и token_version. При смене пароля версия увеличивается триггером, а refresh-токены отзываются в той же транзакции. Подписание access JWT, работа Cookie и HTTP-фильтр будут реализованы на транспортном этапе; Identity используется здесь как внутренний результат проверки учётных данных.

SubscriptionService фиксирует параметры выбранного тарифа в платёжном заказе. Повтор createOrder с тем же ключом возвращает прежний заказ только для того же пользователя и тарифа. Учебный payDemo проверяет владельца и берёт сумму из сохранённого заказа. Он не принимает произвольную сумму от пользователя и не является открытым callback внешнего провайдера. Продление выполняет lf_activate_subscription: сохраняет событие оплаты один раз и добавляет срок после уже оплаченного периода.

18.2. Объявления и справочник метрополитена

ListingService создаёт объявления только для проверенного подписчика. Время и место представлены идентификаторами справочников и моментами события; проверяются тип объявления, заголовок и описание. На пользователя допускается не более десяти новых объявлений за календарные сутки по времени PostgreSQL. Пользователь отправляет черновик на проверку, а независимый модератор публикует или отклоняет его.

Существенное редактирование возвращает объявление в состояние pending посредством триггера. Нельзя менять тип и автора, редактировать вещь во время незавершённого аукциона или после согласования передачи. Закрытые признаки доступны только автору. Публичная карточка строится по явной проекции lf_public_listings и не содержит email, хешей, доказательств заявителей или закрытых признаков.

attachImage сохраняет метаданные уже проверенного объекта: принадлежность ключа объявлению, JPEG/PNG/WebP, размер до 5 МиБ и позицию от 1 до 5. Установка MinIO, загрузка двоичных данных, проверка содержимого изображения и удаление EXIF относятся к файловому адаптеру последующего этапа. Наличие строки метаданных не означает, что двоичный файл уже загружен.

Поиск выполняется через lf_search_listings с текстом, видом, категорией, местом, временем и курсором. Метод searchByState вызывает lf_search_listings_state для отбора по публичному статусу; черновики и скрытые объявления эта функция не возвращает. Справочник организаций содержит официальный источник, дату проверки и признак устаревания после 30 суток. Для Петербургского метрополитена возвращаются телефон и инструкция, подготовленные на втором этапе. Если вещь хранит организация, ReturnService направляет владельца в эту организацию и не создаёт обычный процесс выдачи пользователем.

18.3. Заявки, сообщения и возврат

Заявитель указывает закрытые доказательства владения. Создание заявки и диалога выполняется в одной транзакции. Доступ к сообщениям проверяется как при чтении, так и при записи: участниками являются автор находки и заявитель. Модератор получает доступ при открытой жалобе на эту находку и теряет его после закрытия дела. Ограничение отправки составляет 30 сообщений за минуту. При истёкшей подписке подтверждённый пользователь сохраняет доступ к уже созданному диалогу.

Согласование возврата вызывает lf_reserve_found. Функция проверяет автора находки, незавершённые торги и доступность вещи, принимает одну заявку, резервирует объявление и создаёт передачу. Отмена принятой обычной

заявки согласованно снимает резерв, пока ни одна сторона не подтвердила передачу. Конкурирующая заявка не может получить вторую принятую передачу той же вещи.

Для завершения вызывается lf_confirm_transfer отдельно от каждой стороны. Первое подтверждение сохраняется, но не завершает возврат. Второе переводит заявку, вещь и передачу в завершённое состояние. Уже согласованный возврат можно закончить после истечения подписки. Спор разрешается через lf_resolve_return с полномочиями независимого модератора и обязательным обоснованием.

18.4. Допуск к аукциону и торги

AuctionService принимает заявку на продажу с основанием и ключом документа. Проверку выполняет модератор. Ограничения второго этапа запрещают продажу документов, банковских карт и вещей организации-хранителя. Для основания six_months требуется истечение шести месяцев с официального сообщения о находке, а не с публикации объявления. Самостоятельного автоматического признания права собственности в приложении нет.

После одобрения допуска продавец задаёт начальную цену, шаг и окно торгов. Ставки передаются только в lf_place_bid. PostgreSQL проверяет статус торгов, серверное время, подписку, продавца, текущий максимум и шаг. Java не формирует альтернативную вставку в lf_bids. Новая заявка владельца приостанавливает аукцион; заблокированный участник также приводит к приостановке связанных торгов.

Завершение отдельного лота вызывает lf_finalize_auction. Фоновое пакетное завершение вызывает lf_close_due_auctions. Приостановленные торги не закрываются автоматически. Без ставок вещь сохраняется в каталоге. Для победителя PL/pgSQL создаёт принятое дело с auction_id, диалог и передачу; вещь переходит через auctioned в reserved. Повтор завершения не создаёт вторую передачу. Продавец и победитель используют обычные два подтверждения; истёкшая подписка этому не препятствует. Односторонняя отмена результата торгов и отмена уже подтверждённой одной стороной

передачи запрещены; спор разбирает модератор. Оплата купленного лота не входит в предметную область курсовой.

18.5. Жалобы, администрирование и события

AdministrationService принимает жалобы, проверяет независимость модератора, блокирует пользователей и управляет служебными данными. Обычный модератор не может блокировать сотрудника без роли admin. Блокировка и изменение роли требуют причины. Администратор назначает и отзывает роли, создаёт и отключает тарифы, обновляет справочник организаций. Отзыв роли последнего действующего администратора запрещён; проверка выполняется под блокировкой записи роли и повторной проверкой полномочий инициатора. Аудит доступен только служебным ролям с ограниченным размером страницы. Для проверки профилей, жалоб, изменения ролей, тарифов и справочников сервисы дополняют триггерный аудит явными записями в той же транзакции.

OutboxService выбирает недоставленные события с FOR UPDATE SKIP LOCKED. Уведомления и delivered_at сохраняются в одной транзакции. Сообщение уведомляет другую сторону без копирования частного текста. Окончание последнего периода подписки создаёт одно устойчивое событие; активное продление исключает ложное уведомление. Решение по профилю и жалобе видно получателю в кабинете, в том числе до одобрения профиля. Событие активации подписки помечается доставленным без повторной вставки: исходное уведомление уже создаёт функция оплаты. SMTP-доставка с собственным повтором и внешняя рассылка ещё не подключены.

Таблица 20 — Соответствие прецедентов первого этапа реализации

Прецедент	Реализованные методы	Граница третьего этапа
UC-01 Получить доступ	register, confirmEmail, submitProfile, reviewProfile, authenticate, createOrder, payDemo, periods; восстановление доступа	Решение профиля и оплата в БД; HTTP и внешняя почта относятся к следующему этапу
UC-02 Опубликовать объявление	create, edit, attachImage, putPrivateAttribute, submit, moderate, archive	Автор готовит материал, независимый модератор публикует; двоичный адаптер — следующий этап

Прецедент	Реализованные методы	Граница третьего этапа
UC-03 Вернуть вещь	search, searchByState, detail, claim, send, messages, reserve, reject, cancel, confirm, organizations	Владелец и нашедший взаимодействуют через одно дело; хранитель-организация показывается как альтернатива
UC-04 Разрешить спор	complain, messages, send, reviewComplaint, resolve, block	Доступ сотрудника к диалогу только при открытой жалобе, независимое решение
UC-05 Продать вещь	requestPermission, reviewPermission, create, bid, finalizeAuction, closeDue, confirm	Допуск, конкурирующие ставки, итог PL/pgSQL и передача победителю
UC-06 Изменить полномочия	grantRole, revokeRole, audit	Актуальные права, обязательная причина и защита последнего администратора
UC-07 Актуализировать инструкцию	updateOrganization, organizations, audit	Администратор обновляет источник, участник получает справочную запись
UC-08 Изменить тариф	tariff, retireTariff, createOrder, payDemo, audit	Новый заказ получает новую цену; старые заказы и периоды сохраняют условия

19. Проверка и демонстрация на helios

19.1. Изолированные интеграционные проверки

Проверки выполняются с настоящим PostgreSQL на helios. Java-команда verify модуля tooling создаёт временный набор lf3check_* из SQL второго этапа в схеме s465826, заполняет его синтетическими данными и запускает Gradle test. Java подключается через локальный SSH-туннель; пароль базы передаётся в окружение дочернего процесса и не сохраняется в исходных файлах. Отдельная команда verify-db проверяет целостность и конкуренцию на наборе lfcheck_*.

Имена таблиц, функций, триггеров, индексов и последовательностей меняются согласованно. Проверочный набор не использует lf_* и объекты другой лабораторной. Перед запуском проверяется отсутствие lf3check_*. Если они уже существуют, скрипт останавливается и не удаляет их автоматически. После выполнения используются штатный безопасный drop.sql, проверка каталога объектов и сравнение значений исходных последовательностей.

ServiceIntegrationTest выполняется с откатом каждого сценария. TransactionIntegrationTest проверяет реальные коммиты, параллельные соединения и завершение аукциона по серверному времени. Все коммиты происходят внутри одноразового набора, который затем удаляется целиком. SqlNamesTest проверяет допустимые идентификаторы. Встроенная H2 и мок-реализация SQL-функций не используются.

Таблица 21 — Группы проверок третьего этапа

Группа	**Что проверяется**
Аккаунты	BCrypt, регистрация, одноразовый токен, модерация, восстановление и отзыв сессии
Подписки	Повтор заказа и события, продление, чужой заказ, откат ошибочной оплаты
Объявления	Публикация, новая модерация после правки, время, собственник, метаданные изображений
Приватность	Явная публичная карточка, закрытые признаки и доступ к диалогу
Возврат	Один резерв, отмена, два подтверждения, истечение подписки, решение модератора
Торги	Повтор ставки, смена содержимого ключа, продавец, заблокированный участник, приостановка, победитель
Администрирование	Полномочия, жалобы, тарифы, роли, уведомления и аудит
События	Доставка outbox и отсутствие дубликатов при повторе
Конкуренция	Два резервирования одной вещи и два подтверждения одного платежа
Изоляция	Сохранность исходных объектов, последовательностей и удаление проверочного набора

Актуальный общий прогон содержит 113 тестов, без ошибок и пропусков. Из них 57 проверяют хранение, предметные сервисы и идентификаторы SQL, а 56 — веб-уровень и файловый адаптер четвёртого этапа. Компиляция и bootJar завершены успешно. Подробные протоколы находятся в docs/part3/validation/integration.txt и isolation.txt; краткий результат по тестовым классам — в tests.json. Протокол запуска собранного приложения на helios сохранён в helios.txt, контрольная сумма установленного JAR — в deployment.txt.

Дополнительно повторно выполнена проверка базы второго этапа: 50 условий целостности, конкурирующие резервы и ставки, повтор оплаты, пакетное завершение и удаление только проверочного набора. Результат сохранён в database-regression.txt. Основные объекты курсовой и лабораторной сохранены.

19.2. Сборка и запуск

Локальная сборка требует JDK 17 и доступа к Maven Central. Gradle устанавливается через Wrapper; контрольная сумма дистрибутива задана в gradle-wrapper.properties.

```
cd D:\IS\course\backend
$env:JAVA_HOME='C:\Users\minec\.jdks\corretto-17.0.14'
.\gradlew.bat test bootJar
```

Без RUN_DB_TESTS интеграционные проверки пропускаются. Для полного прогона используется следующая Java-команда; SSH-пароль запрашивается скрытым вводом:

```
cd D:\IS\course
& "$env:JAVA_HOME\bin\java.exe" -Xmx256m `
-jar tooling/build/libs/course-tools.jar verify
```

На helios JAR расположен только в каталоге курсовой. Launcher использует DB_URL, DB_USER и DB_PASSWORD, а при отсутствии пароля читает подходящую запись .pgpass. Глобальные настройки пользователя и файлы другой лабораторной не меняются.

```
cd ~/poteryashki-course
java -Xmx256m -jar tooling/build/libs/course-tools.jar launch --demo
```

Для JVM заданы -Xms64m и -Xmx256m, поскольку автоматический выбор максимальной памяти на helios превышает доступный объём. Приложение запускается на установленном OpenJDK 21, а байткод собран для Java 17. Демонстрация читает синтетический аккаунт нашедшего, выполняет поиск функцией PL/pgSQL, показывает контакты метрополитена и число уведомлений.

На старте Hibernate проверяет соответствие 27 сущностей существующей схеме в режиме ddl-auto=validate. Приложение не создаёт и не удаляет таблицы, не использует общий flyway_schema_history лабораторной и

не запускает миграции другой работы. Скрипты DDL второго этапа остаются источником структуры базы.

19.3. Границы реализации

Третий этап завершает хранение и бизнес-логику. Веб-страницы, контроллеры Spring MVC, выпуск JWT, защищённые Cookie, HTTP-ограничение попыток входа, SMTP и двоичный MinIO-адаптер относятся к последующему этапу. Доступ к Java-сервисам предполагается из доверенного транспортного слоя: он должен получать пользователя из проверенной сессии, а не из произвольного параметра запроса. Отдельного публичного endpoint с передачей actorId нет.

Проверки третьего этапа охватывают доступ к базе, переходы состояний, транзакции, права участников и доставку внутренних уведомлений. Измерение производительности интерфейса и испытания резервного копирования, HTTPS и внешних интеграций в этот этап не входят.

20. Результаты третьего этапа

Разработаны диаграммы классов, 27 отображений таблиц, репозитории и семь предметных сервисов с общим контролем доступа. Критичные процессы используют функции и процедуру PL/pgSQL второго этапа. Работа Java-кода проверена на настоящем PostgreSQL, включая конкуренцию и откат ошибок. Собранный JAR запускается на helios в отдельном каталоге курсовой.

Источники третьего этапа

1. Пример структуры Java-проекта и отчёта: MyFilmList, фиксированный коммит.

2. Spring Boot 3.5: требования к среде.

3. Spring Framework: JdbcTemplate и обработка SQL.

4. Spring Framework: совместная работа JPA и JDBC.

5. PostgreSQL: блокировки и конкуренция.

Задание четвёртого этапа

1. Реализовать уровень представления для бизнес-процессов первого этапа.

2. Сформировать итоговый отчёт, включающий предыдущие этапы.

3. Подготовить и представить проект.

21. Уровень представления

21.1. Технологии и структура

Веб-интерфейс реализован на Java 17, Spring MVC и Thymeleaf. Браузер получает готовые HTML-страницы; формы отправляют запросы на контроллеры приложения. Для оформления используется локальный файл CSS. Отдельная сборка frontend и прикладной JavaScript не требуются. Серверный вариант интерфейса сохраняет выбранный на предыдущих этапах стек Spring Boot, Hibernate/JPA, PostgreSQL, Lombok и Gradle.

Контроллеры сгруппированы по процессам: AccountPages обслуживает доступ к аккаунту; MainPages — личный кабинет, подписку, уведомления и справочник; ListingPages — каталог и объявления; ReturnPages — заявки и диалоги; AuctionPages — допуски, лоты и ставки; StaffPages — служебные операции. MediaPages выдаёт вложения после проверки полномочий. WebQueries формирует проекции для шаблонов без паролей, хешей токенов и закрытых примет чужих вещей.

Контроллер передаёт идентификатор участника из проверенного WebIdentity. Идентификатор действующего пользователя не принимается из формы. Предметные операции выполняют существующие сервисы третьего этапа. Поиск, резервирование, подтверждение передачи, активация подписки, ставка и завершение торгов продолжают использовать функции и процедуры PostgreSQL второго этапа.

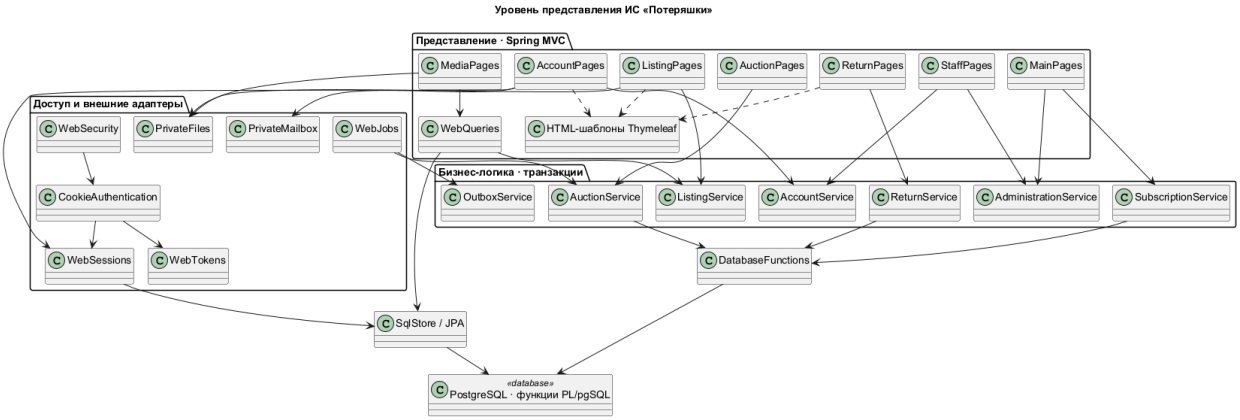


Рисунок 15 — Архитектура уровня представления

21.2. Страницы и прецеденты

Таблица 24 — Соответствие интерфейса прецедентам

Прецедент	**Страница и действие**
UC-01. Получить доступ	Участник: /auth/register, /auth/confirm, /account, /subscriptions; модератор: очередь профилей /staff; оплата и доступ к каталогу. Вход и восстановление — /auth/login, /auth/forgot, /auth/reset
UC-02. Опубликовать объявление	Автор: /listings/new, фото, закрытые приметы и отправка; модератор: /staff?tab=listings, просмотр и решение; другие участники: опубликованная карточка
UC-03. Вернуть вещь	Владелец: /catalog, карточка и заявка; нашедший: /claims, проверка и резерв; обе стороны: /conversations/{id} и два подтверждения. Хранитель-организация — инструкция /metro
UC-04. Разрешить спор	Участник: жалоба из карточки или диалога и решение в кабинете; модератор: очередь жалоб, доступные материалы и служебное решение
UC-05. Продать вещь	Продавец: допуск и лот /auctions; модератор: проверка основания; покупатели: /auctions/{id}, ставки; WebJobs: итог; продавец и победитель: /claims и подтверждения
UC-06. Изменить полномочия	Администратор: роли в /admin; сотрудник: доступ либо отказ на служебной странице после изменения; история — /staff?tab=audit
UC-07. Актуализировать инструкцию	Администратор: организация в /admin; участник: обновлённые контакты, источник и дата на /metro
UC-08. Изменить тариф	Администратор: создание и закрытие тарифа в /admin; участник: новые условия заказа в /subscriptions; старые заказы сохраняют цену

Регистрация и личный кабинет доступны до подписки. Для каталога, публикации и участия в торгах нужны подтверждённая почта, проверенный

профиль и действующая подписка. Чтение собственного диалога и завершение согласованной передачи сохраняются после окончания оплаченного периода. Служебные операции требуют роли модератора или администратора.

21.3. Организация интерфейса

Главная страница объясняет порядок возврата вещи и открывает поиск либо создание объявления. Основная навигация содержит каталог, инструкцию метро и личный кабинет. Для авторизованного пользователя боковое меню показывает объявления, заявки, подписку и письма. Служебные пункты появляются при соответствующей роли.

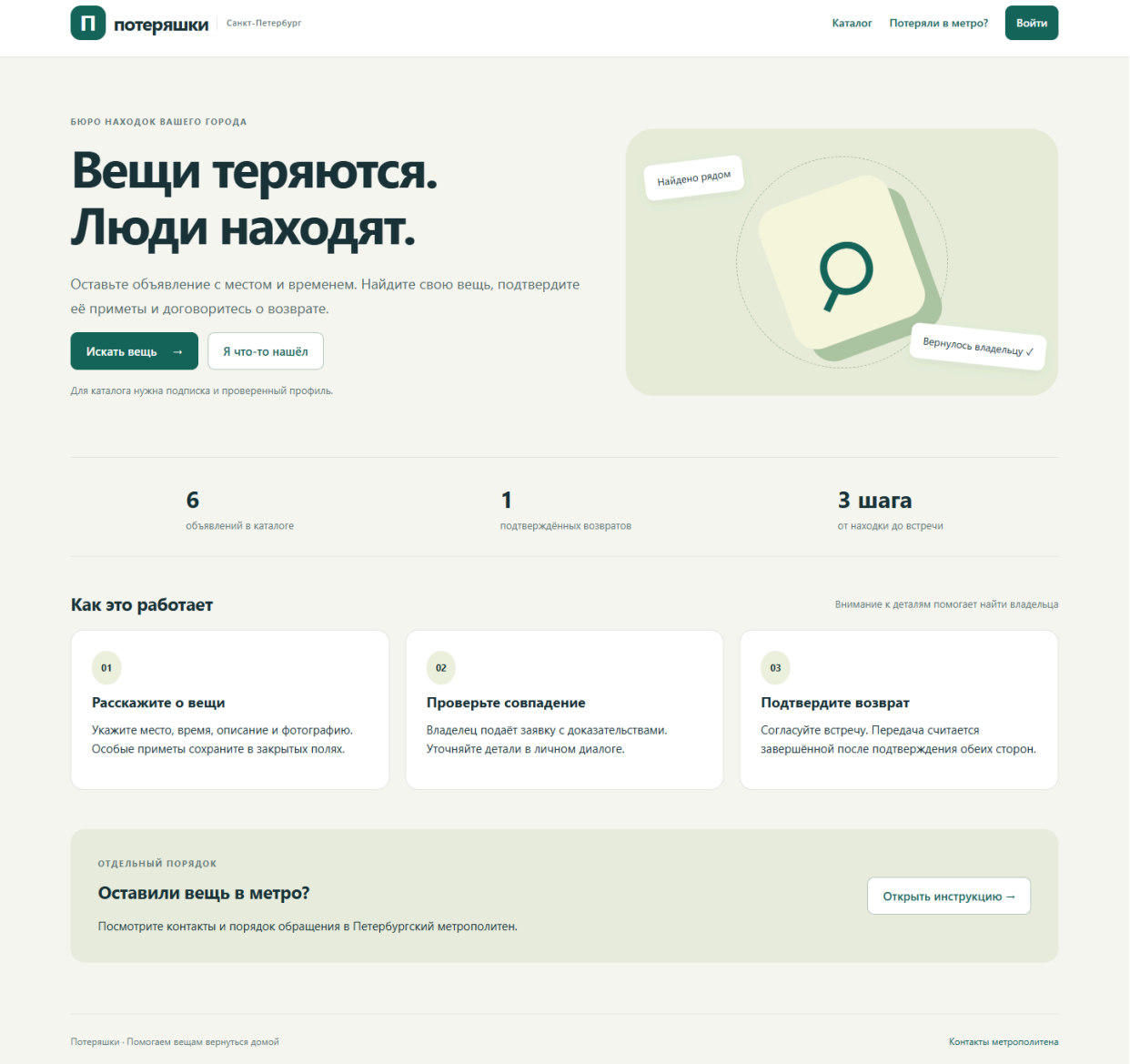


Рисунок 16 — Главная страница

В каталоге фильтры передаются параметрами GET, поэтому результат можно открыть по сохранённой ссылке. Следующая страница использует идентификатор последней записи как курсор и сохраняет фильтры. При отсутствии совпадений выводится пояснение с предложением изменить условия.

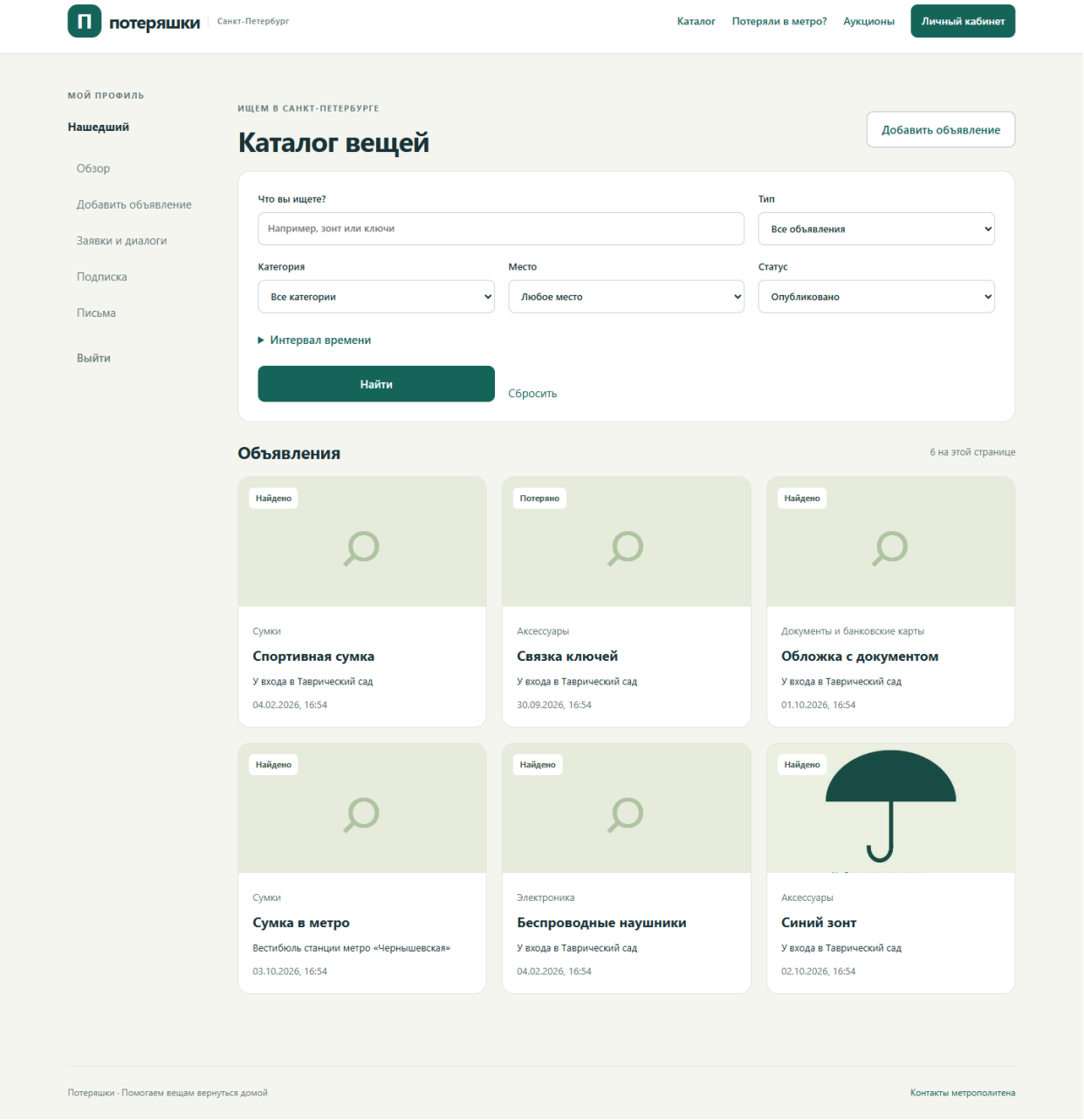


Рисунок 17 — Каталог объявлений

Редактор разделяет публичное описание и закрытые признаки. После сохранения автор может загрузить фотографии и отправить черновик на модерацию. Изменение опубликованного объявления требует повторной

проверки согласно правилам предыдущего этапа. Дата и время вводятся по часовому поясу Europe/Moscow.

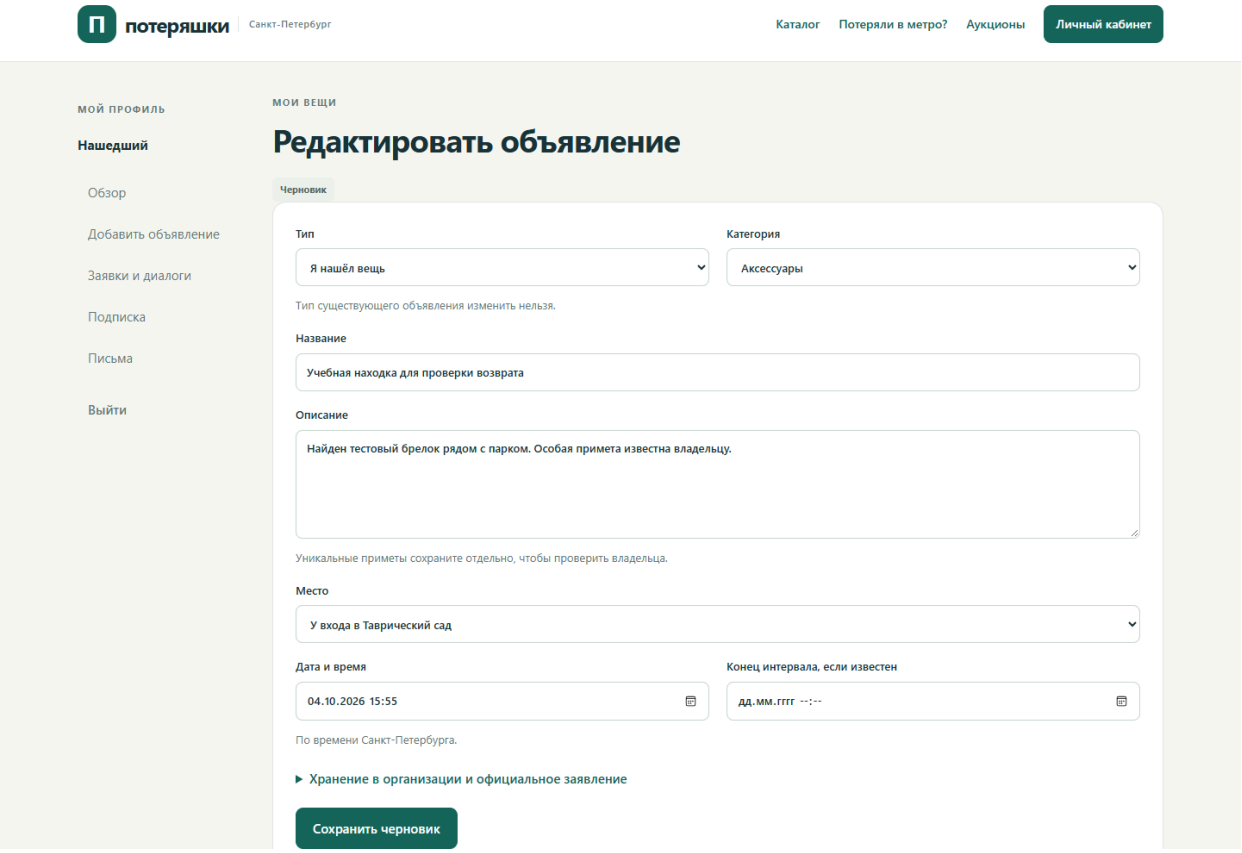


Рисунок 18 — Редактор объявления

Модератор рассматривает отдельные очереди. Для проверки профиля, допуска к продаже и разрешения спора вводится мотивированное решение. Проверка не позволяет сотруднику одобрить собственное объявление. Изменение ролей и тарифов доступно администратору.

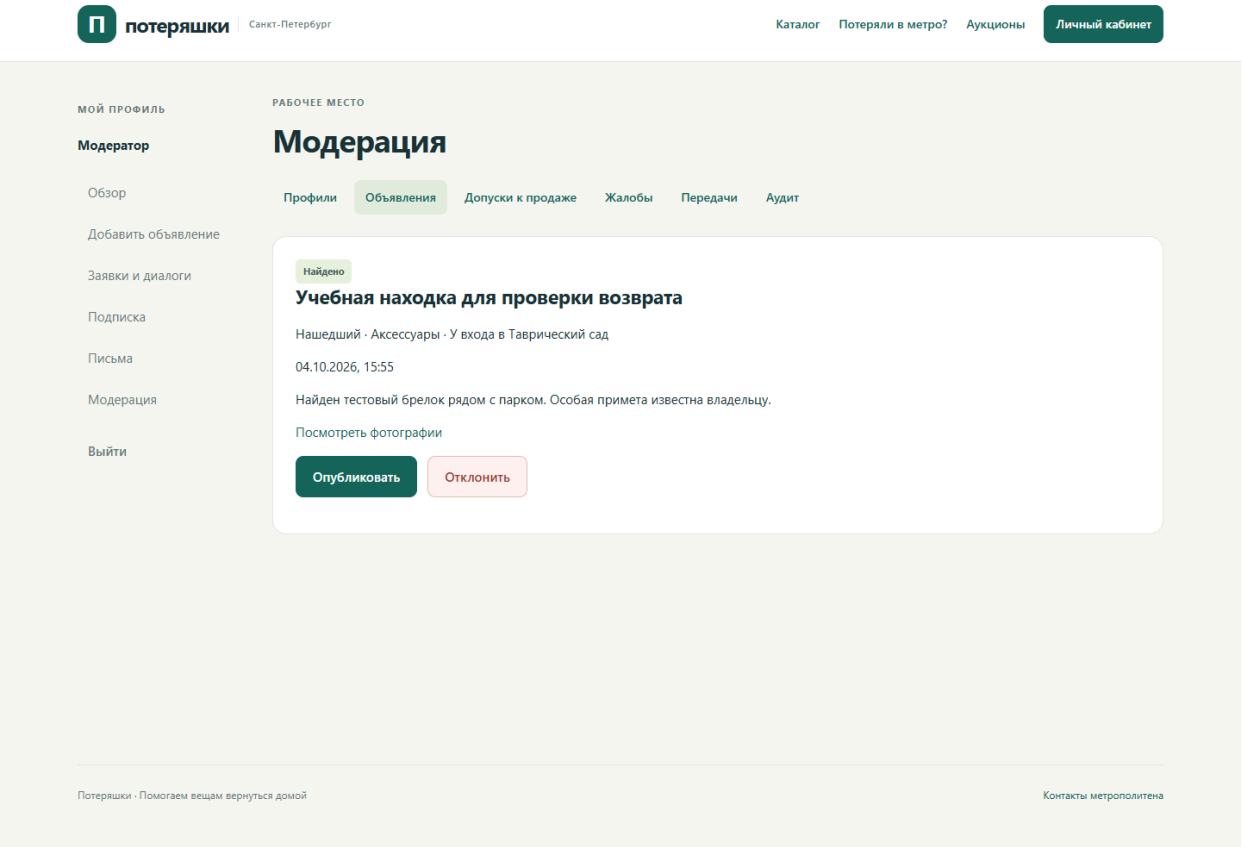


Рисунок 19 — Очередь объявлений модератора

Диалог доступен участникам заявки. Доказательства владения не выводятся в публичной карточке. После резервирования карточка заявки показывает два независимых подтверждения. Одно нажатие не завершает передачу; окончательное состояние устанавливает функция базы данных.

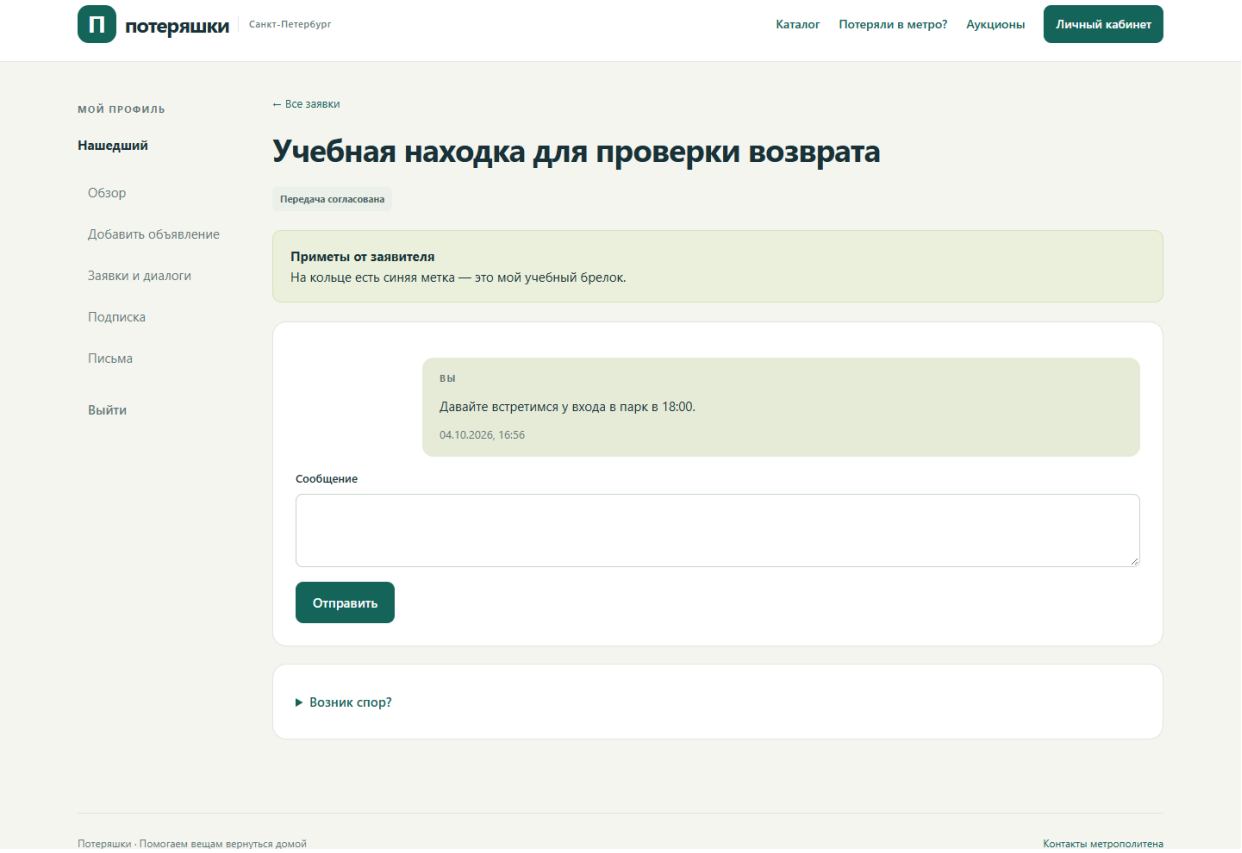


Рисунок 20 — Диалог о возврате

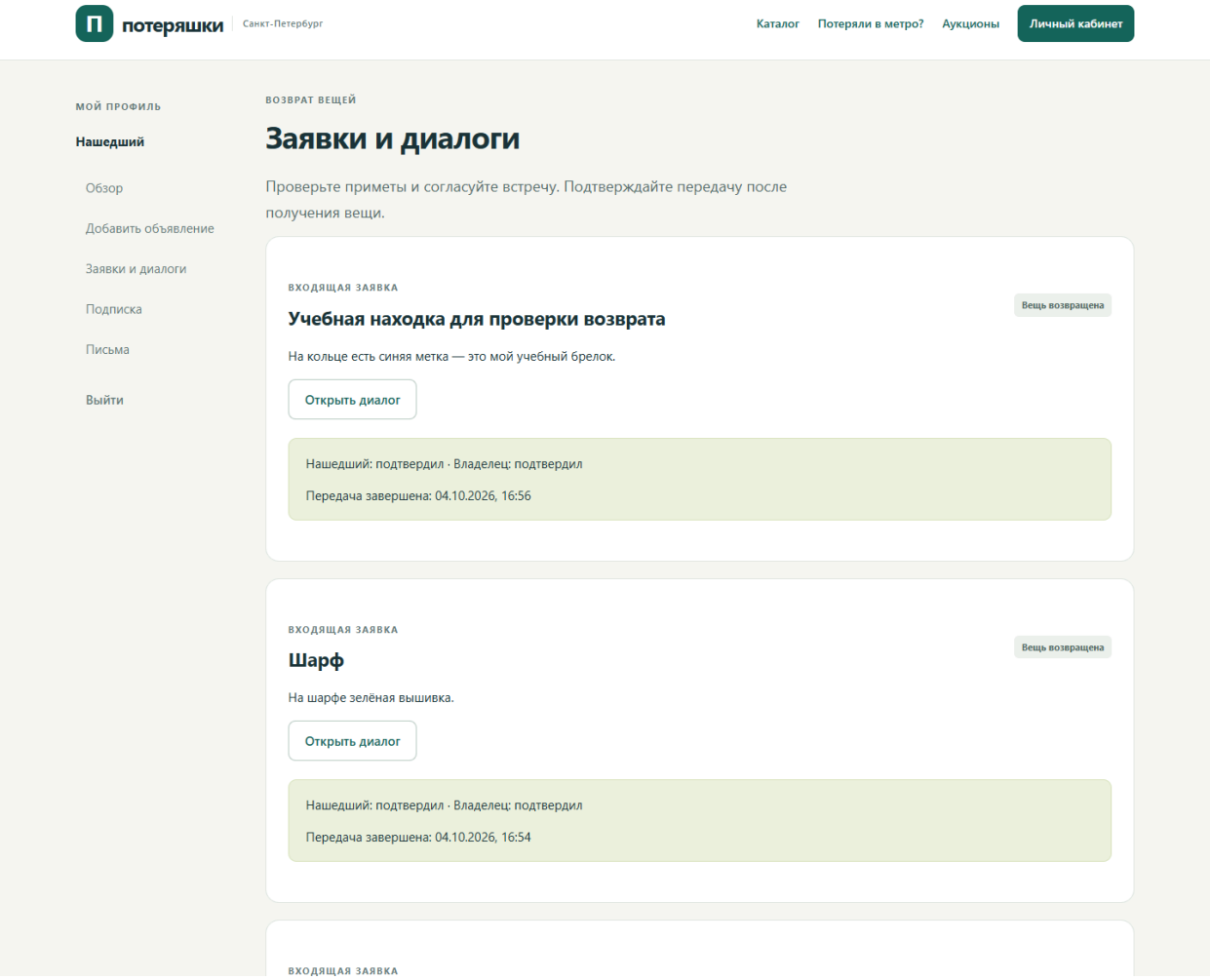


Рисунок 21 — Подтверждённая передача

Карточка аукциона показывает начальную цену, текущую ставку, шаг, начало и окончание торгов. Форма не показывается продавцу. Сервер повторно проверяет ограничения при отправке запроса; сохранённая или изменённая вручную форма не обходит проверки PostgreSQL.

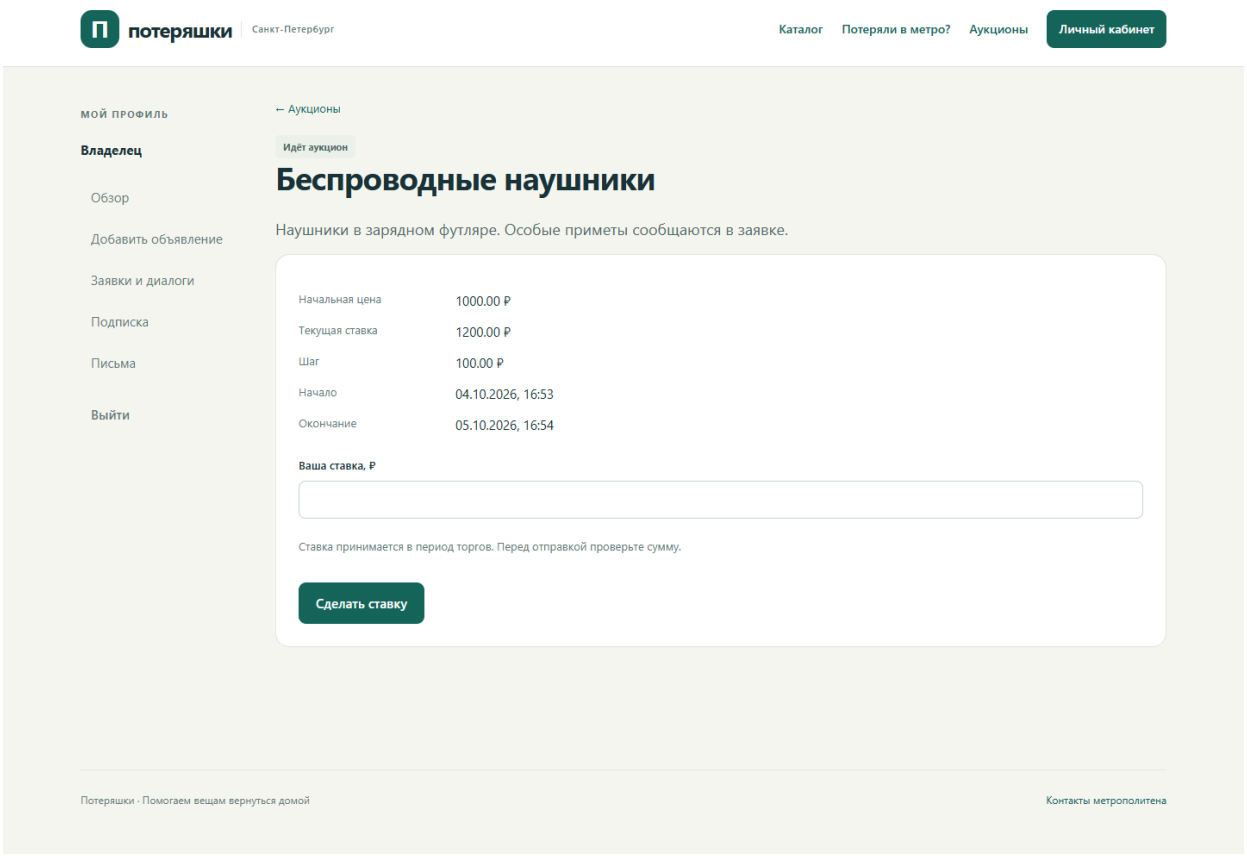


Рисунок 22 — Карточка аукциона

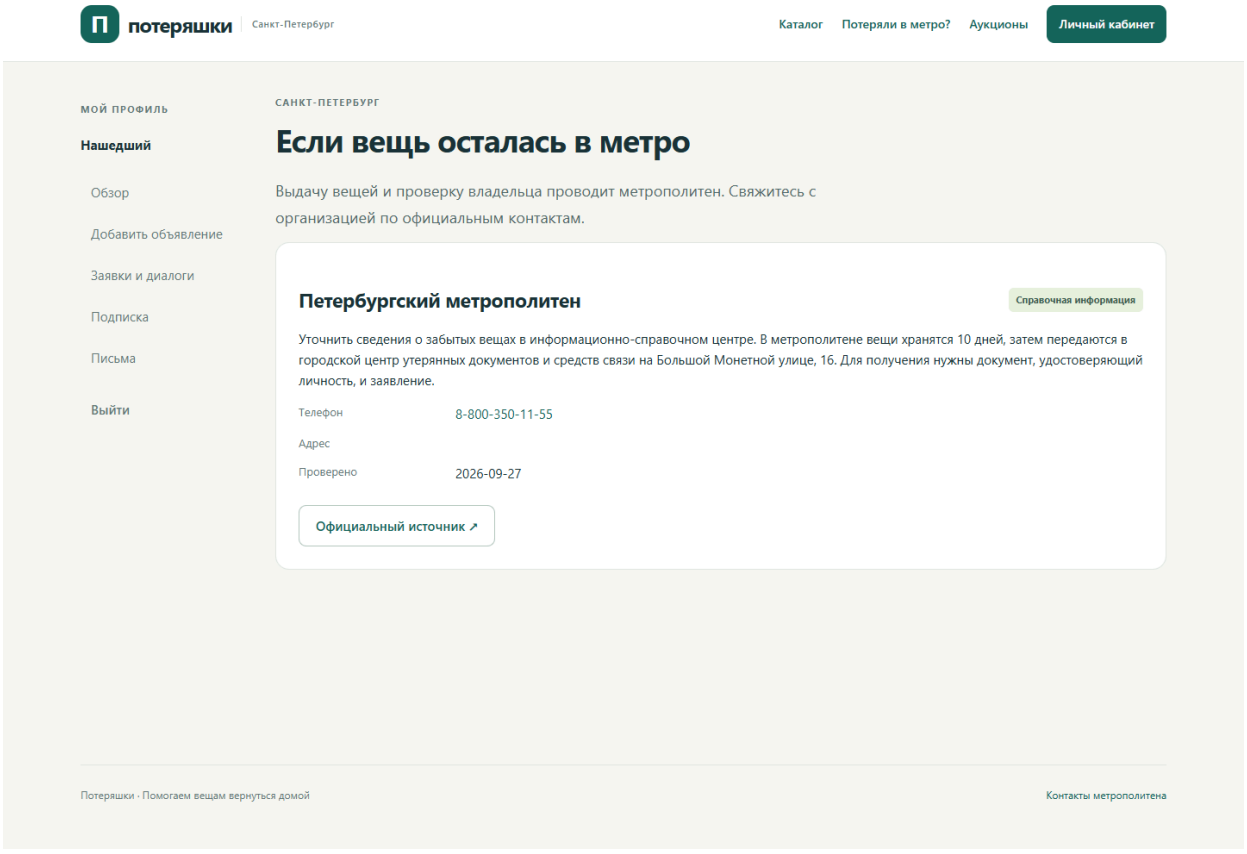


Рисунок 23 — Инструкция по обращению в метрополитен

Администратор видит пользователей и роли, управляет новыми тарифами и обновляет официальные контакты. Закрытие тарифа не меняет условия уже созданных заказов. При отзыве роли проверка доступа учитывает актуальные данные базы.

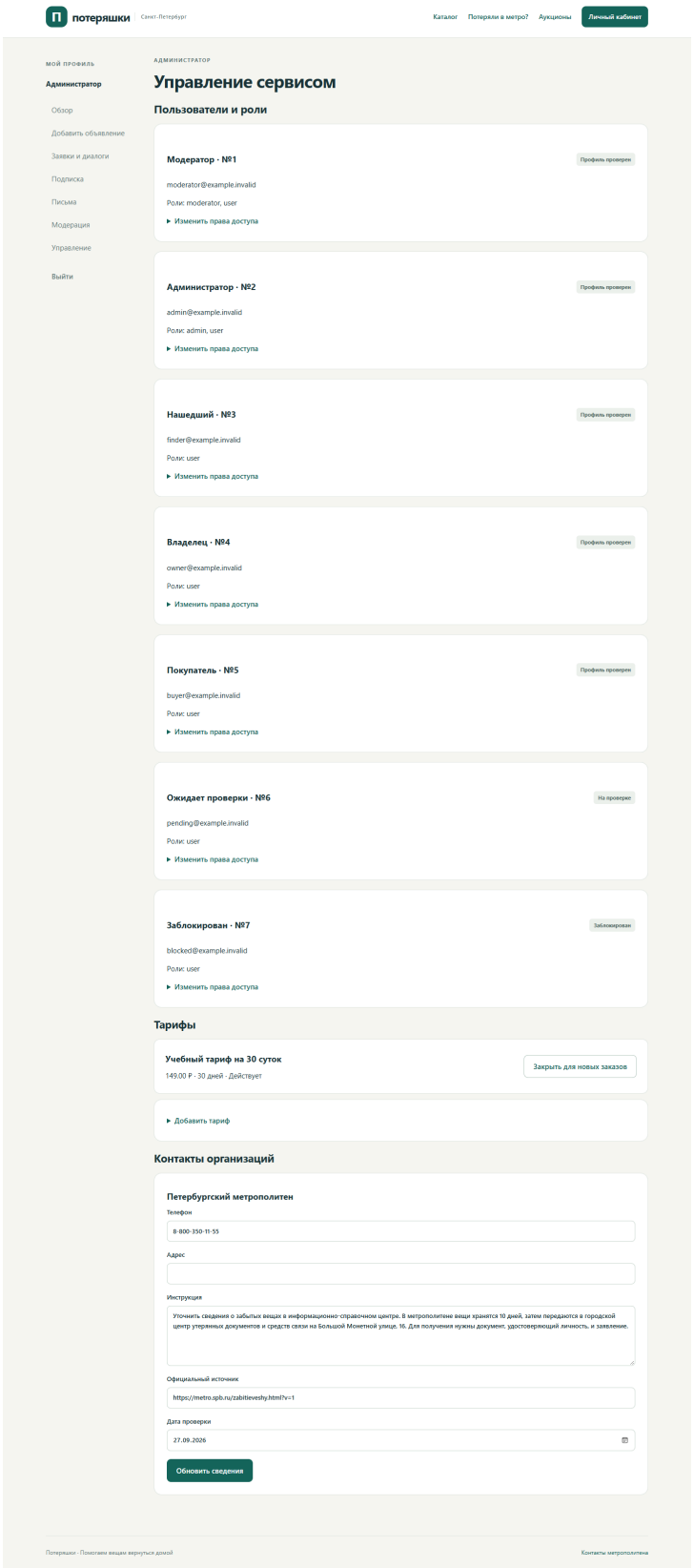


Рисунок 24 — Управление сервисом

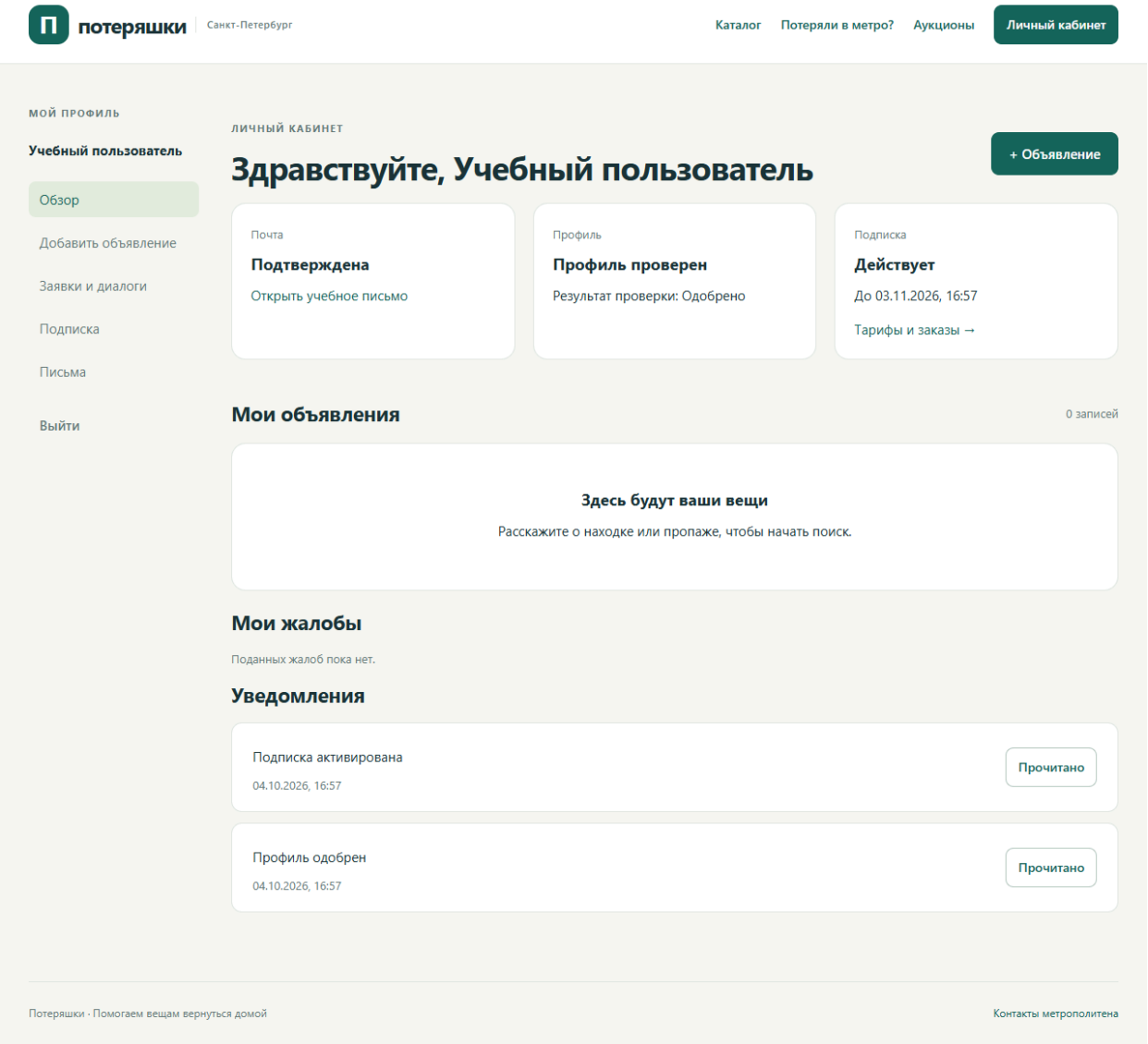


Рисунок 25 — Личный кабинет после учебной оплаты

При ширине 390 пикселей карточки и формы выстраиваются в одну колонку, меню переносится на несколько строк. Поля имеют подписи и видимый фокус; минимальная высота полей ввода и основных кнопок — 44 пикселя. Проверка на этой ширине не заменяет проверку всех устройств и средств доступности.

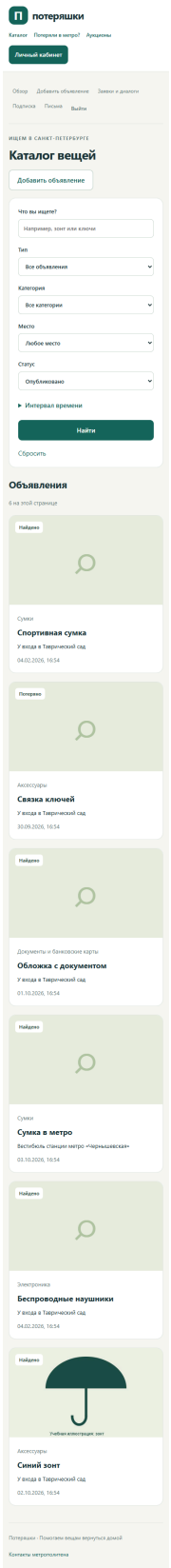


Рисунок 26 — Каталог на узком экране

22. Доступ и обработка данных

22.1. Сеансы и защита форм

WebSessions проверяет пароль BCrypt и создаёт случайный refresh-токен. В PostgreSQL сохраняется только его SHA-256. Access JWT действует 15 минут; refresh-токен — 30 суток. Оба передаются в HttpOnly cookie с SameSite=Lax. На каждом запросе проверяются версия токена пользователя, блокировка, срок и отзыв сеанса, текущие роли. Обновление атомарно отзывает прежний refresh-токен. Выход и смена пароля прекращают действие ранее выпущенных сеансов.

Spring Security проверяет CSRF-токен изменяющих запросов. Thymeleaf добавляет его к формам POST. Политика CSP запрещает скрипты, внешние стили и встраивание приложения во фрейм. Referrer-Policy не передаёт адрес страницы с одноразовым токеном сторонним сайтам. В публичном размещении требуются HTTPS и COOKIE_SECURE=true; учебный запуск выполняется на loopback через SSH-туннель. Секрет JWT можно задать переменной JWT_SECRET; при отсутствии создаётся случайный ключ на время запуска.

Пять неудачных попыток входа на один адрес в 15-минутном окне блокируют дальнейшие попытки до окончания окна. Счётчик находится в памяти одного процесса, поэтому перезапуск его сбрасывает; распределённая защита входа в этой версии не реализована.

22.2. Почта и вложения

MAIL_MODE=demo сохраняет учебные письма в памяти и показывает их только авторизованному получателю. При регистрации новый пользователь входит в собственный кабинет и открывает подтверждение. Публичный запрос восстановления всегда возвращает одинаковый результат и не раскрывает одноразовый токен. Полноценное восстановление забытого пароля без действующего входа требует MAIL_MODE=smtp и настроенного SMTP-сервера; учебный ящик для этого не заменяет внешнюю почту.

SMTP-адаптер использует STARTTLS и получает реквизиты из окружения; подключение к внешнему почтовому серверу не проверялось. Учебные письма исчезают после перезапуска. Платёжный адаптер работает

без списания средств, что указано на кнопке оплаты. Сумма берётся из сохранённого заказа.

PrivateFiles хранит файлы вне каталога статических ресурсов. При загрузке фотографий проверяются сигнатура JPEG/PNG, размер до 5 МБ и разрешение до 16 миллионов пикселей. Изображение декодируется и сохраняется заново без исходных метаданных. Имя формируется из UUID; путь проверяется на выход из корневой папки и символические ссылки. При отказе базы новый файл удаляется.

Подтверждение допуска принимает PDF либо изображение. PDF проверяется по сигнатуре и выдаётся как скачиваемое вложение, а не встроенная страница. Антивирусная проверка и полная проверка структуры PDF не реализованы. Документ доступен заявителю и модератору. Частные ответы не кешируются. В тестовом наборе старые ключи demo/ соответствуют явно помеченным учебным вложениям из закрытых ресурсов приложения; они не являются юридическими документами.

Файлы хранятся в закрытом локальном каталоге. MinIO не подключён; механизм восстановления файлов и автоматическая уборка объектов после аварийного завершения процесса не реализованы.

22.3. Фоновые операции

WebJobs каждые 30 секунд вызывает существующую процедуру закрытия истёкших аукционов и доставляет события Outbox во внутренние уведомления. Действия выполняются через предметные сервисы. Ошибка оставляет событие для следующего прохода. Планировщик отключается в интеграционных проверках, чтобы тестовые сценарии не изменялись параллельно.

Завершение торгов создаёт в PL/pgSQL одну заявку победителя, диалог и передачу. Победитель и продавец видят дело в /claims и отдельно подтверждают фактическую передачу. Отмена после первого подтверждения и односторонняя отмена результата аукциона запрещены. Жалоба из диалога относится к находке и доступна участнику при истёкшей подписке; модератор теряет доступ к чужому диалогу после закрытия дела. Получатель сообщения, автор проверенного профиля и автор рассмотренной жалобы получают внутренние

уведомления. Истёкший последний период подписки уведомляет один раз, без ложного события при активном продлении.

23. Проверка веб-приложения

23.1. Условия проверки

JUnit и MockMvc проверяют реальные сервисы и PostgreSQL через SSH-туннель. Создаются временные таблицы lf3check_; уже существующий набор с этим префиксом не принимается. После проверки набор удаляется, сравниваются исходные объекты схемы и значения остальных последовательностей. Пароли подключения не записываются в отчёт и репозиторий.

Браузерная проверка выполняется Playwright for Java в установленном Microsoft Edge. Запускается отдельный процесс приложения на свободном loopback-порту. Профили участников изолированы разными контекстами браузера. Используются тестовые адреса example.invalid и учебная оплата. Снимки в этом отчёте получены из работающего приложения.

Таблица 25 — Проверяемые группы сценариев

Группа	Проверка
Страницы	Главная, формы доступа, кабинет, каталог, редактор, заявки, аукционы и служебные очереди
Полномочия	Анонимный доступ, неоплаченный профиль, чужой редактор и диалог, служебные URL
Сеансы	HttpOnly/SameSite, отзыв при выходе и смене пароля, ротация refresh, актуальные роли
Формы	Отказ без CSRF, экранирование HTML, отсутствие секретов в публичных проекциях
Бизнес-процессы	Ставка через MVC и функцию БД; передача победителю; подтверждение при истёкшей подписке; запрет отмены после подтверждения
Жалобы и уведомления	Собственное решение в кабинете; доступ сотрудника только при открытой жалобе; уведомления без раскрытия текста диалога; окончание подписки без повторов
Файлы	Реальный PNG, повреждённые PNG/JPEG, подмена MIME/имени, SVG, размер и выход из каталога

Группа	Проверка
Браузер	Создание и фото, публикация, заявка, диалог, два подтверждения, жалоба и её рассмотрение, ставка, регистрация и подписка
Узкий экран	Каталог при ширине 390 пикселей и отсутствие горизонтального переполнения

Пройдены 113 тестов приложения и 8 тестов Java-инструментов. Протоколы находятся в docs/part4/validation и docs/part3/validation; результаты по классам тестов приведены в tests.json, журналы прогонов — в integration.txt и browser.txt. Нагрузочные испытания, проверка RPO/RTO и испытания внешних почтового и платёжного сервисов не проводились.

24. Демонстрация и итоговые материалы

Приложение размещается в собственном каталоге poteryashki-course на helios, подключается к таблицам lf_ схемы s465826 базы studs. JVM ограничена 64–256 МиБ. Веб-сервер слушает только 127.0.0.1:18081. Для доступа с рабочей машины используется SSH-туннель. Другая лабораторная не включается в состав установки.

Итоговый отчёт этапов 1–4 находится в docs/part1-4/report.md и report.pdf. К защите подготовлены презентация в PDF и HTML и сценарий демонстрации в presentation/defense.md.

Реализованы страницы для всех групп прецедентов первого этапа. Уровень представления использует существующие транзакции и функции БД. Система позволяет пройти путь от регистрации и проверки профиля до публикации, возврата вещи либо продажи с проверенным основанием. Учебные адаптеры и непроверенные эксплуатационные требования указаны отдельно.

В текущей версии отсутствуют автоматическое резервное копирование и восстановление, обезличивание профиля и единый идентификатор HTTP-запроса. Интерфейс проверен в Edge на компьютере и при ширине 390 px; испытания в Chrome и Firefox не проводились. Подключение MinIO и платёжного провайдера остаётся задачей дальнейшей разработки.

Источники четвёртого этапа

1. Spring Security. CSRF: https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html.

2. Spring Security. JWT: https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html.

3. Thymeleaf. Spring integration: https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html.

4. Playwright for Java. Установка и работа с браузером: https://playwright.dev/java/docs/intro; https://playwright.dev/java/docs/browsers.